Université du Littoral – Côte d'Opale – Calais
Unité d'ouverture, licence MSPI et SV, 2me et 3me année
Licence MSPI non-spécialisée, 1re année S1
Département de physique

Enseignement autour de micro-contrôleur *Arduino Uno*

Robin Bocquet, montages éléctroniques, rédaction de l'énoncé 2017-2020
Arnaud Cuisset, carnet des charges et gestion du projet 2017-2019
Pierre Kulinski, réalisateur montages Arduino, soutien technique, Dk 2017
Pascal Masselin, montages éléctroniques, intervention en TP L1 S1, Calais 2021
Wilfried Montagnier, soutien technique, Calais
Dmitrií Sadovskií *, développement et traitement informatique, rédaction de l'énoncé

Calais, automne 2025

1 Introduction

Le but de ces travaux pratiques est de vous initier à l'utilisation des microcontroleurs qui ont littéralement envahis le monde technologique d'aujourd'hui. Un microcontrôleur n'est rien d'autre qu'un microprocesseur à jeu d'instructions limité spécialisé dans les communications avec l'extérieur. On retrouve ce type de composants dans les voitures par exemple où ils gèrent l'ensemble des capteurs et éléments de sécurité du véhicule, dans les drones où ils gèrent les capteurs de vitesse, d'altitude, de lacet, roulis et tangage, dans les robots où bien souvent des radars anti-collisions sont mis en place ou bien encore dans les imprimantes 3D où les moteurs ainsi que les positionnements sont gérés par ce type de composant. Les microcontrôleurs ont été développés dans les années 80 mais ont réellement diffusés dans le grand public depuis 2005 grâce au travail d'un groupe d'italiens dans le monde du logiciel libre qui a développé une plateforme logiciel simplifiant l'utilisation de ces composants. Il s'agit des développements connus sous la bannière ARDUINO et repris maintenant sous le nom GENUINO, adhérant à la charte du développement «libre». L'ensemble des documentations sur la carte ARDUINO, ainsi qu'une mine d'exemples sont consultables sur son site. Enfin, dans cette introduction, notez que vous trouvez nombre de tutoriaux sur le « net » pour vous aider à découvrir le matériel et le logiciel.

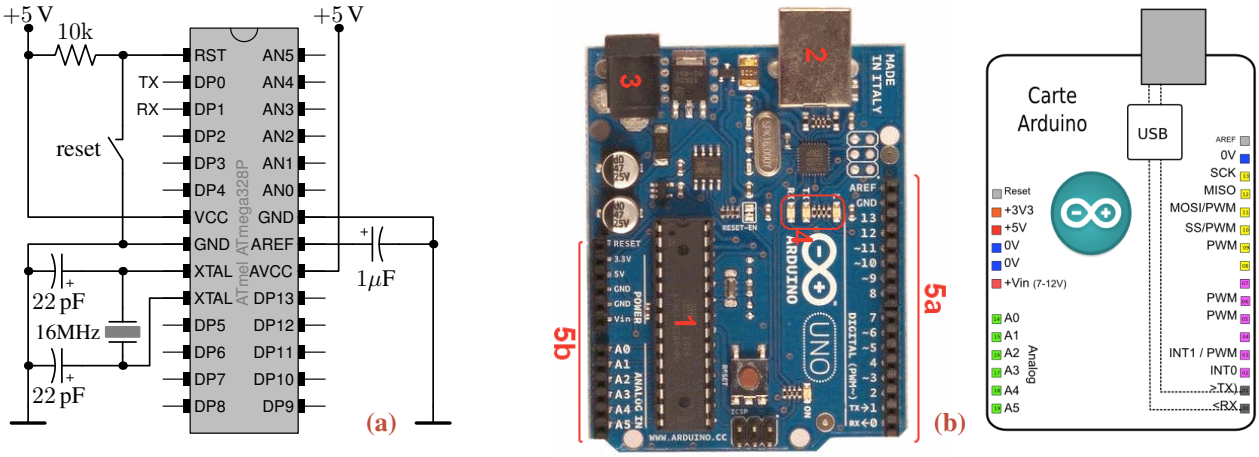


FIG. 1 – Brochage principal du microcontrôleur ATMEGA328P-PU (a) et carte ARDUINO UNO (b) où on distingue (1) microcontrôleur ATMEGA328, (2) connecteur USB pour relier l'ordinateur, (3) jack de connection d'une alimentation extérieure (≤12V DC), (4) diodes de fonctionnement de la carte dont TX et RX pour visualiser le passage de données dans le port série, et (5) connecteurs d'entrée/sortie pour «le monde extérieur», avec (5a) digitales DP_0–DP_{13} et masse GND fonctionnant en niveaux TTL 5V et (5b) analogiques $A0$–$A5$ accompagnés par les broches de sortie d'alimentation (3.3 V stabilisée, 5 V, et la masse GND) dont la VIN permet également entrer une alimentation extérieure de 5 V DC à la carte.

* sadovski@univ-littoral.fr, responsable du projet BQE HTLC (Hight Tech Low Cost) ULCO 2017

High performance, low power AVR® 8-bit microcontroller

- Advanced RISC architecture
  - 131 powerful instructions, most single clock cycle execution
  - 32×8 general purpose working registers
  - fully static operation
  - Up to 20 MIPS Throughput at 20 MHz
  - On-chip 2-cycle Multiplier
- High endurance non-volatile memory segments
  - 4/8/16/32K Bytes of in-system self-programmable flash program memory
  - 256/512/512/1K bytes EEPROM
  - 512/1K/1K/2K bytes internal SRAM
  - Write/Erase cycles : 10,000 Flash/100,000 EEPROM
  - Data retention : 20 years at 85°C/100 years at 25°C (1)
  - Optional Boot Code Section with Independent Lock Bits
  - In-System Programming by On-chip Boot Program
  - True Read-While-Write Operation
  - Programming Lock for Software Security
- Operating Voltage : 1.8–5.5V
- Temperature Range : −40°C à 85°C
- Speed grade : 0–4 MHz@1.8–5.5 V, 0–10 MHz@2.7–5.5.V, 0–20 MHz@4.5–5.5 V
- Power consumption at 1 MHz, 1.8 V, 25°C
  - Active Mode : 0.2 mA
  - Power-down Mode : 0.1 μA
  - Power-save Mode : 0.75 μA (Including 32 kHz RTC)
- Peripheral Features
  - Two 8-bit timer/counters with separate prescaler and compare mode
  - One 16-bit timer/counter with separate prescaler, compare mode, and capture mode
  - Real Time Counter with separate oscillator
  - Six PWM channels
  - 8-channel 10-bit ADC in TQFP and QFN/MLF package temperature measurement
  - 6-channel 10-bit ADC in PDIP package temperature measurement
  - Programmable serial USART
  - Master/Slave SPI serial interface
  - Byte-oriented 2-wire serial interface (PHILIPS I2C compatible)
  - Programmable watchdog timer with separate on-chip oscillator
  - On-chip analog comparator AC
  - Interrupt and wake-up on pin change
- Special microcontroller features
  - Power-on Reset and Programmable Brown-out Detection
  - Internal Calibrated Oscillator
  - External and Internal Interrupt Sources
  - Six Sleep Modes : Idle, ADC Noise Reduction, Power-save, Power-down, Standby, and Extended Standby
- I/O and packages
  - 23 Programmable I/O Lines
  - 28-pin PDIP, 32-lead TQFP, 28-pad QFN/MLF and 32-pad QFN/MLF

TAB. 1 – Caractéristiques générales du microcontôleur ATMEGA328P

2 Initiation à ARDUINO

Nous utiliserons le microcontôleur[1] ATMEGA328 dont le brochage est montré dans la figure 1a. Ses caractéristiques générales se trouve sur le site du son fabricant ATMEL. Rassurez vous cependant, que vous n'aurez à utiliser le MCU seul que dans les phases finales d'intégration de vos développements, lors des projets libres, si vous en avez le temps. Vous utiliserez la plupart du temps le MCU dans son environnement de développement ARDUINO. La documentation complète et les explications des différentes broches et entrées sorties sont données sur le site de ATMEL.

D'ores et déjà vous pouvez noter (table 1) l'existence d'un port de communication série (RXD et TXD, broches 2 et 3), de convertisseurs analogiques-digitaux (broches ADCi), d'un bus I^2C (SDA et SDC) très utilisé avec les capteurs, d'une interface série synchrone dénommée SPI (MISO, MOSI et SCK) et de broches dénommées digitales à collecteur ouvert. Enfin vous disposez d'une mémoire flash d'une capacité de 32 kO pour stocker votre programme de manière permanente mais, bien évidemment, effaçable. Notez que les broches digitales sur la carte ARDUINO (fig. 1, centre, 5b) sont des niveaux TTL 5 V, ce qui signifie, que le niveau logique 1 (HIGH) correspond à une tension comprise entre 2.4 et 5 V, et que le niveau logique bas (LOW) correspond à une tension comprise entre 0 et 1.4 V, voir la fig. 3. Chaque sortie numérique est limitée en courant à 20 mA.

2.1 Programmation de ARDUINO



Talk is cheap. Show me the code.
Linus Torvalds

Le MCU ARDUINO est programmé en C++ dans un environnement spécialement dédié (sec. 2.1.1). Nous allons couvrir les bases de cette programmation et de l'utilisation de cet environnement en développant un petit code qui nous permettra de façon évolutive :

[1] on utilisera certaines formes abréviative anglaises devenues très courantes, voir sec. J. Ainsi on désigne microcontôleur comme MCU, μCU, ou μC.

1. brancher et reconnaître votre carte, faire clignoter une LED propre à Arduino (Uno)
2. modifier le code pour faire clignoter le signal SOS avec cette LED
3. allumer les 4 LEDs externes une par une en séquence dans le `setup`
4. passer des messages via port série/usb (aka «Terminal» ou «Console»)
5. communiquer avec l'Arduino via son Terminal par les touches à 1 caractère
6. interpréter les touches, retourner leur code ASCII
7. pour les touches 0..9, a..f, ou A..F les interpréter comme des chiffres hexadécimaux
8. donner la valeur 0..16 du chiffre correspondant
9. donner sa représentation binaire (en 4 bits), utiliser opérations «bitshift» et «bitmask»
10. allumer les 4 LEDs externes (voir point 3) selon cette représentation binaire
11. (option) interpréter l'entrée analogique d'un potentiomètre pour allumer les 4 LEDs selon la tension obtenue (aka «barographe»)

2.1.1 Installer et utiliser le logiciel ARDUINO (IDE)

Un microcontrôleur ARDUINO est en fait très sommaire avec beaucoup moins de possibilités qu'un ordinateur. Pour s'en convaincre, il suffit de regarder sa quantité de mémoire. Aussi, sa programmation se fait séparément sur un ordinateur (*offline*) et le programme est ensuite transféré dans le MCU pour son exécution. Pour cela, on utilise un environnement de développement intégré, le «desktop IDE» (basé sur l'interface sous JAVA). Pour son installation sous Linux, voir `https://www.arduino.cc/en/Guide/Linux`.

```
> tar xvf arduino-1.8.3-linux32.tar.xz
> ln -s arduino-1.8.3/ arduino
> cd arduino
```

Taper la commande `arduino` ouvre la fenêtre de l'IDE (sous Java).

2.1.2 Brancher de la carte ARDUINO à l'ordinateur

Vous disposez de

- un ordinateur sur lequel le programme de développement ARDUINO est installé
- une carte ARDUINO UNO ou équivalent avec son câble USB idoine
- une plaquette d'essais électronique à trous pour les montages électroniques
- des fils dénudés à chaque bout ainsi que les composants électroniques pour les montages envisagés

Lorsque l'on insère le câble USB (type B coté carte, type A coté ordi), on observe en utilisant la commande

```
crw-rw---- 1 root dialout 188,   0 Aug  3 10:51 ttyUSB0
crwx-w---- 1 dima tty       4,   1 Aug  3 12:30 tty1
crw------- 1 root root    189, 513 Aug  3 12:48 usbdev5.2
crw-rw---- 1 root dialout 166,   0 Aug  3 12:48 ttyACM0
```

que les liaisons série–USB sont reconnus par l'ordinateur et les interfaces usb `ttyUSB0` et série `ttyACM0` sont établis. Pour communiquer avec l'ARDIUNO, à partir de l'IDE, nous sélectionnons le port série `ttyACM0` via TOOLS ▷ SERIAL PORT. Ensuite, nous pouvons voir avec TOOLS ▷ GET BOARD INFO que la carte ARDUINO est bien reconnu :

```
BN: Arduino/Genuino Uno
VID: 2A03
PID: 0043
SN: 95536333830351807052
```

Enfin, dans les OUTILS ▷ TYPES DE CARTES on choisit/confirme la carte GENUINO UNO.

Langage de programmation

Il est basé sur `c++`, consulter le site de ARDUINO pour les informations sur ses principales fonctions. A noter la syntaxe `c,c++` de base, tel que la terminaison obligatoire de ligne par `;` et les commentaires. Tout codage ARDUINO présente la même structure :

- les définitions des constantes, des fonctions, et des variables qui seront utilisées dans tout le programme
- la partie `setup(){...}` qui n'est exécutée qu'une seule fois au tout début du programme.
- le programme principal `loop(){...}` qui est exécuté dans une boucle infinie. Repérez sur votre carte le bouton de redémarrage, dit «reboot», qui permet d'interrompre cette boucle et de revenir au `setup(){...}`.

2.1.3 Programme Blink

Nous allons tester le matériel à disposition en faisant l'expérience d'allumage de la diode électroluminescente (LED) de la carte ARDUINO. Parmi les exemples de codage les plus basiques, le programme se trouve dans FILE ▷ EXAMPLES ▷ BASICS ▷ BLINK

```
/* Blink: Turns on/off the onboard LED on for 200 msec */

void setup() {                    // runs once when you press reset or power on
  pinMode(LED_BUILTIN, OUTPUT);   // initialize digital pin LED_BUILTIN
}

void loop() {                     // runs over and over again forever
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (set HIGH voltage level)
  delay(200);                     // wait for a 200 milliseconds
  digitalWrite(LED_BUILTIN, LOW);  // turn the LED off (set the voltage LOW)
  delay(200);
}
```

On charge ce programme FILE ▷ OPEN dans le logiciel IDE. Pour compiler et vérifier ce programme, nous utilisons SKETCH ▷ COMPILE AND VERIFY

```
Sketch uses 928 bytes (2%) of program storage space. Maximum is 32256 bytes.
Global variables use 9 bytes (0%) of dynamic memory, leaving 2039 bytes for local variables. Maximum is 2048
    bytes.
```

et pour le faire tourner on passe par COMPILE-UPLOAD (téléverser le programme dans la carte) ou on utilise le raccourci clavier CTRL-U. La LED de la carte doit clignoter à une fréquence de $1/(2 \times 0,200)$ Hz confirmant le fonctionnement de la carte et du logiciel IDE. Vous êtes alors prêts à utiliser l'environnement ARDUINO.

2.1.4 Programme BlinkSOS

On modifie le programme précédant pour faire clignoter «SOS» en morse. Ce signal de détresse consiste de trois appels courts (lettre S) suivis par trois appels longues (lettre O) puis encore 3 courts. Pour cela, on essaie d'utiliser une boucle **`for`** :

```
 for(i=3; i==0; i--) {
    digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
    delay(200);                        // wait for a 200 milliseconds
    digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
    delay(200);
  }
  delay(200);
```

ici `i` est déclarée auparavant comme une variable du type **`int`** par une commande du type **`int`** `i;` placée soit avant le `setup` (déclaration globale), soit au debut de `loop` (locale). L'instruction **`for`** réalise une boucle et prend trois arguments separés par point-virgule. Ici `i = 3` initialise la valeur de compteur `i` à 3 et la condition `i == 0` permets de continuer la boucle tant que `x` reste positif. La dernière partie est exécutée à la fin de la boucle, apres les instructions groupées par `{ }`. Ici `i--` décrémente la valeur de `i` par pas de 1. En fait, l'instruction `i--` est équivalente à `i = i-1`, mais se fasse plus efficacement en etant une operation de processeur, et prend moins de place en mémoire.

Pour coder «SOS», il nous faudra trois boucles **`for`** successives. Alternativement, en plus de `loop` et `setup`, nous pouvons définir une petite *fonction* qui effectuera cette boucle avec le delay variable demandé comme son parametre.

```
void blink(unsigned char n, int d, int dd, unsigned char pin=LED_BUILTIN) {
  while(n--) {                  // blink n times
    digitalWrite(pin, HIGH);    // turn the LED on (HIGH is the voltage level)
    delay(d);                   // wait for d ms
    digitalWrite(pin, LOW);     // turn the LED off by making the voltage LOW
    delay(d);                   // wait for d ms
  }
  delay(dd);                    // mark the end of sequence by extra dd msec
}
```

Maintenant, dans le `loop`, notre code se simplifie à

```
  blink(3, 200, 250);           // letter S
  blink(3, 500, 250);           // letter O
  blink(3, 200, 500);           // letter S and final pause
```

2.1.5 Montage d'une LED

Les pins digitaux (DP's), initiés (dans le `setup`) comme `OUTPUT`, peuvent servir à passer et à commuter les tensions de 5 (V_{CC}) et 0 V (GND) signifiant les niveaux logiques de «oui» et «non», respectivement (voir la sec. 2.1.7). Ainsi notre pin `LED_BUILTIN`, étant déjà

initié pour cette utilisation, peut commuter une LED[2] externe. On branche celle-ci (sec. 2.1.6) par son anode sur le pin 13 correspondant à `LED_BUILTIN` de ARDUINO UNO, et une résistance de 220 Ω entre son cathode et GND. Le code reste inchangé, mais au présent, la LED externe double les clignotements de la LED sur la carte.

2.1.6 Remarque : Connexion de LED's

Pour ceux qui n'ont pas suivi d'enseignement d'électronique, une LED est une diode qui s'éclaire lorsqu'un courant la parcourt. Elle sert beaucoup pour visualiser de façon très simple l'état logique d'une ligne (haut/bas ou vrai/faux). La diode est un dipôle asymétrique constitué d'une anode et d'une cathode. Lorsque elle est polarisée dans le sens direct (+V sur l'anode et 0 V sur la cathode), elle est passante, et un courant circule. Lorsqu'elle est polarisée en inverse, elle est bloquée, et aucun courant ne circule. La figure 2a donne les caractéristiques courant-tension de LED de diverses couleur. Vous remarquerez que lorsque la tension à leur bornes est inférieure à $\sim 1,5$ V, le courant est nul : la diode est bloquée. Il faut que la tension dépasse une certaine valeur appelée «tension de coude» pour que le courant puisse passer. Typiquement, pour les LED que vous avez en TP, la tension de coude est de 1,8 V. Au delà de cette valeur, une petite variation de la tension induit une forte variation du courant.

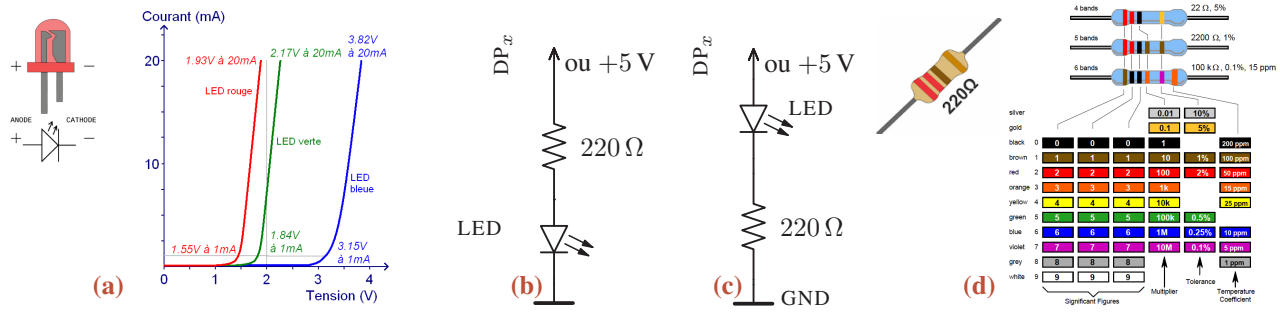


FIG. 2 – Caractéristiques courant-tension de diodes électroluminescentes (LEDs) dont la cathode est repérée par la patte plus courte (a), et leur montage de base biaisé par une résistance de type *pullup* (b) et *pulldown* (c). Les résistances sont marguées par des bandes colorées (d).

Afin de connecter une LED à un pin digital DP$_x$ de ARDUINO ($x = 2..13$, tension de sortie V_{CC} en mode `OUTPUT`), on place une résistance R en série avec la LED pour limiter le courant à 25 mA, cf la sec. 2.2.1. Calculons la valeur de R. La tension d'alimentation V_{CC} est la somme de tensions V_R aux bornes de la résistance et V_{LED} aux bornes de la LED (fig. 2b). Une LED rouge typique doit avoir une différence de potentiel V_{LED} (dit tension directe) de 1.6 à 1.8 V entre ses pins pour un courant, approximativement, de 25 mA. Donc

$$V_R = V_{CC} - V_{LED} = 5\,\text{V} - 1.8\,\text{V} = 3.2\,\text{V}.$$

Par la loi d'Ohm, nous obtenons pour $I = 25$mA une résistance de $R = V_R/I = 128\,\Omega$. Cependant, les tutoriels recommandent d'utiliser 220 Ω. Pourquoi ? La raison est dans la pratique : en regardant les résistances disponibles couramment, on trouve une de 100 Ω. Inférieure à 128 Ω, que nous avons besoin, celle-là est risquée parceque le courant $I = 3.2/100 = 32$ mA serait alors trop important. La valeur usuelle suivante est de 220 Ω, ce qui nous donne un courant $I = 3.2/220$ d'environs 14 mA. Les DELs sont des composants non-linéaires, opérationnelles pour des courants de 10–25 mA, et la différence de courant entre 14 et 25 mA n'est pas nécessairement proportionnelle à la différence de luminosité. Dans la plupart des cas, on ne sera même pas capable d'apercevoir cette différence.

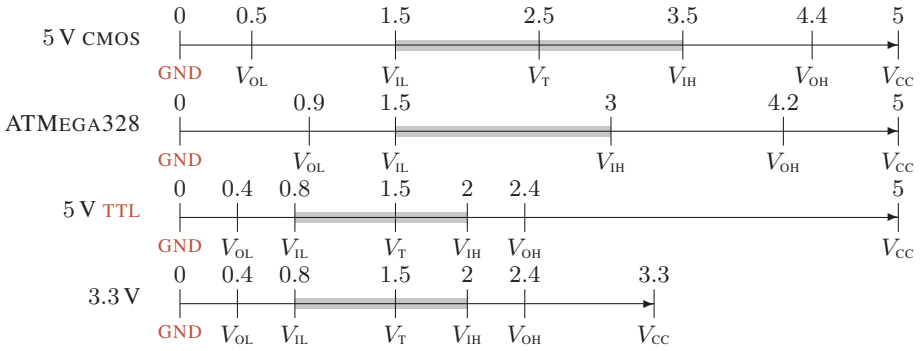


FIG. 3 – Niveaux logiques

2.1.7 Remarque : Niveaux logiques de ARDUINO

En regardant le datasheet de l'MCU ATMEGA328 de la carte ARDUINO UNO, on peut conclure que les niveaux de tension sont légèrement différents de standard CMOS (voir la fig. 3). Ceci fait la plateforme ARDUINO, avec son intervalle des tensions invalides 1.5..3.0 V

[2] L'abréviation omniprésente anglaise LED désigne une diode électroluminescente parfois abrégé DEL en français

raccourcie coté V_{IH}, un peu plus robuste, offrant une marge plus large pour la bruit, et possédant une tolérance élargie pour les signaux d'entrée `HIGH`. En conséquent, la conception des interfaces et les connexions aux autres équipements sont simplifiées.

2.1.8 Capteur de lumière comme interrupteur

Les pins digitaux peuvent servir également à lire les signaux logiques à leur entrée après être déclarés (dans le `setup`) en mode `INPUT`. Comme indiquée dans la sec. 2.1.7, les plages de voltage que ARDUINO tolère comme «oui» et «non» (aka `HIGH` et `LOW`) sont assez larges. Nous allons exploiter ces plages pour brancher une photorésistance (LDR), un capteur de lumière décrit dans la sec. 3.2. Considérez le branchement en série d'une résistance ohmique R coté V_{CC} et une LDR coté masse, avec l'entrée DP_2 correspondant au point entre R et LDR de resistance r. La tension d'entrée v sur DP_2 égale à la tension au bornes de la LDR,

$$v = \frac{r}{r+R} V_{CC}.$$

La résistance r de la LDR dépende de l'intensité de la lumière qu'elle reçoit. Elle varie entre $r = 100\,\text{K}$ dans le noir, et 2 à 5 K en plein jour. Par conséquent, si on choisit $R = 10\,\text{K}$, le signal v variera entre 4.5 V et 0.8 à 1.6 V (1.3 V pour $r = 3.5\,\text{K}$). En regardant les niveaux logiques dans la fig. 3, on conclut que dans ce montage, DP_2 donne «oui» dans le noir, et «non» si la LDR reste bien illuminée. Maintenant, nous pouvons réagir aux niveaux de luminosité. Les changements dans notre code sont minimes. On initialise le pin DP_2

```
#define pin_LDR 2                       // photo-resistor pin
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);         // for onboard LED
  pinMode(pin_LDR, INPUT);              // for LDR state
}
```

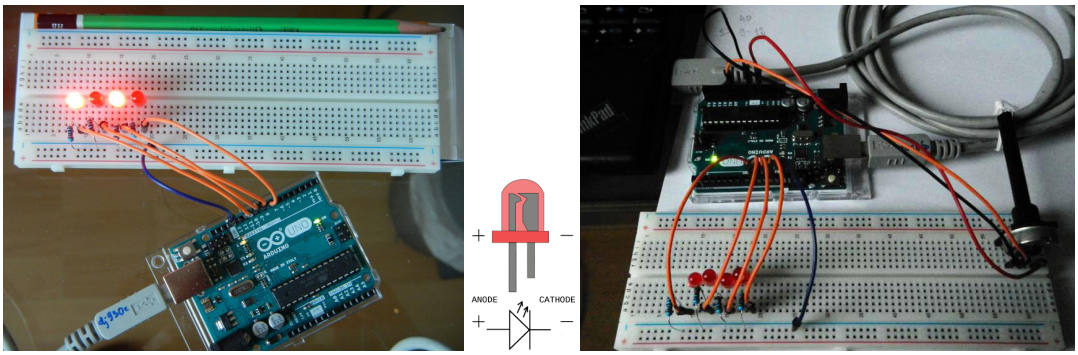
et il suffit de modifier une seule ligne dans notre fonction `blink` dans la sec. 2.1.4

```
    if(digitalRead(pin_LDR))    // if no ambient light is present
      digitalWrite(pin, HIGH); // turn the LED on (HIGH is the voltage level)
```

pour permettre les clignotements «SOS» seulement en absence de la lumière ambiante (ou la LDR reste couverte).

2.2 Montage à 4 LED's

Par la suite, on monte quatres LED comme montré sur la figure ci-dessous (les résistances entre les cathodes de LEDs et GND sont de 220 Ω). Dans la partie `setup()`, nous pouvons initialiser les quatre sorties digitales avec une petite boucle :




```
  for(j=PIN_BASE,i=4; i--; pinMode(j++, OUTPUT));
```

où l'utilisation des opérations `++` et `--` est à noter. Et par la suite, nous pouvons aussi allumer nos LEDs une par une

```
for(j=PIN_BASE,i=4; i--; j++) {
   digitalWrite(j, HIGH);   // turn the LED on (HIGH is the voltage level)
   delay(200);              // wait for a 200 milliseconds
   digitalWrite(j, LOW);    // turn the LED off by making the voltage LOW
   delay(200);
 }
```

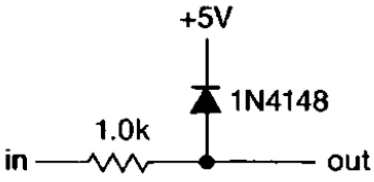
pour tester notre montage.

Câblage : dans les montages de prototypage on utilise les cavaliers avec les embouts DUPONT M/M ou M/F, ou tout simplement les fils de diamètre 0.51 mm, section $0.2\,\text{mm}^2$, et résistance linéique 0.084 Ω/m, connu aussi comme $1\times$AWG24.

2.2.1 Remarque : Le courant maximal qu'on peut tirer des pins digitaux de ARDUINO en mode OUTPUT

La carte ARDUINO est conçue pour être alimentée de deux manières (sec. 2.3) : par son port USB ou par son connecteur externe. Dans le cas de USB, la limite maximale de courant tiré est de ≈500 mA. Pour une alimentation externe qui utilise le régulateur $+5\,V_{DC}$ de la carte, la limite est plutôt fixée par la dissipation de chaleur de ce régulateur et en conséquence, est déterminée par la valeur spécifique de la tension sur le connecteur d'alimentation externe. Une limite de 300 mA sera certainement correcte pour les deux sources. Une autre limitation est le courant maximal de 40 mA que les broches d'entrée/sortie (E/S) peuvent gérer individuellement, et la consommation totale de courant par toutes ces broches ensemble limitée à 200 mA. C'est la dernière contrainte à laquelle vous vous heurtez le plus souvent, et il est préférable de faire fonctionner les broches d'E/S en toute sécurité à environ de 20 mA maximum.

2.2.2 Remarque : protection des pins digitaux de ARDUINO en mode INPUT



Considérez le cas où on réagit au changement d'une tension d'entrée U_{IN} supérieure à la tension maximale V_{CC} (de 5 V pour les UNO) qu'est tolérée[3] sur les pins digitaux DP_i en mode INPUT (voir la sec. 2.1.7). Dans ce cas, au plus simple, on limite U_{IN} par un circuit dit «clamping» (voir ci à gauche). Notons qu'en effet, la carte ARDUINO UNO integre déja ce type de protection qui suffit à combler une surtension de courte durée, telle qu'une ESD. Toutefois, pour des U_{IN} élevées, une durée plus longue, et/ou pour une protection plus réactive et immédiate, on peut ajouter en exterieur une diode rapide *Schottky* (une diode qui possède un seuil de tension directe très bas, de 0.2 à 0.3 V) et, afin de limiter le courant, une résistance de shunt en série de quelques kΩ's selon la valeur de $U_{IN} - V_{CC}$. Ce montage protège la DP_i si le μCU est en marche, aussi bien qu'à son démarrage, quand l'état de ses DPs n'est pas établi, et même lorsque le μCU n'est pas alimenté (e.g., pendant un «changement des piles»). En ce moment, les DPs reste «suspendues» et la surtention $U_{IN} \gg V_{CC} = 0$ sur elles présente un danger immédiat pour le μCU. Notons qu'une DP peut également être protégée par une diode *Zener* de $V_Z = 5$ ou 5.1 V montée en stabiliseur/référence de tension. Cette protection ne s'applique qu'en mode «marche» de l'μCU. Par ailleurs, elle nécessite des courants plus élevés (de ≈5 mA) sur le shunt pour atteindre le seuil en tension inversé V_Z (et en conséquence, plus de pertes).

2.2.3 Interaction sur le port série

Le port série classique est émulé par l'ARDUINO à travers de son interface USB. Pour initier ce port à la vitesse de transfert de 9600 baud (la vitesse typique des anciens terminaux et modems sur le port série RS232), nous ajoutons dans l'initialisation `setup()`

```
Serial.begin(9600);
```

Pour communiquer les informations par ce port, une fois que notre carte a été identifiée (comme `ttyACM0` dans ce document, ou `COM1`, `COM2`, etc sous WINDOWS), il faut ouvrir le terminal de l'environment Arduino IDE. Notez, qu'à l'ouverture du terminal, ainsi qu'après le téléchargement d'un nouvel exécutable, la carte se redémarre en exécutant le `setup`.

Sortir l'information : le texte (string) Nous pouvons sortir un message de démarrage, par exemple

```
Serial.println("\nSOS blinking and Serial interaction via keys");
```

Notez, qu'à la différence de `Serial.print` la commande `Serial.println` y ajoute automatiquement le changement de ligne. Pour ces deux commandes, le texte, dit «string», doit être entouré par les signes double de citation `"`.

> **Le caractère, le byte, le bit :** Chaque string consiste en un ou plusieurs caractères, et chaque caractère est représenté par un nombre 0..255 (où les caractères imprimables/lisibles démarrent à 32). Un tel nombre occupe un *byte ou octet*, c. à d., *huit* registres binaires 0/1 dits *bits* ($255 = 2^8 - 1$). Dans le langage `c, c++|, on déclare ces nombres avec le type` **`char`** `ou plus précisément` **`unsigned char`**`. Il existe plusieurs façons de définir la correspondance (codage) entre les caractères et ces nombres. Actuellement, le plus répandu et simple est le codage ASCII.`

> **Les types représentant les nombres entiers :** A la différence de **`unsigned char`**, le type **`char`** définit les nombres entiers dans l'intervalle $-127 \ldots 127$ qui occupent aussi un byte dont les 7 bits inférieures gardent la valeur ($127 = 2^7 - 1$) et le 8ième bit sert à donner le signe. Les nombres entiers, quant à eux, sont définis par la classe **`int`** ou encore **`unsigned int`** et occupent *deux* bytes. Par conséquent, ces nombres vont jusqu'à $\pm(2^{15} - 1)$ ou bien $2^{16} - 1$.

Entrer l'information Pour envoyer des informations à la carte, nous les entrons dans la ligne de commande du terminal suivies par la touche «Enter» qui les envoie. Les informations sont transmises comme un texte qui est mis en attente dans le tampon de l'interface série/usb dit «buffer». Pour savoir si les informations nous attendent à l'entrée, on appelle `Serial.available()`

[3] Tout DP de ARDUINO UNO sera détruite par la surtension U_{IN} d'entrée au-dessus de 5.5 V.

2.2.4 Montage du potentiomètre et le convertisseur A/N (ADC)

On branche les extrémités d'un potentiomètre de $R = 10$ à $100\,\text{k}\Omega$ entre GND et la sortie 5 V coté analogique ; le connecteur amovible se branche à l'entrée analogique, par exemple $A0$ (voir fig. 1). Le courant consommé de $V/R = 5/10^4 = 5\,\text{mA}$ est inférieur à celui que ARDUINO peut soutenir, voir sec. 2.2.1. La programmation est simple, voir l'exemple «AnalogInput» dans FILE ▷ EXAMPLES ▷ ANALOG. La partie essentielle du code est

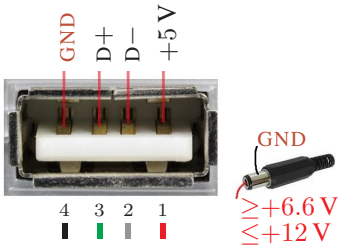
```
#define sensorPin A0
int sensorValue = 0;
sensorValue = analogRead(sensorPin);
```

où nous appelons `analogRead` pour lire la tension présente sur la broche $A0$ (l'entrée analogique). Cette fonction effectue la conversion de la tension analogique en signal numérique (A/N) avec un convertisseur à 10 bits pour la plupart des μCU ARDUINO. Cela signifie que le signal numérisé peut prendre $2^{10} = 1024$ valeurs comprises entre 0 et 1023, 1023 correspondant à 5 V.

On transforme ainsi la tension présente sur l'entrée $A0$ en une valeur lisible et exploitable par le μCU. La conversion consiste à comparer cette tension à une valeur de référence fixée à $+5$ V par défaut. Il est possible de comparer par rapport à une autre valeur de tension comprise entre 0 et 5 V qui est appliquée à la borne A_{ref} de la carte. Il faut alors le spécifier lors de l'utilisation de la commande `analogReference(reftype)`.

Programme final (avec option potentiomètre) On sauvegarde les modifications dans `BlinkSOS.ino` (voir appendice A).

2.3 Options d'alimentation



La carte ARDUINO[4] possède plusieurs possibilités d'alimentation. Jusqu'au présent, nous avons exploité les $+5\,V_{\text{CC}}$ et la masse GND de son cable USB (connecteur de type A coté ordinateur, voir ci à gauche). Cette option est la seule dans le cas où on ne possède pas d'une alimentation DC satabilisée, ni d'une batterie, et on l'utilise souvant au départ d'un prototypage. Cependant, ces $+5\,V_{\text{CC}}$ sont fournis par (le *hub* de) l'ordinateur, et par conséquent, (a) ils sont très «pollués» par les différents signaux logiques qui sautent fréquement entre 0 et $+5$ V ; et (b) la puissance (l'ampèrage) maximale de cette source est spécifique à l'ordinateur utilisé et elle n'est pas garantie d'être suffisant. Ces limitations rendent les $+5\,V_{\text{CC}}$ du cable USB particulièrement mal adaptés pour les mesures analogiques précises (avec un ADC interne ou externe, voir les sec. 2.5, 3.1, 3.3, ou bien 4.1.3). Dans ce cas, il est fortement recommandé de fournir une alimentation stable *externe* et supérieure à 5 V sur le jack dans la fig. 1-3. Ce jack est «centre-positive» (voir ci à gauche) et est protégé par une diode de la polarité inversée. Il connecte à un régulateur linéaire NCP1117 pour produire la tension V_{CC} stabilisée de $+5$ V. A son entrée (donc sur le jack) ce type de régulateur requit une tension supérieure d'au moins 1 V.

> **Le jack et le cable USB :** Dans le cas où le cable USB reste également connecté à l'ordinateur, la carte ARDUINO UNO bascule en alimentation externe fourni par le jack (fig. 1-3) ou le pin VIN si elle y sense une tension supérieure à $+6.6$ V.

Bien évidemment, la tension externe ne doit pas dépasser une valeur maximale, qui dans notre cas est de 12 V. Enfin, il existe une troisième option de connecter $+5$ V *directement* sur le pin 5 V (donc V_{CC}, à ne pas confondre avec VIN !). On rencontre cette option parfois dans les montages embarqués avec une source de $+5$ V stabilisées commun. Autrement, elle est à éviter comme dangereuse.

2.4 Traiter les interruptions et programmer les boutons

Cet exemple simple nous apprend l'idée très importante, celle de demandes d'interruptions externes IRQ et leur traitement par les fonctions ISR. On apprend aussi le branchement et l'utilisation d'une bouton poussoir (état normal ouvert) en mode «high-end switch» entre $+5\,V_{\text{CC}}$ et, à travers d'une résistance «pull-down» de $R = 10\,\text{k}\Omega$, la masse. La broche DP2 est liée à la bouton *avant* la résistance pour détecter la montée de la tension (dit RISING). Une LED branchée sur pin 10 (et la résistance de $220\,\Omega$ comme nous avons déjà appris dans la sec. 2.2), ou, tout simplement, la LED interne sur le pin LED_BUILTIN sert pour la visualisation. Un filtre passe bas optionnel avec une resistance R' et un condensateur C' sert à atténuer les perturbations HF (voir a sec. 2.4.1). On se rappelle d'une alternative dans la sec. 2.1.8 avec une photorésistance LDR (en gris dans la fig. 4b et 4c) pour commuter la DP2 à la place de bouton. Ce dernier montage est couramment utilisé comme un compteur de passage d'un obstacle, connu sous le nom de *photogate* (cf. la sec. 3.2). Notons que le temps de réaction d'une LDR (voir la sec. 2.4.5) est d'ordre de 50 ms pour l'arrêt de la lumière (donc pour la montée de sa résistance jusqu'aux quelques centaines de kΩ) et seulement de 10 ms quand elle devient illuminé. Par conséquent, si on cible de détecter le passage de noir vers blanc, correspondant au signal RISING et FALLING sur la DP pour les cirquits dans la fig. 4b et 4c, respectivement, le filtre optionnel peut avoir la fréquence de coupure $f_c = (2\,\pi\,R'\,C') \approx 1\,\text{kHz}$.

```
/*-*- mode: c++; coding: utf-8; current-input-method: latin-1-prefix; ispell-local-dictionary: "english"; fill-
    column: 79; comment-column: 0; c-syntactic-indentation: t; indent-tabs-mode: nil; show-trailing-whitespace:
    t; eval: (local-set-key "\C-c\C-\M-u" 'browse-url-firefox); -*-*/
#define _PROGRAM_ "intbut"          // program name
#define _VER_ "Nov 18 2025"         // program version
```

[4] Spécifiquement, nous considérons ici la UNO Rev.3. Pour les autres cartes, consultez leur datasheet.

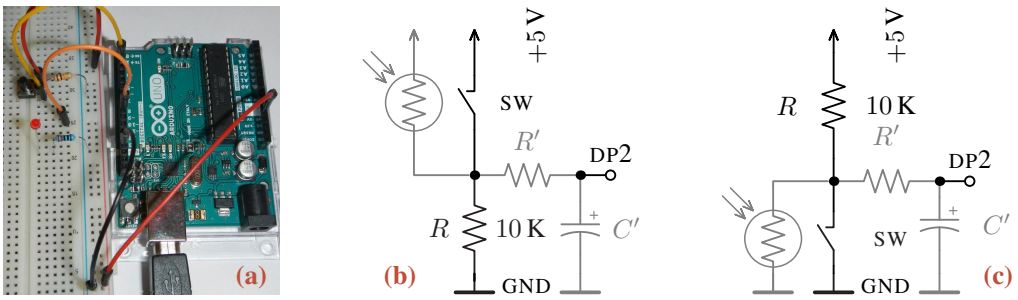


FIG. 4 – Bouton poussoir SW de type «high-end» (a)-(b) ou «low-end» (c) montée avec une resistance R de type «pull down» or «pull up», respectivement, pour commander une interruption IRQ externe sur le pin DP2 ou 3. Le circuit anti-rebond et une photorésistance LDR optionnels sont en gris.

```
#define DEBUG 1                    // set to 0 to suppress extra msg
#define LIB_DEBUG 0
/*
  Button Switch using an external interrupt. DS 2021-11-29

  This code is in the public domain and may be used without restriction and
  without warranty. The code is largely inspired by tutorials of Ron Bentley

  https://create.arduino.cc/projecthub/ronbentley1/button-switch-using-an-external-interrupt-7879df
  https://create.arduino.cc/projecthub/ronbentley1/understanding-and-using-button-switches-2ffe6c

  the button status is flagged as 'switched' AFTER the button is pressed
  AND then released, AND a debounce period has elapsed

  The button is wired in a standard configuration with an external 10K pull
  down resistor, which ensures that the digital interrupt pin is kept LOW until
  the button is pressed and raises it to HIGH (+5V).

  Operation of the button is demonstrated by toggling a LED on and off.
*/

// ATTN. on Arduino Uno and other 328-based boards, digital pins 2 and 3
// are the only ones that are usable for programmable external interrupts
#define BUTTON_PIN  2    // external interrupt pin trigerred by the button
#define LED_PIN     9    // digital pin connected to the LED indicator
#define LED_STATE   LOW  // initial LED indicator state (code testing only)

#define USE_SERIAL
#define SERIAL_BPS 115200           // => up to 14400 bytes/sec, 70 usec/byte

#define INT_TRIGGER_TYPE     RISING // interrupt triggered on a RISING input
#define DEBOUNCE             30     // time in msec to wait

// declare volatile to assure safe ISR access at any time
// 0 at first, counts subsequent incoming interrupts after
volatile unsigned int int_proc_stat = 0;
// begin with no switch press pending, i.e. false (not triggered)
volatile bool     switching_pending = false;
volatile long int elapse_timer = 0; // store the time of the interrupt

bool init_complete =         false; // inhibit the ISR during setup

// ISR for handling interrupt triggers arising from associated button switch
void button_int_handler() {
 if (init_complete && ((int_proc_stat+=digitalRead(BUTTON_PIN)) == 1)) {
     digitalWrite(LED_BUILTIN,HIGH);// the onboard LED on signals "busy"
     switching_pending = true;      // begin handling new button press
     elapse_timer = millis();       // reset DEBOUNCE elapse timer
   }
}

int button_state() {
 int k;
 if ((k=int_proc_stat)
     // interrupt flag k>0 has been raised on this button and requires handling
     && switching_pending && !digitalRead(BUTTON_PIN)
     && millis() - elapse_timer >= DEBOUNCE) {
   // switch was pressed, now released, and DEBOUNCE time has elapsed
```

```
#ifdef USE_SERIAL
   Serial.print(k);
   Serial.println(" ints");
   digitalWrite(LED_BUILTIN, LOW);  // the onboard LED off signals "done"
#endif /* USE_SERIAL */
   switching_pending = false;       // clear for new interrupt service cycle
   int_proc_stat = 0;               // reset interrupt counter
   elapse_timer = 0;
   return k;  // k>0 signals that switch has been pressed "k times" and released
 }
 return 0;    // either no press has occured or DEBOUNCE period has not elapsed
}

void setup() {
 int i;
 pinMode(LED_BUILTIN, OUTPUT);
 digitalWrite(LED_BUILTIN, HIGH);   // turn the onboard LED on = "busy"
#ifdef USE_SERIAL
 Serial.begin(SERIAL_BPS);
 while(!Serial);   /* wait for serial port to connect (native USB port only) */
 Serial.println(F("# button interrupt handler"));
#endif /* USE_SERIAL */
 pinMode(LED_PIN, OUTPUT);          // initialize switch indicator,
 digitalWrite(LED_PIN, LED_STATE);  // and trigger it four times
 for(i=8; i--; delay(500)) digitalWrite(LED_PIN, !digitalRead(LED_PIN));
 pinMode(BUTTON_PIN, INPUT);        // declare and set interrupt pin
 attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), // pins 2,3 give 0,1 on UNO
     button_int_handler, INT_TRIGGER_TYPE);
 init_complete = true;              // open interrupt processing for business
 digitalWrite(LED_BUILTIN, LOW);    // turn the onboard LED off = "done"
} // end setup

void loop() {
 interrupts(); /* re-enable interrupts (if/after disabled by noInterrupts() */
  if (button_state()) {             // test if the button is pressed
    digitalWrite(LED_PIN, !digitalRead(LED_PIN)); // toggle LED state
  } else {                          // do other things....
#ifdef USE_SERIAL
    if(digitalRead(BUTTON_PIN)) Serial.println(F("# button ON"));
#endif /* USE_SERIAL */
    delay(200);
  }
}
```

2.4.1 Mesures anti-rebond

Lorsque un interrupteur ou une bouton poissoir mécaniques sont enclenchés, les deux plaques métalliques ne rentrent pas en contact dans un coup. En réalité, elles vibrent légèrement en se rapprochant et s'éloignant très rapidement, et elles font parfois quelques dizaines d'oscillations pendant quelques millisecondes. Ceci engendre des interférences dans le circuit, que l'on appelle du «rebond» (dit *bouncing* en anglais). Ainsi, selon différentes sources (e.g., voir la note 6), les boutons poussoirs de «bon marché» nécessitent parfois jusqu'aux environs 10 ms pour etablir une connexion solide, alors que la durée d'un appui typique courte varie entre 50 et 100 ms (cf. fig. 5a). En même temps, on trouve des boutons, notamment des mini-boutons poissoirs destinés aux cirquits imprimés, qui assurent une connexion quasi-parfaite. Comme on peut voir dans la fig. 5b, ils passent de leur état normale ouvert à l'état fermé en 10 μs seulement et sans rebonds. Ce fonctionnement est assuré par leur mécanisme «à cliquet» qui provoque une fermeture franche et rapide quand une

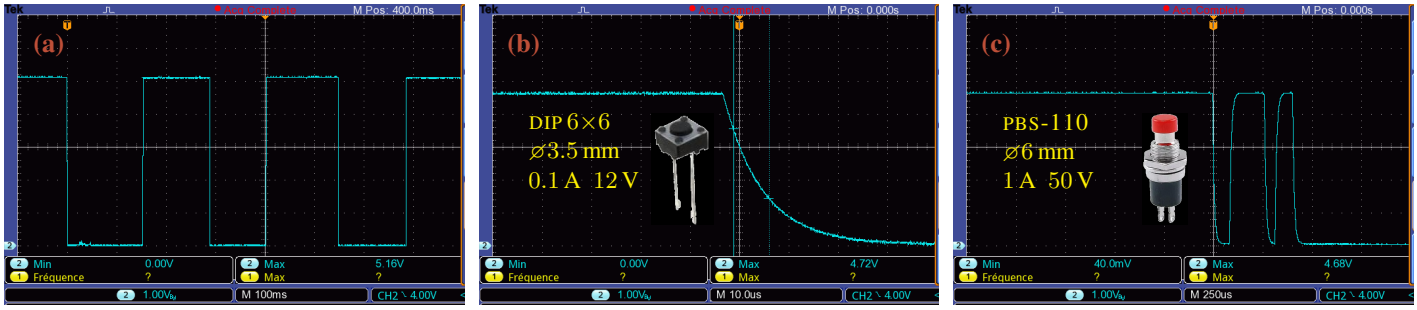


FIG. 5 – Boutons-poussoirs en action (circuit de la fig. 4c sans filtre passe bas) : trois appuis rapides (a) et profile du signal de fermeture (b) du bouton DIP 6×6 mm «à cliquet» qu'on trouve couramment sur des circuits imprimés ; les rebondissements du signal d'un bouton poissoir classique (c).

force d'appui supérieure à 1 N est appliquée. En méme temps, comme illustré dans la fig. 5a, ce mécanisme limite la durée minimale

d'appui à 100 ms. Les boutons poussoirs «classiques» permettent des appuis plus courtes[5] sans cliquet, mais du coup, l'appui n'est pas controlé, et s'il est effectuée assez lentement, on provoque des rebonds (pendant 250 ms dans la fig. 5c) avant un contact ferme.

Pour corriger ces anomalies du signal, i.e., pour supprimer le rebond (dit *debounce* en anglais), deux strategies sont envisagées couramment : (i) filtrer le signal électriquement pour éliminer les HF parasites[6] ; (ii) aménager la fonction ISR pour entamer un delai d'attente de quelques msec avant de valider l'appui du bouton. Cependant, l'approche «hardware» (i) reste assez limitée, car pour un résultat satisfaisant, elle nécessiterait un circuit assez complex avec des composants électroniques relativement chers, notamment $\times 2$ portes NON-ET à deux entrées (e.g. NAND *gate* TTL IC 7400N ou CMOS IC 4011), ou un filtre passe bas avec une fréquence de coupure très baisse, dans les 10 Hz, mais suivi par une bascule de *Schmitt* pour redresser les pentes du signal. Dans le cas d'un filtre passe bas semple dans la fig. 4, nous devons préserver les flancs RISING et FALLING du créneau de signal assez raides afin de permettre à μCU de les détecter et emmètre une requête d'interruption IRQ. Cela limite la fréquence de coupure f_c aux valeurs assez hautes. Néanmoins, un filtre de $f_c \approx 1$ kHz reste utile pour nous débarrasser de toutes HF parasites. Pour rendre les flancs inclinés symétriquement, on doit choisir $R \ll R'$=100 k, et avec $C' = 10$ nF, on obtient $f_c = 160$ Hz. Un tel filtre atténue les perturbations aux périodes inférieures aux ≈6 ms sans affecter de façon critique les pentes du créneau et sans compromettre la reconnaissances des appuis courtes.

Cote logiciel, la requête IRQ initiale de bouton est servie par la ISR `button_int_handler` qui demmarre le compteur de temps `Timer2` de μCU pour mesurer un delai de `BUTTON_GAP` ms avant de reconnaitre l'appui. Le compteur est réglé pour compter chaque 1/16 ms jusqu'au maximum de 16 ms (quand son registre achève la valeur maximale de 255). Aprés avoir fini son compte, `Timer2` provoque à son tour une IRQ servie par `ISR(TIMER2_OVF_vect)` qui vérifie l'état de bouton et clignote la LED.

```
#include <avr/io.h>                // direct uC register access
#include <avr/interrupt.h>         // see hardware/tools/avr/avr/include/avr/iom328p.h
#define BUTTON_PIN  2              // external button IRQ trigger
#define BUTTON_TYP HIGH            // normal HIGH, i.e. with a pull-up resistor
#define BUTTON_INT digitalPinToInterrupt(BUTTON_PIN)
// EICRA External Interrupt Control Register A for interrupt sense control
#if BUTTON_INT == 1
#define BUTTON_ISC ISC10           // LSB of ISC10,1 bits (2) Table 12-1 p.54
#else
#define BUTTON_ISC ISC00           // LSB of ISC00,1 bits (0) Table 12-2 p.54
#endif
#define BUTTON_SEN ((EICRA&(0x3<<BUTTON_ISC))>>BUTTON_ISC)
// boxcar wait interval [ms] during which subsequent button interrupts will be
// considered spurious and ignored to reduce jitter of the button switch
#define BUTTON_GAP 4               // [ms] should be in 1 .. 16
#define BUTTON_SLOPE FALLING       // i.e., goes from HIGH to LOW
#define BUTTON_CS 7                // default time resolution of 64us (1/16 ms)
#define BUTTON_FREQ 16             // [kHz] (approx), overrun in 16ms
#define BUTTON_CNT (0xFF - (BUTTON_FREQ*BUTTON_GAP-1))
// ISR for action on button gap expiring after timer 2 overflow (OVF vector 9)
volatile char button_state=BUTTON_TYP;
ISR(TIMER2_OVF_vect) {
  char s=digitalRead(BUTTON_PIN); // button state when delay has expired
  switch(BUTTON_SEN) {
    case 3: if(s==LOW) return;     // RISING  => expect high button state
    case 0:                        // LOW => check if low state persists, or
    case 2: if(s==HIGH) return;    // FALLING => expect low button state
    default:                       // SWITCH => check if button state has changed
    case 1: if(s==button_state) return;
  }
  if(s) {                          // toggle button LED indicator
    digitalWrite(BUTTON_LED, !digitalRead(BUTTON_LED));
    button_state = s;              // save current stable button state
  }
}
// ISR for handling interrupts triggered by the push button switching
void button_int_handler() {        // start Timer 2 to wait for button gap delay
 TCNT2  = BUTTON_CNT;              // clear (preset) 8-bit counter of Timer 2
 TCCR2B = BUTTON_CS;               // set Timer 2 system clock divisor
}
void setup() {
 pinMode(LED_BUILTIN, OUTPUT);
 pinMode(BUTTON_PIN, INPUT);         // declare pulldown push button or switch
 attachInterrupt(BUTTON_INT, button_int_handler, BUTTON_SLOPE);
 noInterrupts();                     // protected code
 TCNT2 = TCCR2A = TCCR2B = TIMSK2 = 0;   // reset Timer 2 registers
 TIFR2 &= ~0x7;                      // clear all Timer 2 interrupt flags
 TIMSK2 |= 1;                        // enable Timer 2 Overflow Interrupt
 interrupts();                       // enable all interrupts (unfreeze)
 digitalWrite(LED_BUILTIN, LOW);     // turn the onboard LED off
}
```

[5] Ainsi, la durée du signal court (dit «*dot*» ou «*dit*») dans une transmission de code *Morse* peut être seulement 50 ms !

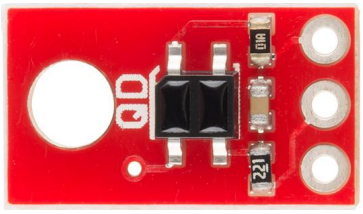
[6] cf. le commentaire de Michal Podmanický, Switch debouncing, EE SE 703816 (2024-03-01).

2.4.2 Réduire la consommation inutile : ARDUINO SLEEP MODE

En regardant notre code, on observe pour, que la plupart de temps, notre MCU ARDUINO reste en attente des appuis sur une bouton et donc tourne «à vide» dans son `loop()`, tout en consommant approximativement 30 mA ! Pour éviter ceci, il souffit de le faire «endromir», c.à.d de passer en mode SLEEP, dans lequel le MCU est à l'arrêt, il ne suit plus son programme, et ne peut être remis en marche que par un signal extérieur. Pour activer ce mode, apportez des modifications suivants à notre sketch.

```
#include <avr/sleep.h> // AVR library for sleep modes
. . . . . .
void button_int_handler() {
   . . . . . .
   sleep_disable();     // disable sleep mode
}
void setup() {
  . . . . . .
  set_sleep_mode(SLEEP_MODE_PWR_DOWN); // full sleep mode
} // end setup
. . . . . .
void loop() {
 interrupts(); /* re-enable interrupts (if/after disabled by noInterrupts() */
 sleep_enable();   // enable sleep mode
  if (button_state()) {             // test if the button is pressed
    digitalWrite(LED_PIN, !digitalRead(LED_PIN)); // toggle LED state
  } else {                          // do other things....
    if(digitalRead(BUTTON_PIN)) Serial.println("# button ON");
    delay(500);     // wait 500 msec before going to sleep
    digitalWrite(LED_BUILTIN,LOW);  // turn onboard LED off
    delay(200);     // wait 200 msec to allow the led to turn off
    sleep_cpu();    // activate full sleep mode
    Serial.println("Woke up!");     // first command after interrupt
    digitalWrite(LED_BUILTIN,HIGH); // turning onboard LED on
  }
}
```

2.4.3 Detecteurs d'obstacles et suiveurs de ligne



Comme une situation très courante (sec. 4.11.3), où les interruptions externes sont indispensables, considérez la détection des objets avec une *photocellule* (l'obstacle opaque passe entre l'émetteur et le récepteur de la lumière visible ou infrorouge), ou un «radar» (l'obstacle réfléchissant passe devant un émetteur-récepteur). Ainsi un *suiveur de ligne* QRE1113 est un variant d'un radar infrarouge (IR) à courte distance, typiquement de 3−5 mm. Les couleurs claires réfléchissent d'avantage. Ceci permets de détecter une ligne blanche sur le fond noir et vice versus. Le QRE1113 est composé d'un DEL-IR (émetteur) et un photo-transisteur (récepteur). On le trouve prémonté sur les petites cartes («breakout boards») ROB-09453 (analogique) et ROB-09454 (digital, ci à gauche) de SPARKFUN[7]. La première carte possède une résistance de 10 K en série à l'entrée du transisteur constituant un diviseur de tension avec la sortie V_{OUT} : sans réflexion (couleur noir), le transisteur est fermé et V_{OUT} reste HIGH. Ainsi le QRE1113 peut y être employé en continu comme un bouton du style «pull-up» aka «low-end». La deuxième carte possède un condensateur de 10 nF en série avec le transisteur et une résistance de 200 Ω protégeant la V_{OUT}. Nous pouvons reconvertir cette carte en détecteur analogique (continu) en ajoutant une resistance de 10 K (ou[7] 47 K) entre la V_{CC} et la V_{OUT}. Concrètement, sur un ARDUINO UNO avec $V_{CC} = 5$ V, une telle carte modifiée donne 0.5 à 0.7 V pour une réflexion par une plaque métallique coloré blanche située à une distance de ≈ 4 mm, et 4.9 V sans aucune réflexion (le détecteur pointe dans le vide). Ces valeurs suffisent largement pour déclencher les évènements HIGH/LOW logiques sur les pins digitales[8].

Ayant connecté la sortie VOUT de ROB-09453 ou de ROB-09454 modifiée au pin DP_2 de ARDUINO, et afin de compter le nombre des instances de détection `button_count` et connaître le temps depuis la première instance, on utilise le service d'interruption suivant.

```
#include <time.h>           // modified C header file for avr-libc and AVR-GCC
volatile time_t        button_start=0, button_timer=0; // time in ms of the first and last trigger events
volatile unsigned int  button_count=0;                 // number of button trigger instances
#define BUTTON_INT digitalPinToInterrupt(BUTTON_PIN)
#define BUTTON_LED LED_BUILTIN
void button_int_handler() { // ISR for handling interrupt triggers arising from associated button switch
 button_timer = millis();   // time of the event and number of events
 if(!button_count++) button_start=button_timer; // reset time of first event
#ifdef BUTTON_LED           // toggle LED indicator
 digitalWrite(BUTTON_LED, !digitalRead(BUTTON_LED));
#endif
}
```

[7] Appelés souvent *reflectance sensors* ou *encoders* en anglais, ces détecteurs sont fournis également par Pololu comme QTR-1A ou QTR-1RC, voir la réf. 0J13.
[8] Les tolérances de cartes ARDUINO de $V_{CC} = 5$ et 3.3 V pour les niveaux logiques HIGH/LOW sont élargies par rapport aux standards TTL et CMOS.

Se service[9] doit être declaré (=installé) dans le `setup()` avec `attachInterrupt` :

```
void setup() {
 pinMode(LED_BUILTIN, OUTPUT);
 digitalWrite(LED_BUILTIN, HIGH);    // turn the onboard LED on = "busy"
#ifdef BUTTON_LED
#if BUTTON_LED != LED_BUILTIN
 pinMode(BUTTON_LED, OUTPUT);        // special LED indicator of the trigger
#endif
 digitalWrite(BUTTON_LED, LOW);      // flush LED indicator of the trigger
#endif
 pinMode(BUTTON_PIN, INPUT);         // pulldown push button or switch
 attachInterrupt(BUTTON_INT, button_int_handler, FALLING); // the mode can be LOW, CHANGE, RISING, FALLING

 . . . .

 digitalWrite(LED_BUILTIN, LOW);    // turn the onboard LED off = "done"
}
```

Ici le mode FALLING est choisi pour marquer exclusivement l'instance de passage HIGH → LOW du signal V_{OUT} de capteur QRE1113.

> Ainsi, dans le cas de pendule de *Pohl* (sec. 4.11.3), LOW correspond à sa position de l'équilibre (où une petite flèche blanche réfléchit la lumière du capteur). L'interruption aura lieu seulement à l'entrée dans cette position ; le départ ne la provoquera pas. Cela permet de lancer une acquisition «en attente» d'un premier passage à l'équilibre, décaler le pendule à la main pour attribuer les conditions initiales (amplitude, phase) au système, et lâcher. L'enregistrement commencera au premier arrivée à l'équilibre avec une phase initiale $\varphi_0 = 0$ ou π selon la direction de lancement.

Maintenant, le temps de la première détection sera sauvegardé dans `button_start`, et le numéro consécutif et le temps de la détection la plus récente se trouveront respectivement dans `button_count` et `button_timer` à tout moments au cours de l'execution de `loop()`. Ceci permet, par exemple, estimer l'intervalle entre instances périodiques :

```
 if(button_count>1) Serial.println( floor((float)(button_timer-button_start)/(button_count-1)), 0);
```

Notez, que pour réinitialiser le comptage, on remet `button_count` à 0. On peut également accéder à l'état momentané (HIGH ou LOW) du capteur avec `digitalRead(BUTTON_PIN)`. Par ailleurs, on peut choisir suspendre l'activité et attendre l'instance de détection :

```
#include <avr/sleep.h>      // AVR library for sleep modes
#include <avr/power.h>
void loop() {

 . . . .

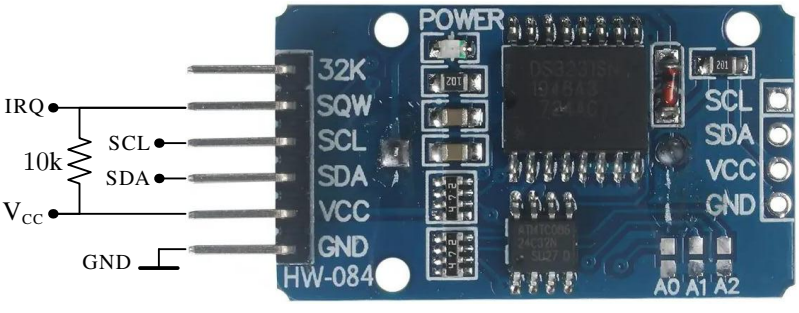
    Serial.flush();                       // clear all output on serial line
    button_count=0;                       // reset trigger count
    set_sleep_mode(SLEEP_MODE_PWR_DOWN);  // select full sleep mode
    sleep_enable();                       // (re)enable sleep mode
    sleep_cpu();                          // ***** ACTIVATE SLEEP *****
    sleep_disable();                      // on comeback: disable the sleep mode

 . . . .

}
```

2.4.4 Utilisation d'une horloge RTC pour reveiller ARDUINO

Le mode SLEEP est évidemment très utile dans des nombreuses situations courantes, où le MCU agit aux moments de temps precis et ne travaille que quelques centaines de msec (par ex., pour prendre des mesures) par minute (heure, jour, mois, année, etc).



Pour des intervalles de temps assez courts, inférieurs ≈ 4 sec, nous pouvons nous servir de l'un de compteurs de temps de 8 bit (`timer0` et `2`) et 16 bit (`1`) internes standards que possède l'IC ATMEGA328P, et qui sont pilotés par l'horloge système de 16 Mhz de la carte ARDUINO UNO (en version 5V, et 8 MHz en version 3.3V) ou un horloge externe. Par ailleurs, en utilisant le WDT (watchdog timer) de ATMEGA328P nous pouvons aller jusqu'à ≈ 8 sec, tout en minimisant la consommation. Ayant un oscillateur interne dédié de 128 kHz pour sa source, le WDT est le seul capable de fonctionner en mode de sommeil profond `SLEEP_MODE_PWR_DOWN`, et sa cadence minimale est de 60 Hz (16 msec),

[9] La procédure de service d'interruption (ISR) est engagé à l'instance d'interruption et bloque tout autre activité du MCU. Par conséquent, une ISR doit être très rapide afin de ne pas empêcher le fonctionnement normal interrompu.

Les intervalles de temps plus longues demandent un «montre» que le MCU ARDUINO ne possède pas d'office. Il faudra ajouter au MCU un module externe RTC capable de passer le signal TTL d'un reveil programmable. Ainci nous allons utiliser une mini-carte (au dessus à gauche) basée sur le IC chip DS3231 (2 alarmes, capteur temperature interne), puce mémoire AT24C32 (32 K), et l'interface de communication I^{2}C (adresse `0x68`). Ces pins SCL et SDA sont connectés[10] aux pins horloge ($A5$) et données ($A4$) I^{2}C de ARDUINO, voir la fig. 1b, tout en gardant les fils de liaison I^{2}C au plus court. Le pin SQW de DS3231 communique l'alarme via le pin DP$_3$ (IRQ1) de ARDUINO par une interruption externe[11]. Ce pin est du type «open drain» et en cas d'alarme, il est mis à la masse (LOW). Par conséquent, cela demande un montage «pull-up», avec V_{CC} passé à SQW via une résistance[12] de 10 kΩ. On ajoute les et definit/initialise la structure `dt` pour garder le temps courant bibliothèques

```
#include <Wire.h>   // I2C
#include <DS3231.h> // RTC
DS3231 myRTC;                                // RTC data object
DateTime dt = DateTime(2022,10,21,12,30,0); // set default date
```

L'interruption d'alarme RTC passé sur `RTC_PIN` est servie de façon très simple : on signale l'alarme par un «flag» et réveille le μCU.

```
// declare volatile for safe ISR r/w access at any time
// RTC alarm flag: bits 0 and 1 signal DS3231 alarms 1 and 2
volatile unsigned char alarm_flag = 0;
// an ISR cannot talk via I2C (to the RTC) because interrupts are disabled!
// so, in particular, we cannot check RTC alarm flags with checkIfAlarm
void rtc_int_handler() {
 alarm_flag |= 0b1000;              // signals a new alarm interrupt (bit 3)
}
// bits 3,2 and 1,0 are set when alarms 2,1 are enabled and call an interrupt
// NB: the interrupt flags are automatically cleared, see checkIfAlarm
// WARNING: RTC interrogations use I2C and interrupts must be enabled
unsigned char rtc_alarm_state() {
 unsigned char i,k=0;
 for(i=2; i; k<<=1) k |= myRTC.checkAlarmEnabled(i--); // enabled bits
 for(i=2; i; k<<=1) k |= myRTC.checkIfAlarm(i--);      // interrupt flag
 return (k>>1);
}
```

On initialise le module d'horloge dans le `setup` et demarre ces deux alarmes (une chaque seconde, l'autre chaque minute à 0 secondes)

```
#define BUTTON_PIN  2    // external interrupt pin trigerred by the button
#define RTC_PIN     3    // external interrupt pin trigerred by the RTC alarm
#define RTC_INT    digitalPinToInterrupt(RTC_PIN)
void setup() {
  int i;
  . . .
  Wire.begin();  // initialize I2C
  myRTC.setClockMode(false);                 // 24h clock mode
  myRTC.setEpoch( dt.secondstime(),false);   // default RTC date
  for(i=2; i; myRTC.turnOffAlarm(i--));      // disable all RTC alarms
  myRTC.enable32kHz(false);                  // just in case it is in awrong state
  myRTC.enableOscillator(false,false,0);     // alarm interrupts on the INT/SQW pin
  myRTC.setA1Time(1,10,30,0, 0b1111111, true,false,false); // every second
  myRTC.setA2Time(1,10,30,   0b1111111, true,false,false); // every minute
  // INPUT_PULLUP involvs an internal resistor of approx 50k, which is too large
  // use INPUT and an extenal 10k pull-up resistor between 5V VCC and SQW
  pinMode(RTC_PIN,INPUT);                    // receive alarm(s) from RTC pin SQW
  attachInterrupt(RTC_INT, rtc_int_handler, FALLING);
  rtc_alarm_state();                         // clear stale alarm flags, just in case
  for(i=2; i; myRTC.turnOnAlarm(i--));       // enable both RTC alarms
  . . .
}
```

Le reste est fait dans le `loop` parceque en mode «deep sleep» le bus I2C ne fonctionne pas et nous ne pouvons pas interroger le RTC. Ici, pour un teste, on imprime les secondes chaque seconde et le temps/date complets et la temperature (de RTC) chaque minute.

```
// actions that should be taken in response to the raised RTC alarm flag
// because the MCU should never go to sleep with stale RTC alarm interrupts
// this should either be executed repeatedly and/or right before going to sleep
unsigned char alarm_action() {
 unsigned char t;
 if (t=alarm_flag) {                // find and store which RTC alarm was called
   // the value HIGH=1 of the DS3231 pullup interrupt pin means no interrupt
   alarm_flag |= (digitalRead(RTC_PIN)^1)<<2;
   alarm_flag |= rtc_alarm_state() & 3; // autoclear DS3231 interrupt flags
```

[10] Les cartes UNO récentes (Rev. 3) possèdent deux entrées dédiées à cote de pin AREF, qui sont connectées à $A5$ et $A4$ et sont référencées ainsi dans le code.
[11] ATMEGA328P possède *deux* pins DP$_2$ et DP$_3$ programmables pour les interruptions externes IRQ0 et IRQ1, et dans la sec. 2.4, nous réservons le DP$_2$ pour le bouton.
[12] Ainsi limitant le courant à 0.5 mA. Rappelons (sec. 2.2.1) que chaque I/O pin DP$_i$ de ARDUINO peut source/sink un maximum de 40mA (donc on vise $\leq$20mA).

```
 }
 dt = RTClib::now();                  // update current time
 if (alarm_flag&1) Serial.println(dt.second());
 if (alarm_flag&2) {
   Serial.print(dt.year());    Serial.print("-");
   Serial.print(dt.month());   Serial.print("-");
   Serial.print(dt.day());     Serial.print(F(" "));
   Serial.print(dt.hour());    Serial.print(":");   Serial.print(dt.minute());
   Serial.print(F(" tRTC="));
   Serial.print(myRTC.getTemperature()); Serial.println(F("C "));
 }
 alarm_flag = 0;                      // clear all alarm flags
 return t;                            // return the flag acted upon
}

void loop() {
 interrupts();     // re-enable interrupts (if/after disabled by noInterrupts()
 set_sleep_mode(SLEEP_MODE_PWR_DOWN);   // full sleep mode
#if defined(DEBUG) && DEBUG > 2
 Serial.print(F("--loop-- "));
#endif
 sleep_enable();                      // (re)enable sleep mode
 digitalWrite(LED_BUILTIN,LOW);       // turn onboard LED off
 delay(10);                           // wait to allow the led to turn off
 rtc_alarm_state();                   // just in case alarms went off unattended
 sleep_cpu();                         // ***** activate sleep *****
 sleep_disable();                     // on comeback: disable the sleep mode
 power_all_enable();                  // re-enable the peripherals, do we need it?
 digitalWrite(LED_BUILTIN,HIGH);      // turn onboard LED on to signal "busy"
#if defined(DEBUG) && DEBUG > 2
 Serial.print(F("Woke up! "));
#endif
 alarm_action();
}
```

2.4.5 TP : Le temps de réponse et la fréquence d'échantillonage. Photorésistance vs photodiode

Afin de comparer la photoresistance (LDR, petite ou moyenne de diamètres respectifs de 3 ou 5 mm) et la photodiode (PD, SFH203 diamètre 5 mm) comme *capteurs de passage rapide*, considérez le montage suivant[13].

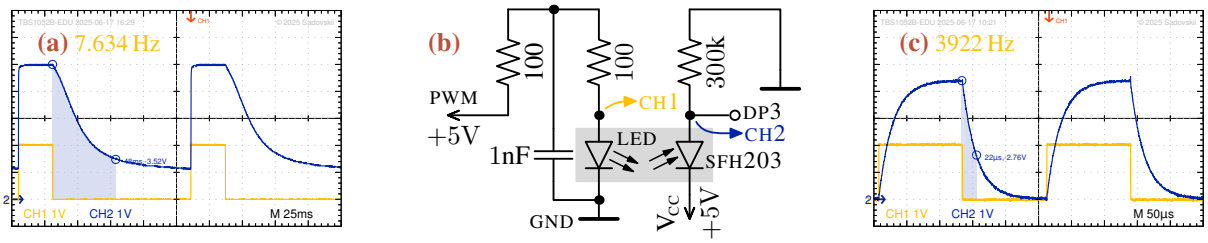


FIG. 6 – La réponse (a) d'une LDR et (c) de la photodiode SFH 203 à une LED (⌀ 5 mm, rouge) alimentée par le µCU ARDUINO UNO en mode PWM (pin DP9) à 7.63 Hz avec rapport cyclique de 20% et 4 kHz avec 50%, respectivement ; les traces jaune (CH1) et bleue (CH2) correspondent à la tension aux bornes de la LED, et celle aux bornes de la résistance biais de la LDR (10 k) ou de la PD (300 k) dans le circuit (b).

1. Dans le premier temps, utilisez la sortie d'un GBF (créneau symétrique 50:50% de 0 à ≈5 V) pour allumer une LED rouge de 3 ou 5 mm en série avec R_1 de 150 ou 220 Ω. Penser à éliminer (réduire) la lumière ambiante.
2. Pour biaiser les capteurs en mode «high-end switch», i.e., placés côte V_{CC}, prenez une source stabilisée de 5 V DC et la résistance diviseur R_2 de 10 k pour la LDR et de 300 à 330 k pour la PD (voir le circuit dans la fig. 6b). Notez que la PD est montée en mode *photoconductif*, i.e., elle est branchée «à l'invers», avec son anode côte masse. En option, testez le cirquit «low-end switch».
3. Branchez le canal CH I (trace jeune) de l'oscilloscope sur l'alimentation de la LED (donc GBF), et le canal CH II aux bornes du capteur (la LDR ou la PD). Joignez les masses de l'oscilloscope, du GBF, et de la source DC.
4. Vérifiez que dans les deux cas, en mode «noir», la tension U aux bornes de capteur (la trace bleue dans la fig. 6) monte assez haut, spécifiquement, au dessus de $V_{IH} = 3$ V, pour que ATMEGA328 la reconnaît comme HIGH (consulter fig. 3). Si cela n'est pas le cas, refait l'expérience : Explorez les valeurs de R_2 plus baisses pour monter U_{max}. Pour augmenter ΔU, testez une LED plus luminieuse (de ⌀5 mm ou une 3 mm à puissance élevée). Changez sa couleur.

[13] Le temps typique d'allumage complet d'une LED est de ≤ 1 µs, cf. MartinC, What is the latency of a LED ?, EE SE 675933 (2023-07-31). Par ailleurs, sa capacité nominale de 50 pF contribue aussi à la constante de temps de même grandeur pour la tension directe $V_{LED} \approx 2$ V (légèrement au dessus de son «tension de coude») et le courant typique de 20 mA (voir la sec. 2.1.5). Par conséquent, le signal PWM présentant, surtout s'il parvient d'un µCU, un bruit HF, peut être coupé sans pertes sensibles à $f_c \approx 1$ MHz. On achève cela plus facilement en insérant un filtre passe bas dans le montage «pullup» (fig. 2b).

5. Dans le deuxième temps, utilisez ARDUINO UNO pour créer la tension créneau (en mode PWM sur pin DP_9, voir note 13), pour biaiser le capteur avec son V_{CC} de 5 V, et aussi pour détecter le signal (pin DP_3) en mode compteur des interruptions sur la borne positive du capteur, voir la sec. 2.4, et mesurer sa fréquence (avec fréquencemètre + générateur PWM dans l'appendice C).
6. Enfin, on se propose d'effectuer la même étude pour un phototransistor NPN. Ansi on observe une réponse similaire à celui dans la fig. 6c pour un vieux Honeywell SD5491 ou ses analogues avec $R_2 = 200$k et la base déconnectée.

Dans la figure 6, on observe que la réponse d'une LDR est limitée à ≈10 Hz, et en plus, elle est fortement asymétrique : la résistance de la LDR monte très lentement sans lumière (la tension descende de 3 et 1.5 V en 30 ms) et chute rapidement en mode illuminé (en ∼3 ms) ; cela corresponde bien à son datasheet. Notons, que le temps de 30 ms rende la LDR très inefficace, voir inutile, pour détecter une projectile à la sortie d'un canon … En revanche, la réponse symétrique de 25 μs de la PD nous permet aller jusqu'à 10 kHz. Son datasheet nous annonce 5 ns, et on peut suspecter des capacités «parasites» dans le cirquit (de la PD et/ou de la LED). Par ailleurs, la PD laisse passer des très petits courants de ≈20 μA et l'utilisation directe (i.e., sans préamplification) de l'entrée sur le pin DP_3 n'est possible que si son impédance en mode INPUT reste bien dans les 100 M comme annoncé dans la documentation de μCU ATMEGA328P.

2.4.6 IRQs de comparateur analogique

Jusqu'au présent, pour placer une IRQ externe (INT0 ou INT1 sur le UNO), nous utilisons la tension u_R sur une résistance de biais R du pont diviseur (cf la sec. 3.1) basculant entre les niveaux HIGH et LOW de μCU (voir la fig. 3). Evidemment, cela depende de l'impédance du capteur et n'est pas toujours possible. Ainsi, la lumière peut manquer dans l'expérience de la sec. 2.4.5 ci-dessus si on insère

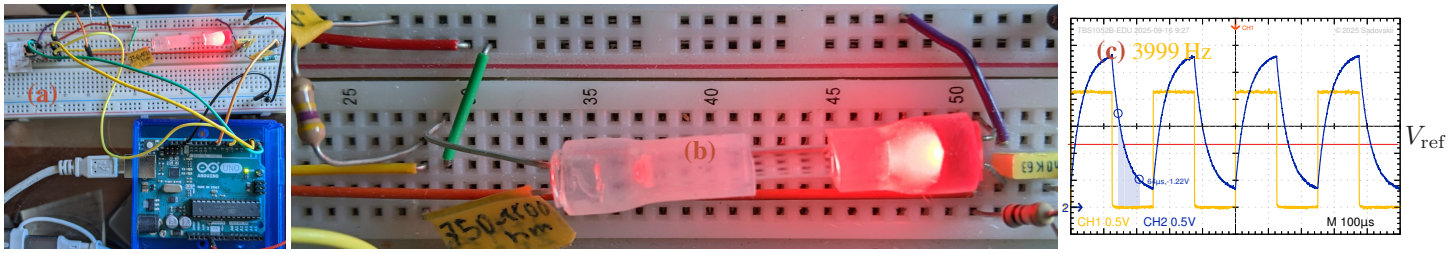


FIG. 7 – Le teste de la sec. 2.4.5 modifié (a) par le guide de lumière (b) (PMMA ⌀ 3 mm, longueur ≈20 mm) placé entre la LED et la photodiode SFH 203 avec R_{biais} de 470 K dans le circuit de la fig. 6b ; la réponse (c) de la photodiode à 4 kHz.

un «obstacle» entre la LED et la PD, tel qu'un guide de lumière dans la fig. 7b. Le courant dans le circuit de PD (voir la fig. 6b) est diminué. Il est possible d'augmenter R dans ce circuit tout en restant bien en dessous de l'impédance d'entrée[14] de l'μCU, et surtout de l'oscilloscope (1 M), si on l'utilise. Pour la PD, la valeur de $R_{\text{bias}} = 470$ reste adéquate. Néanmoins, si une R_{bias} augmentée n'arrange pas les choses, comme dans la fig. 7c, où la réponse (trace bleue) ne dépasse 2.5 V, on peut utiliser le *comparateur analogique* AC qui génére une IRQ lorsque la tension u_R devient égale à une tension de référence V_{ref}. Sur la carte UNO, les entrées de comparateur AIN0 et AIN1 correspondent aux DP_6 et DP_7, dont seule la DP_7 sera utile pour nous parce que la *référence interne* de 1.1 V servira comme V_{ref}. Il suffit de basculer la sortie du PD de DP_3 à DP_7, ou, autrement, on peut tout simplement lier les deux DPs. La IRQ de DP_3 est servie par la petite ISR dont le vecteur est INT1_vect. Puisque la IRQ de la DP_7 est servie par la méme routine (voir l'appendice C) :

```
ISR(ANALOG_COMP_vect, ISR_ALIASOF(INT1_vect));    // same IRQ handler for AC and DP3
```

On active le comparateur AC dans le setup() en configurant son registre ACSR, notamment le bit 7 (ACD pour activer le AC), 6 (ACBG pour choisir V_{ref} de 1.1 V à sa borne positive), 3 (ACIE pour activer l'interruption), et 0-1 (0, 2, ou 3 selon le mode d'interruption).

```
ACSR = (1<<ACD)|((1<<ACBG)|(1<<ACIE)|ACIS_SLOPE); // mode 0 SWITCH, 2 FALLING, or 3 RISING
```

2.5 Échantillonnage rapide avec ARDUINO ADC

L'échantillonnage en temps réel avec des intervalles de temps équidistants est une tache très courante dans de nombreuses applications[15]. Le temps de mesure d'un échantillon par l'ADC de microcontrôleur ARDUINO UNO étant de 100 μsec, le taux maximal théorique peut atteindre 10 kHz. Cependant, le MCU possède peu de mémoire (2K) pour sauvegarder les données acquises. L'alternative est de transférer les mesures immédiatement par son port série. Dans ce cas, on doit compter le temps de transfert. A la vitesse de 115200 baud, ce temps est estimé à 70 μsec/byte. Ainsi, si on utilise le mode texte (ASCII), un nombre inférieur à 1024 est représenté par quatre caractères, chacun étant codé sur un byte. Il nous faudra donc *quatre* bytes maximum en format décimal, ou bien *trois* bytes en format hexadécimal pour les valeurs inférieures à 1024[16]. A ceci on ajoute le caractère de terminaison de ligne (LF sous LINUX). Donc en tout, nous allons avoir besoin de quatre bytes minimum, et le temps pour un échantillon s'élève à $100 + 4 \cdot 70 \approx 400$ μsec. En conclusion, dans cette méthode, il nous sera possible d'échantillonner aux fréquences inférieures ou égales à 2.5 kHz. Pour le cas de transfert binaire, avec seulement *deux* bytes envoyés par échantillon, ce taux remonte à 4 kHz.

[14] supérieure à 10 M selon la documentation pour les DPs, mais bien plus baisse pour les entrées de l'ADC et du comparateur analogique
[15] comme une application possible, voir TP «réaction oscillant»
[16] il s'agit ici du nombre maximal de bytes nécessaires, pour les petites valeurs ce nombre décroît

Nous allons échantillonner un signal AC de 0..5V et de fréquence 100 Hz avec l'ADC. On peut utiliser un GBF comme source, mais à l'occasion ici, pour prendre un exemple, nous avons pris un ancien chargeur des piles NiCd et un potentiomètre variable, voir la figure 8a, de 10 kΩ pour régler la tension de sortie V_{out} à 0..5 V. Le chargeur utilise un transfo et deux diodes (two diode rectifier), les bornes A0 (entrée analogique 0) et GND de ARDUINO sont branchées sur le potentiomètre. Une fois téléchargé dans le MCU, le

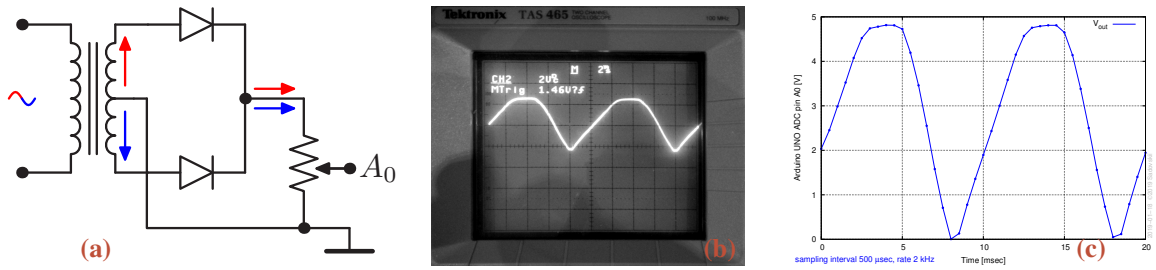


FIG. 8 – Montage pour tester l'échantillonneur rapide en teps réel (a), la tension passée sur le pin A0 (b), les données (c).

programme `~/Arduino/adcsampler/adcsampler.ino` (voir l'appendice B) permet de communiquer avec lui[17] en utilisant les clés T, N, H, et S, pour définir l'intervalle de l'échantillonnage (`READ_PERIOD`), le nombre des échantillons (`npnt`), ainsi que le format (décimal ou hex), puis démarrer les mesures. Par exemple, avec `T500N100HS` on définit l'intervalle de 500μsec, 100 échantillons et format hex. La figure 8c montre le résultat sous forme graphique.

2.6 Simple DAC

Le MCU ATMEGA328 de la carte ARDUINO ne possède pas d'un convertisseur digital vers analogique (dit dac) permettant de sortir de façon programmable des différentes fractions de son V_{CC} de 5 ou 3.3 V. Cependant, avec peu de composantes, nous pouvons exploiter sa sortie PWM pour créer un circuit DAC simple pour les applications à précision et le temps de réaction basses à moyennes. Dans le cas le plus simple, on applique le filtre RC passe bas (22 μF, 3.3 k) au signal PWM et on protège la sotie avec un amplificateur opérationnel[18] en mode «tampon». On reconnaît qu'il s'agit en effet d'un filtre actif passe bas de *Sallen-Key*. Ainsi, une fréquence de coupure de ≈250 Hz (pour la fréquence porteuse de PWM à phase correcte de 490 Hz qu'on obtient avec `analogWrite` sur les pins digitaux $\text{DP}_{9,10}$) peut être achevée avec $R_1 = R_2 = 6.2\,\text{k}$ et $C_1 = C_2 = 100\,\text{nF}$. Par ailleurs, pour diminuer le temps de réponse, on peut utiliser un filtre RLC en mode critique, mais encore mieux—augmenter la fréquence porteuse de PWM. Ainsi, le ATMEGA328 à 16 MHz (carte ARDUINO UNO) peut sortir sur ses pins $\text{DP}_{9,10}$ (`Timer 1`) le signal PWM de résolution 10 bit (0.1% duty) et fréquence max 15.6 (fast) ou 7.8 (phase correct) kHz. Une bonne base pour un DAC assez versatile avec un simple filtre RC.

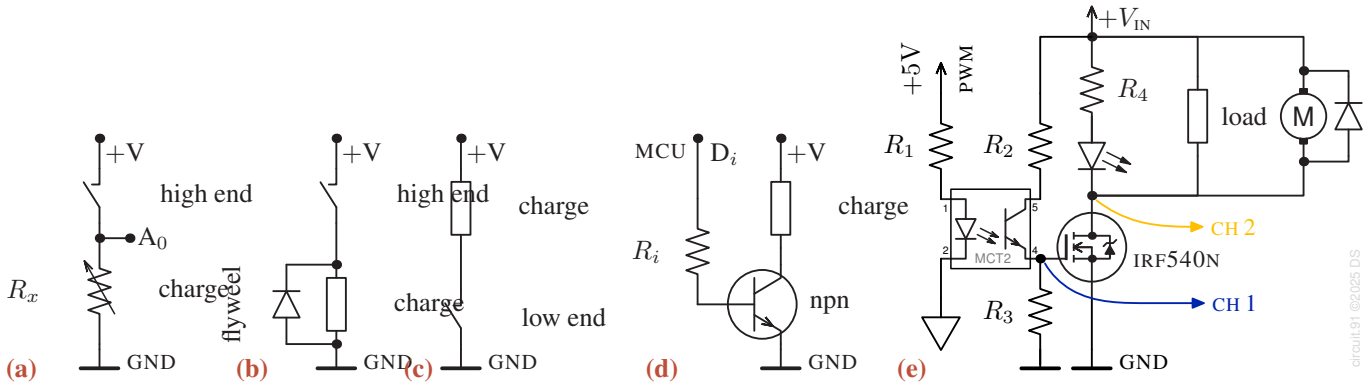


FIG. 9 – Les possibilités différentes de commander l'alimentation d'une charge à forte puissanse (dit «load») : (a) interrupteur côte alimentation (dit «high end power switch») permettant des mesures de tension (pin A_0 de μCU) aux bornes d'une charge ohmique ; (b) le même, mais avec une charge inductive (moteur, bobine d'un relais) et une diode (dit «flywheel» ou «snubbing») protégeant l'interrupteur au moments de déconnexion ; (c) interrupteur déconnectant la masse (dit «low end») ; (d) le cirquit (c) dont l'interrupteur est remplacé par un transistor npn commandé par le pin digital DP_i de μCU (0 pour ouvrir et $+V_{\text{DD}}$ pour enclencher). Notez, que la tension +V dans (d) peut parvenir d'une source d'alimentation dédiée à forte consommation avec sa masse *connectée* à celle de μCU, et peut être différente (supérieure !) de (à) la $V_{\text{CC}} = V_{\text{DD}}$ operée par le μCU. Au contraire, le curquit (e) utilise un optocoupleur et un N-MOS-FET pour commander les appareils «externes» dont V_{IN} et la masse GND sont *isolées* de l'μCU.

2.7 Commande des appareils à forte puissance

On est parfois obligé de commander des instruments ou des équipements (dit *charge* ou *load*) qui peuvent consommer des intensités incompatibles avec les sorties du μCU. On est amené à *découpler* le circuit d'utilisation (forte intensité) et de commande (faible intensité).

[17] en utilisant le moniteur série de IDE, la vitesse du port série est réglée à 115200 baud

[18] mais à ce point, on entre de nouveau dans un «trade-off»—dépenser 40+ centimes pour le IC tampon, ou 70 centimes pour un DAC minuscule à résolution 8-bit ?

Pour cela on utilise classiquement un transistor en commutation, soit un transistor bipolaire BJT ou un MOS-FET. Ce sont des éléments électroniques à 3 broches dénommées comme indiqué ci-dessous.

BJT	Base	– Collecteur (collector) –	Emetteur (emitter)	TIP120
MOS-FET	Grille (Gate) –	Drain	– Source	IRF520 (5V), FQP30N06L (3.3V)
Connexion	sortie μCU –	power through load	– masse (ground)	

Pour modéliser la charge (fig. 9), nous prendrons une LED, une résistance ohmique, ou un moteur DC. La masse de la source d'alimentation à commander et la masse de μCU sont connectées.

Dans le cas d'un moteur DC, ou toute autre charge inductive, nous addons une diode de puissance 1N400X en parallèl avec le collecteur et émotteur du transistor, pointée away from ground. La diode protège le transistor from back voltage generated when the motor shuts off, or if the motor is turned in the reverse direction. Used this way, la diode est appelée de «protection», «flyback» ou encore «snubbing» You can omit the diode if you don't have one, as the transistors recommended here all have a built-in protection diode

2.7.1 Régler la vitesse d'un moteur DC conventionnel

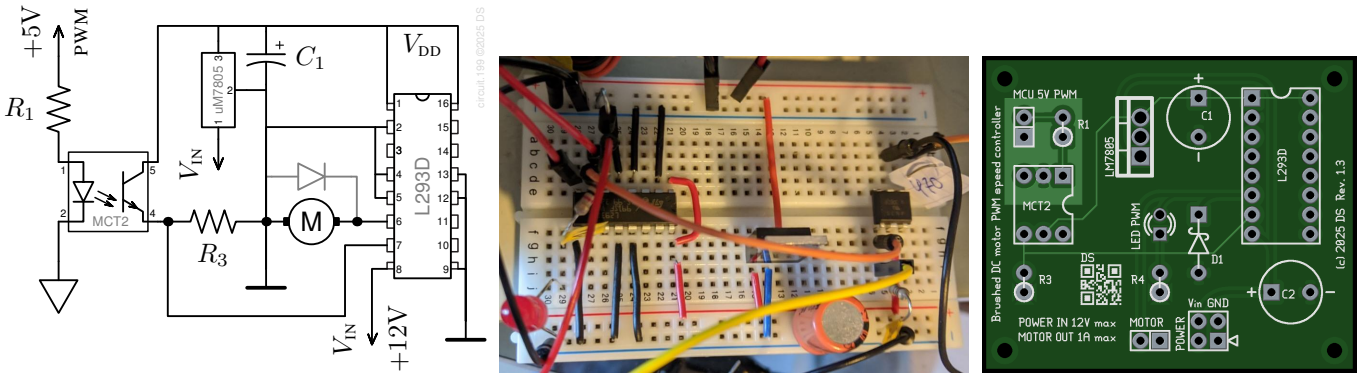


FIG. 10 – Contrôleur de la vitesse d'un moteur DC conventionnel (i.e., avec un collecteur et balais ou charbons) sans charge mécanique importante

3 Les capteurs

Le microcontrôleur est réellement adapté pour l'utilisation de capteurs au sens large. Il s'agit bien souvent de mesurer une grandeur électrique (tension ou courant) ou un temps qui sont proportionnels à la grandeur physique que mesure le capteur. Nous vous proposons d'utiliser et de mettre en oeuvre un capteur de température basé sur une thermistance et un capteur de lumière basé sur l'utilisation d'une photo-résistance. Vous mettrez en place la liaison série vers le moniteur série pour visualiser les données de votre capteur.

3.1 La thermistance

On connaît plusieurs méthodes de mesurer la température qui exploitent les phénomènes d'expansion, l'éffet thermoélectrique, et la radiation thermique. Les capteurs à la base de semi-conducteurs ont été développés en 20me siècle. Ils répondent rapidement et précisément aux variations de la température par changements de leur conductivité (éffet thermoélectrique), mais manquent de la linéarité[19]. La thermistance[20] dont vous disposez est une CTN (Coefficient de Température Négative), c'est à dire que sa résistance diminue avec l'augmentation de la température. Il existe également des CTP (Coefficient de Température Positif). La résistance d'une CTN varie en fonction de la température de façon *non-linéaire* suivant la relation de *Steinhart-Hart* définie par un polynôme de degré 3

$$T^{-1} = a + b\,\log(R_T) + c\,\left[\log(R_T)\right]^3$$

avec trois coefficients phénoménologiques (a, b, c). Pour une plage de T restreinte, on peut également utiliser une approximation

$$\log\frac{R_t}{R_x} = B\,\frac{x-t}{(T_0+t)(T_0+x)} = B\left(\frac{1}{T_0+t} - \frac{1}{T_0+x}\right) \text{ où } t \approx x \text{ en °C, } B = B_x = \frac{\alpha_x}{100}\,(T_0+x)^2 \text{ et } T_0 = 273.15\,\text{°K.}$$

Ici t et x représentent la température de référence (typiquement 25 °C) et celle mesurée actuellement. Voici la réalisation C++.

[19] Les circuits intégrés contemporains corrigent la non-linéarité de la réponse de la thérmistance, et sont très faciles et peu coûteux à déployer. Ainsi, le signal de la sortie analogique d'un TMP36 de 0 à 1.5 V couvre les températures de -50 à 100°C avec la précision de ± 1°C.

[20] en anglais on dit thermistor NTC ou PTC

```
#define ADC_MAX   1023
/* 1023 vs 1024, cf this https://skillbank.co.uk/arduino/adc.htm#adc4 */
#define ABS_T0  273.15   // 0C in Kelvin
/* parameters for the NTC thermistor TDK B57164K104J K164 100, 100K at 25C */
#define NTC_t  25        // reference temperature in Celsius (25)
#define NTC_B 4600       // value of parameter B [K] at reference temperature
#define NTC_Rt 100000    // [Ohm] resistance at t = 25
#define NTC_Rref 100000  // [Ohm] fixed resistor on the V divider
#define NTC_r ((float) NTC_Rref/NTC_Rt)
/* get thermistor temperature reading using simplified Steinhart-Hart eqn */
float therm2temp(float r) {
 r/=(ADC_MAX+1)-r;       // resistance Rx relative to Rref
 return 1 / ( log(r * NTC_r)/NTC_B + 1.0/(ABS_T0 + NTC_t) ) - ABS_T0; }
```

La thermistance dont vous disposez (fig. 11a) présente une résistance de 100 kΩ à 25 °C. On la branche dans un «diviseur de tension»



NTC TDK B57164K104J K164 100
$R_{25} = 100\,\mathrm{k\Omega}$
$B_{25/100} = 4600 \pm 3\%\ \mathrm{K}$ **(a)**

x°C	R_x/R_{25}	α_x (%K)
0.0	3.6781	5.6
5.0	2.7944	5.4
10.0	2.1391	5.3
15.0	1.6507	5.1
20.0	1.2823	5.1
25.0	1.0000	5.0
30.0	0.78393	4.8
35.0	0.61822	4.7
40.0	0.49053	4.6

(b)

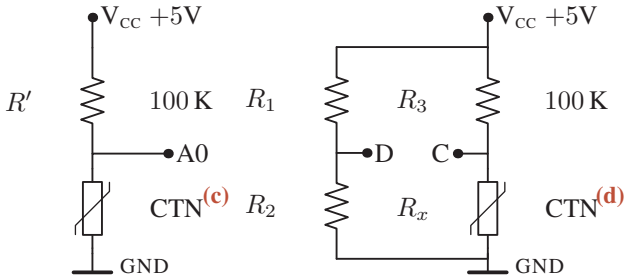


FIG. 11 – La thermistance (a), ses paramètres (b), et son montage dans un diviseur de tension (c) et sur le pont de Wheatstone (d).

comme montré dans la fig. 11c, où on utilise le V_{CC} de la carte, la resistance $R' = R_{25} = 100\,\mathrm{k\Omega}$, et l'entrée analogique A0 du μCU pour mesurer la tension à ses bornes. Notez que ce montage est déjà rencontré dans la sec. 2.2.4.

Diviseur de tension : On étudie des petites variations $\mathrm{d}R$ de la resistance $R = R_0 + \mathrm{d}R$ autour de R_0 en utilisant un diviseur de tension (cf la fig. 11c) avec une resistance R', et en mesurant u_R. Pour déterminer la valeur optimale de R', on considère le branchement en série (loi des mailles), exprime u_R,

$$U_{CC} = u_R + u_{R'} = I\,(R + R') \Rightarrow u_R = U_{CC}\,\frac{R}{R+R'} \approx U_{CC}\,\frac{R_0}{R_0+R'}\left[1 + \frac{R'}{R_0\,(R_0+R')}\,\mathrm{d}R\right],$$

et cherche à maximiser le coefficient devant $\mathrm{d}R$. On trouve que ce coefficient (sensibilité) est au maximum pour $R' = R_0$.

Si les variations de R sont larges : dans ce cas, pour déterminer $R = R_0 \pm \Delta R$, on mesure u_R et applique la formule

$$R = u_R\,u_{R'}^{-1}\,R' = u_R\,(U_{CC} - u_R)^{-1} R' = v\,(1023 - v)^{-1} R', \quad \text{où } 0 \le v \le 1023 \text{ est mesurée par ARDUINO.}$$

En remplaçant le potentiomètre de l'ancien montage par la CTN, on exploite le code de la sec. 2.2.4, qu'on modifie pour afficher les valeurs de T. Afin de tester vos mesures, calculez les valeurs de la résistance R_{CTN} attendues (cf. les données dans la fig. 11b référencées par son fabricant) et les valeurs de la tension correspondantes sur A0 pour les températures T entre 25 et 30 °C.

Pont de Wheatstone : Ce montage est employée couramment pour étudier des *petites variations* de résistance d'un capteur, tel qu'un sonde de température, une jauge de déformation, etc. La tension d'entrée $V_{AB} \equiv V_{CC}$ est appliquée à sa «diagonale», voir la fig. 11d. A l'équilibre du pont, les produits en croix des résistances sont égaux : $R_1\,R_x = R_2\,R_3$, et la tension $V_{CD} =: V$ est nulle. Si on déséquilibre légèrement les résistances, on trouve que

$$\frac{V}{V_{CC}} = \frac{k}{(1+k)^2}\left(+\frac{\mathrm{d}R_1}{R_1} - \frac{\mathrm{d}R_2}{R_2} + \frac{\mathrm{d}R_3}{R_3} - \frac{\mathrm{d}R_x}{R_x}\right), \quad \text{avec } k = \frac{R_2}{R_1} \Rightarrow \max_k \frac{k}{(1+k)^2} = \frac{1}{4} \text{ pour } R_1 = R_2.$$

On mesure $\epsilon := V/V_{CC} \ll 1$ à l'aide d'un amplificateur différentiel, ou mieux, d'un amplificateur de mesure (sec. 3.4).

Pour la thermistance sur le pont de Wheatstone (fig. 11d) avec les résistances constants $\mathrm{d}R_{1,2,3} = 0$ de même valeur $R' \approx R_t$, et vu que

$$\log R_x = \log R_t - B_t\,(x-t)\,(T_0+x)(T_0+t)^{-1} \approx \log R_t - B_t\,(x-t)\left[1 - (T_0+t)^{-1}\,(x-t) + \dots\right] \Rightarrow \mathrm{d}\big(\log R_x\big) = -B_t\,\mathrm{d}x,$$

on obtient $4\,\epsilon = -\mathrm{d}\big(\log R_x\big) = B_t\,\mathrm{d}x$. Dans un plus grande intervalle de $x \approx t$, nous pouvons exploiter la formule exacte

$$(T_0+x)^{-1} = (T_0+t)^{-1} + B_t^{-1}\log\frac{R'}{R_t} + B_t^{-1}\log\frac{1-2\,\epsilon}{1+2\,\epsilon}\,. \tag{3.1}$$

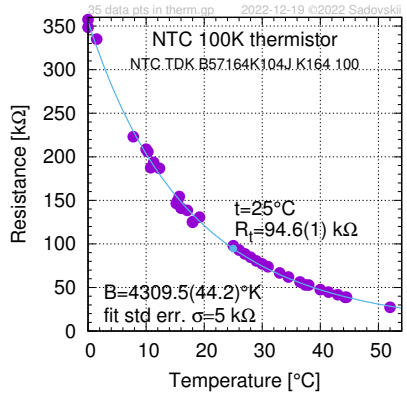


FIG. 12 – Calibration d'une thermistance CTN

Calibration d'une thermistance : Vous utilisez la CTN sur la fig. 11a, trois resistances $R_{1,2,3}$ de 100 kΩ, deux multimètres pour mesurer V et V_{cc}, un thermomètre, un calorimètre muni de deux résistances de chauffage en série et un mélangeur (le matériel de TP mesures calorimétriques), la source de tension DC à deux sorties, dont une simple pour le chauffage et une stabilisée pour le V_{cc} de 5 V, et les glaçons. La thermistance est protégée hermétiquement (vernis, peinture) pour pouvoir être plongée dans l'eau sans provoquer le court-cirquit. Elle est branchée dans le pont de Wheatstone (fig. 11d), un diviseur de tension (11c), ou, tout simplement, au bornes d'un ohm-mètre. Vous chauffez et refroidissez l'eau dans le calorimètre pour prendre les mesures de V dans l'intervalle de température $x = 0 \ldots 50$ °C avec votre ARDUINO. Vous pouvez également mesurer les valeurs de R_x directement avec l'ohm-mètre et les tabuler. Par la suite, vous vérifiez l'eq. (3.1) avec les valeurs de B (±138°) et R_t de réference dans la fig. 11a. Si cela est nécessaire, vous ajoustez (re-calibrez) ces valeurs ; voir la figure 12 ci à gauche.

3.2 La photo-résistance

La conductivité d'un matériau et donc sa résistance, est fonction du matériau lui-même mais également de son environnement : la température comme dans le cas de la thermistance (sec. 3.1) ou encore de la luminosité. C'est cette propriété que nous utilisons ici avec la photo-résistance. La luminosité ambiante peut être mesurée en lux. 1 lux correspond à un niveau de lumière en pleine nuit et 1000 lux

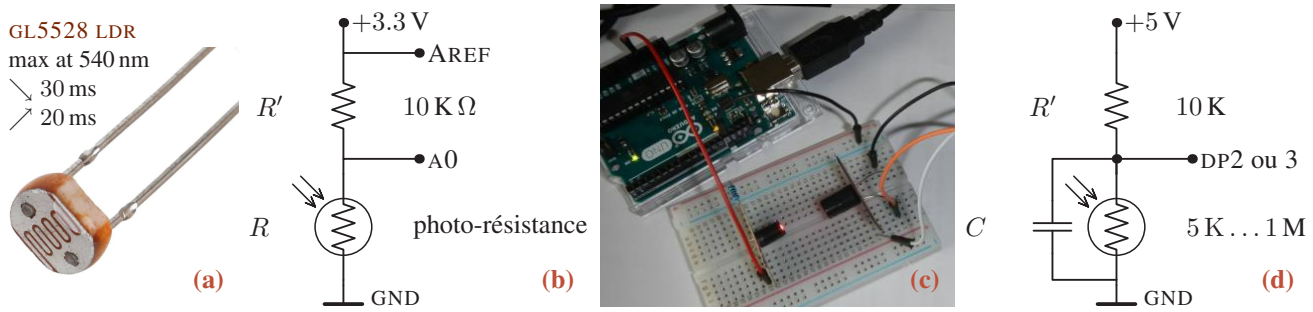


FIG. 13 – La photo-résistance (a) ; cablage en «diviseur de tension» avec V_{cc} de 3.3 V (b) ; mesure de R_{min} (c) ; comme capteur digital (d).

en pleine journée. La valeur de la résistance R d'une photo-résistance LDR suit une loi exponentielle en fonction de la luminosité L :

$$R(L) = R_0\, L^{-k}, \quad \log R = \log R_0 - k\, \log L, \quad \text{où } R_0 > 0 \text{ et } k > 0 \text{ sont des constantes.}$$

Ainsi la courbe $\log R(L)$ est une droite avec une pente négative si bien que la valeur de la résistance diminue avec L depuis quelques centaines de kΩ ou même 1 MΩ dans l'obscurité jusqu'à 10-20 kΩ à 10 lux ou quelques centaines d'Ω en plein jour, ceci, bien évidemment, en fonction des modèles. Comme les thermistances, les photorésistances sont le plus souvent utilisées dans un pont diviseur de tension (fig. 13). Notez que pour gagner en stabilité, on utilise la source interne de 3.3 V comme V_{cc} (plus filtré et plus stable que les 5 V fournis par l'ordi sur son port USB, cf la sec. 2.3) pour alimenter la LED et surtout le diviseur de tension de la photo-résistance $R + R'$. Pour R' de 10 K, les valeurs typiques attendues sur A0 sont entre 0.9 V_{cc} et ≤0.5 V_{cc}.

> **La source de 3.3V :** Les UNO et MEGA possèdent un régulateur linéaire séparé de tension 3.3 V/150 mA qui n'est pas utilisé par la carte et peut servir, par exemple, pour alimenter les capteurs, ou autres modules utilisant cette tension.

Alternativement, attachée à un pin digital, la photo-résistance sert comme un capteur «ON/OFF» de la présence de lumière dans des nombreuses applications style «photo cellule» ou «passerelle encodeur» dont le déploiement ressemble celui d'un bouton (sec. 2.4).

3.3 Capteur de force et poids

Les capteurs de force produisent des petites variations de résistance et sont montés typiquement sur un pont de Wheatstone (cf. sec. 3.1). Ce dernier est alimenté (dit excité) par une tension $u = V_{cc}$ et sorte une petite tension δ due au déséquilibre des résistances (on dénote les connexions $E_\pm$ pour l'excitation du pont, et $S_\pm$ pour la sortie/signal).

> **Jauges de contrainte résistives** de capteurs de force (dit *load cells*) sont caractérisés par leur sensibilité $s = \delta_{max}/u$, dont les valeurs typiques restent autour de 1 à 3 mV/V. Prenons $u = 3$ V. Dans ce cas, une cellule de 2 mV/V chargée à son poids maximal rend seulement 6 mV. Pour poids max de 50 kg, cela done 120 μV/kg ou 8 g/μV.

La fig. 14 ci-dessous montre le schéma standard de branchement de 4 capteurs de force à 3 fils. Chaque capteur de ce type possède deux résistances de, typiquement, 1K en série, un fil donnant leur milieu, deux autres les extrémités. On trouve celui du milieu (rouge dans

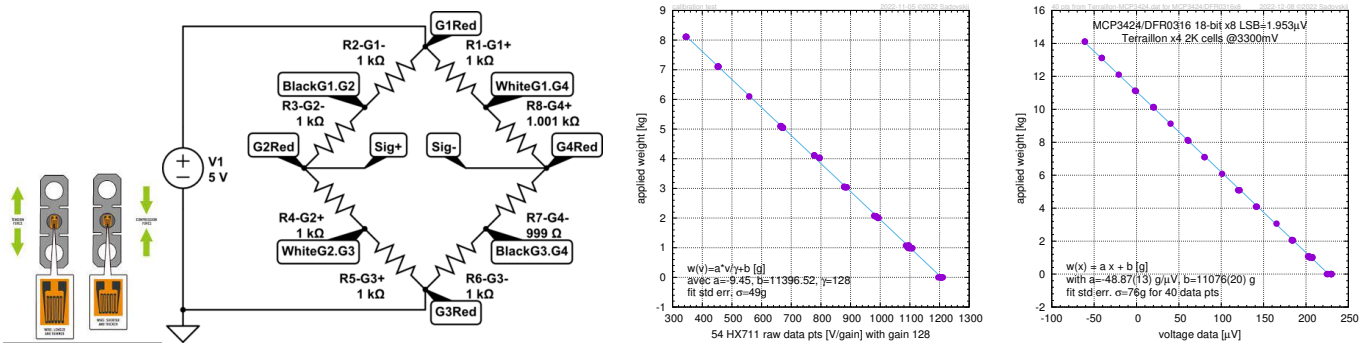


FIG. 14 – Montage de 4 cellules de force à 3 fils et leur calibration

la fig. 14) à l'aide d'un ohmmètre. Les deux résistances sont différents : soit l'une est fixe et l'autre est active (jauge de contrainte), soit les deux sont actives, une en *compression* et l'autre en *élongation*, avec les signes de leur ΔR opposés. Les couleurs des fils distinguent ces résistances. Ceci explique pourquoi on les branche «en cercle» et utilise les 4 fils du milieu comme les entrées/sorties du pont. La résistance du pont est de $\approx$2K.

Pour mesurer la très faible sortie différentielle $S_\pm$ (de quelques mV) avec une précision nécessaire, deux cartes/cirquits peuvent servir : HX711 (on la trouve souvent utilisée dans les balances) et MCP3424. Les deux possedent un ampli différentiel, un ADC, et communiquent leur données sur une ligne série. La carte HX711 propose en plus son propre alimentation stabilisée de 4.2V pour les $E_\pm$ du pont. La carte possède un ampli de $\times$64 ou $\times$128, et l'ADC de 24-bit. Cependant, le bruit est tel que cela revient plutôt à 16-bit «noise-free»[21]. La plage d'entree pour $\times$128 est de 20 mV. Cote interface, on la trouve un peu rudimentaire. HX711 peut rendre une précision de 1 g avec les 4 cellules à 50 kg provenant d'une «balance de la salle de bain» qu'on trouve facilement ou on récupère d'un vieux appareil, mais on constate le problème de stabilité en usage continu : au fil de temps qu'elle reste allumée, sa ligne de base flotte.

Le circuit MCP3424 possède un vrai interface I^{2}C (donc il s'attache facilement aussi à un RASPBERRY PI). Du point de vue des applications, il est plus versatile, mais on a besoin d'alimenter le pont, en utilisant, par exemple, les 3.3V stabilisés de ATMEGA328. Son ampli à gaine $\times\gamma$ de $\times$8 (maxi), $\times$4, ou $\times$2, son ADC va jusqu'à 18-bit, et la plage d'entrée est de $V_{\max} = \pm 2048/\gamma$ mV, donc 256 mV en $\times$8. On peut facilement interpréter les données de MCP3424 en mV (μV et même nV). Elles correspondent aux valeurs de multimètre avec 10 μV d'écart systématique (pb impédance ?). Pour un LSB[22] de 1.95μV, pas pire ... L'autre cote attractive de MCP3424 est son multiplexeur à 4 canaux integré. Ainsi, dans le cas de TP Clément-Désormes (sec. 4.4), par exemple, cela permet de mesurer Δp et ΔT avec la même carte. Le problème pour les applications de MCP3424 est, dans des certains cas, sa résolution insuffisant (face aux bruits). Ainsi, selon la courbe de calibration dans la fig. 14, la sensibilité est de 50 g/μV, donc c'est logique de n'avoir que 100 g de précision avec un LSB de 2 μV ! Le cumule des échantillons ne semble pas d'aider enormement. En plus, si la ligne de base de MCP3424 reste assez stable, qques LSB y flottent quand même... Ce que semble logique est d'essyer placer un pre-ampli d'une gaine de 10..100 devant l'entrée $S_\pm$ pour remonter de 1 a 10..100 mV et exploiter ainsi tout la plage de la carte.

3.4 Amplificateurs opérationnels

ARDUINO ne posséde que l'alimentation positive $V_{\rm CC}$ (dit «single rail») et ne mesure directement que les signaux analogques V positives et inférieures à son $V_{\rm CC}$. Par ailleurs, avec son ADC de bord, la précision varie entre 1 mV si on utilise la référence interne $V_{\rm REF} \approx 1$ V et 10^{-3} $V_{\rm CC}$. Ainsi on ne peut pas mesurer directement les signaux faibles à signe inconnu comme c'est souvent le cas dans les mesures sur le pont de *Wheatstone* (sec. 3.1). Les amplificateurs opérationnels (AOP, ampli-op), étant indispensables pour conditionner de tels signaaux, font fréquemment une partie essentielle de la chaîne d'acquisition. Il existe un nombre des cirquits intégrés AOP qu'on puisse déployer avec ARDUINO en monde «single rail», i.e., avec une seule $V_{\rm CC}$ de $+5$ ou $+3.3$ V et *sans alimentation negative*. Ainsi John Errington nous recommende un MCP6001/2/4 à technologie CMOS ou si non—les bons vieux LM324 et LM358 à technologie bipolaire. Les ampli-op's sont étudiés en cours d'électronique, on trouve également plein des informations et tutoriels sur internet. Un bref sommaire de leurs applications typiques en mode courant continu est donné ci-dessous.

Des faibles signaux électriques issus de capteurs de mesure sont traités à l'aide d'un amplificateur de mesure;[23] qui est constitué par plusieurs ampli-op's permettant d'augmenter l'impédance d'entrée et le taux de réjection du mode commun de l'ensemble.

Amplificateurs Pour mesurer les faibles différences V avec une précision elevée, il existe des cirquits integrés et des mini-cartes «breakout» tout prêtes, notamment HX711, ADS1115, ou encore MCP3424, qui combinent un INA avec réduction de bruit (et souvent à gaine programmable) avec un ADC à une résolution programmable et plus importante que celle de ARDUINO, et parfois avec un commutateur (multiplexer) 2$\times$4, et qui communiquent par I^{2}C.

[21] Pour un signal en courant continu (DC), noise-free bits (noise-free resolution) $= \ln(\text{FS/p-p})/\ln 2$, où FS (*full signal*) est la plage du signal et p-p (*peak-to-peak*) est le bruit crête-à-crête qu'on peut estimer avec l'ecart type (*rms noise*) σ du signal comme $6.6\,\sigma$.

[22] La valeur de *least significant bit* (LSB) pour une résolution de r-bit et gaine γ est donnée par $2^{1-r}\,|V_{\max}|/\gamma$

[23] ou amplificateur d'instrumentation, appelé en anglais *Instrumentation Amplifier*, *in-amp*, ou encore INA

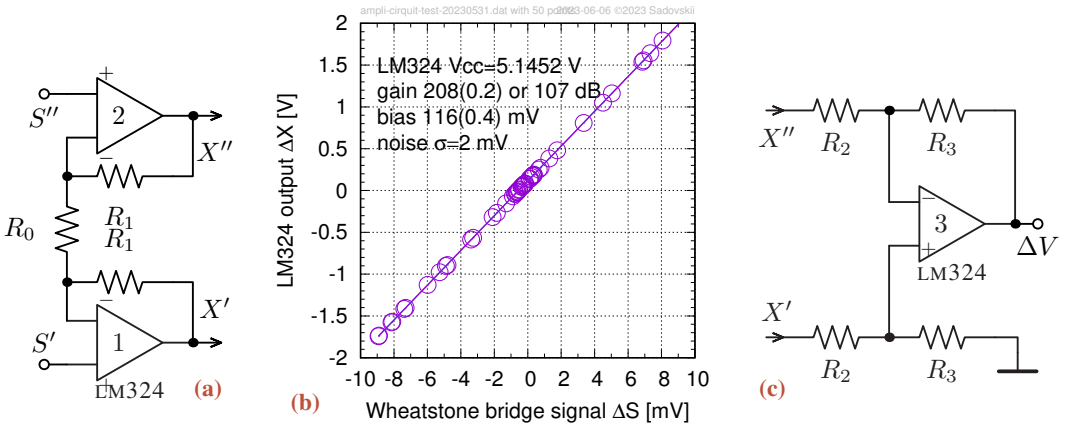


FIG. 15 – Amplificateur de mesure : (a) partie tampon, (b) résultats de sa réalisation (voir texte), et (c) partie differentielle.

Un amplificateur de mesure (INA) est présenté dans la fig. 15. Soit (S', S'') les tensions au bornes de sortie d'un pont de Wheatstone, avec la partie (mode) commune S et la différence ΔS, spécifiquement

$$S' = S + \frac{\Delta S}{2}, \quad S'' = S - \frac{\Delta S}{2}, \quad \text{où} \quad S = \frac{S' + S''}{2} \quad \text{et} \quad \Delta S = S' - S'' ,$$

qu'entrent dans cet INA. Idéalement, à la sortie de sa parie tampon (fig. 15a) on expecte les signaux

$$X' = S + g\,\frac{\Delta S}{2} \quad \text{et} \quad X'' = S - g\,\frac{\Delta S}{2} \quad \text{avec le gaine en mode différentiel} \quad g = 1 + \frac{2\,R_1}{R_0} .$$

En d'autres mots, alors que la différence ΔS est amplifié g fois et $X' - X'' = \Delta X = g\,\Delta S$, le mode commun S reste inchangé.

Réalisation : Dans la figure 15b on voit les résultats d'une réalisation de ce montage sur la base d'un LM324 avec $R_1 = 1\,\text{M}$, et $R_0 = 10\,\text{k}$. Ici, l'ampli-op et le pont de Wheatstone ont été alimenté par V_{CC} de $5.1452(2)\text{V}$ provenant d'un port USB du laptop LENOVO X230 en mode dormant. Les mesures de ΔS et ΔX ont été effectuées (en c.c.) respectivement par les TRMS-millimètres GOSSEN METRA 29S à grande précision de $1\,\mu\text{V}$ et TEKTRONIX DMM916 à précision de $0.1\,\text{mV}$, plus standard mais suffisant pour ΔX. La stabilité de la source était approximativement $2\,\mu\text{V}$, celle de la sortie ΔX était d'ordre de $0.2\,\text{mV}$. On constate que l'ampli reste parfaitement linéaire et très peu bruyant dans l'intervalle d'au moins de $\pm 10\,\text{mV}$ avec les tensions de sortie ΔX dans l'intervalle de $\pm 2\,\text{V}$. La gaine $g = 208$ est très proche à la valeur théorique de 201, mais on observe un bias assez important de $116\,\text{mV}$. Celui-ci est dû aux imprécisions de résistances $R'_1 \approx R''_1 \approx R_1$ d'ordre de 1%, ainsi que aux imperfections de LM324.

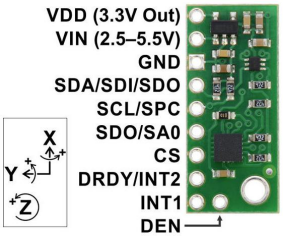
L'amplitude de ΔX à la sortie de la partie tampon de l'INA peut être déjà (comme dans notre exemple concrète) suffisamment grande pour être mésurée avec ARDUINO. Seulement, il nous faut effectuer la soustraction $X' - X''$. Pour cela, on utilise un troisième ampli-op en mode différentiel (fig. 15c) sortant une tension

$$\Delta V = \frac{R_3}{R_2}\,\Delta X\,.$$

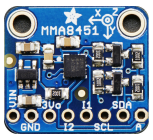
Evidemment, vu que cet ampli-op est en mode «single rail», il faut s'assurer impérativement, que $X' > X''$ et donc ΔV reste positive avec un marge. Et il faut également qu'elle reste inférieure à V_{CC}. En d'autre mots, on ne peux mesurer ainsi que les $0 < \Delta S < V_{\text{CC}}/g$. Si cela n'est pas le cas, on doit ajouter à X' une tension positive connue et fixe à l'aide d'un ampli-op supplémentaire (quatrième).

Estimons la précision du montage concret décrit ci-dessus et illustré dans la fig. 15. Assumant que la partie différentielle (fig. 15c) n'augmente pas le bruit et d'autres erreurs d'ordre de $0.1\,\text{mV}$ déjà présents dans la fig. 15b, et que la ΔV maximale est de 1V, l'imprécision de $1\,\text{mV}$ dans les valeurs de ΔV mesurés sera largement déterminée par la résolution du ADC de ARDUINO. On conclue qu'on pourra ainsi mesurer les signaux $\Delta S < 5\,\text{mV}$ avec une incertitude absolue de $1\,\text{mV}/g = 5\,\mu\text{V}$, i.e., à quatres chiffres significatifs maximum. Supposons par ailleurs qu'il s'agit des mesures de poids (sec. 3.3) entre 0 et $50\,\text{kg}$. Dans ce cas, l'imprécision de nos mesures monte à $50\,\text{g}$. Peut on l'améliorer ? Théoriquement si, car notre ampli-op sorte ΔX avec un bruit de $0.2\,\text{mV}$ (déterminé entre autre, par les instabilités de V_{CC} dans les μV) et notre incertitude peut potentiellement être $10\,\text{g}$. Cependant, une mesure de $1\,\text{V}$ avec l'incertitude de $0.1\,\text{mV}$ nécessitera un ADC à 20 registres du type HX711. C'est deux fois plus que possède ARDUINO UNO, et très probablement bien plus lent. En restant avec ARDUINO, on peut assurer l'incertitude maximale de $10\,\text{g}$ seulement dans l'intervalle de 0 a $5\,\text{kg}$ et avec un gain additionnel d'ordre $R_3/R_2 = 5$ au stade différentiel.

3.5 Accéléromètre et gyroscope



Les capteurs **MEMS!** à six degrés de liberté, tels qu'un MPU6050, combinent un accéléromètre 3D avec un gyroscope 3 axes. Il existe également des modules gyroscope seuls, tels que L3GD20H ci à gauche, aux caractéristiques similaires (16bit, plages `FS` de $\pm 250 \,/\, 500 \,/\, 2000$°/s). Pour utiliser ce module (au plus simple, en I²C), on installe la bibliothèque `l3g-arduino` (voir leurs commentaires dans `L3G.cpp` et `L3G.h` et l'exemple `l3g-arduino-master/examples/Serial/Serial.ino`) ; pour l'alimenter, on passe la sortie $+5\,V_{CC}$ de la carte ARDUINO (fig. 1b) et sa masse sur VIN et GND respectivement ; pour communiquer, on connecte[24] les broches SDA/SCL (données/horloge) du bus I²C (adresse[25] `0x6b` par défaut, ou `0x6a`, si SDO est mise à la masse). Attention, la carte L3GD20H ci à gauche, possède des resistances «pull-up» de 4.7 K entre ces lignes SDA/SCL et VIN, ce que puisse affecter le fonctionnement des autres cartes sur I²C. Ainsi, une combinaison de deux telles cartes descende la résistance «pull-up» du bus à 2.35 K. Notez que `enableDefault()` sélectionne la plage `FS` de ± 250°/s (degrés par seconde, ou dps). Sur cette plage et avec la résolution de 16 bit, on estime la précision du gyroscope comme $\text{LSB} = 2 \times \texttt{FS} / 2^{16} = 0.00763$°/s, soit 7.63 mdps. Pour plus de renseignement, on consulte la page 10 du datasheet de L3GD20H, où la sensitivité `So` de $8.75 \,/\, 17.50 \,/\, 70.00$ mdps/digit selon `FS` est indiquée. Donc une valeur «brut» (dit *raw*) de 114 corresponde approximativement à 1°/s, soit $\approx 114 \times \text{LSB}$.



On utilise également la petite carte d'accéléromètre 3-axes MMA8451 (plages de $\pm 2 \,/\, 4 \,/\, 8\,g$), qu'on branche, comme dans le cas du gyroscope ci-dessus, sur Vin/GND et SDA/SCL (addresse[25] I²C `0x1D` par défaut, ou `0x1C` si la broche A est connectée à la masse), voir ci à gauche. Le codage nécessite deux bibliothèques, `Adafruit_MMA8451` pour lire les données «*raw*» et `Adafruit_Sensor` pour les traduire en unités physiques (m/s^2). A noter, que ce capteur possède également une détection de ses inclinaison et orientation (par rapport au pesanteur) consultables avec `getOrientation()`. Les capteurs accéléromètre-gyroscope se trouvent couramment dans les téléphones mobiles ou tablettes ; quelques applications sont présentées sec. 4.11.

[24] Dans le cas d'un montage mobile, par exemple sur un pendule, souder les fils légers. Pour identifier les broches I²C sur la carte ARDUINO, voir la note 10.
[25] Pour détecter au démarrage toutes adresses actives sur le bus I²C, consulter appendice E.

4 Projets UO ARDUINO 2025



Creativity is just connecting things.
When you ask creative people how they did something,
they feel a little guilty because they didn't really do it, they just saw something.
Steve

Lors de la dernière séance TP ARDUINO, à la fin de la séance d'initiation, on se met en groupes de travail (de 2, 3, 4) et détermine un projet. On peut sélectionner un projet scientifique parmi ceux qui vous sont proposés, mais tout autre projet utilisant Arduino est possible. Il est donc indispensable d'étudier cette section, consulter de nombreux sites internet et toutes autres sources, échanger entre vous, avant la séance d'initiation (la première rencontre).

Vous devrez étudier votre projet d'un point de vue théorique et en discuter avec l'enseignant «mentor» pour commander éventuellement les éléments qui vous seront utiles pour son développement, idéalement à la fin de la 1ère séance.

Vous devrez réaliser le projet ou tout au moins aller le plus loin possible, en concertation avec l'enseignant responsable et présenter vos travaux à l'issue des séances de projets. Vous aurez la possibilité d'emporter vos montages pour travailler chez vous, en dehors de l'université. Nous vous rappelons que vous trouverez sur le réseau la majorité des informations qui vous seront nécessaires à la compréhension de votre projet. Vous aurez ensuite dans la majorité des cas à adapter un programme à votre cas particulier.

Parmi les projets que nous avons recherché (et donc nous pensons qu'ils seront à la fois faisables et intéressants dans le cadre de ce module) nous pouvons envisager les projets suivants.

Traitement des données Dans des certains projets de style «scientifique», on aura besoin de transférer un grand volume des données collectées par ARDUINO et les traiter dans un ordinateur (un PC, MAC, ANDROID). Sous LINUX ou MAC OS, le plus simple est de passer directement par un scripte `bash` pour récupérer les données via un port série dans un fichier texte : rien à compiler, installer etc, s'est un scripte ! Sous WINDOWS, il existe une procédure similare via un port `COM`.

Par la suite, les étudiants traiteront leurs données comme et avec quoi ils veulent. En particulier, on peut suggérer regressi. C'est un util interactif, un peu style tableur, qui peut avaler tout fichiers de données au format texte. Il est possible même de le coupler avec ARDUINO. L'ancien version de ce logiciel existe pour windows, il a ete utilise beaucoup à la fac et dans les lycees, mais ses versions actuelles (en `python`) n'existent que pour MAC OS, LINUX, et ANDROID.

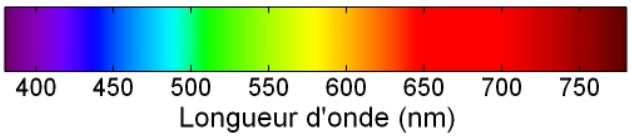
Cote ARDUINO, on exploite le programme `adcsampler` (voir l'appendice B). Il possède un protocole permettant de définir le nombre des échantillons, le taux, etc, et de lancer la collecte des données (sec. 2.5). Il est facilement modifiable selon le projet. L'example complet, avec les scriptes `bash` se trouve dans le topo réaction oscillant. Il est exploité aussi dans le colorimètre (sec. 4.1).

4.1 Colorimétrie

Nous combinons les expériences et le code développé dans les sec. 2.2, 2.5, et 3.2 pour construire et tester un colorimètre (spectrophotomètre monochromatique) permettant de détecter des molécules spécifiques et de mesurer leurs concentrations en temps réel.

4.1.1 Absorption de la lumière, loi de *Beer-Lambert*

Considérons un faisceau de lumière de jour traversant un milieux actif tel qu'une solution aqueuse *colorée*. La lumière de jour étant blanche, elle possède toutes les couleurs (toutes les longueurs d'onde λ) du spectre visible ci-dessous.



On conclut que le milieux retient ou *absorbe* la partie du spectre complémentaire à sa couleur : un objet transparent vert laissera passer le vert et arrêtera toutes les autres couleurs. Par ailleurs, il est clair que l'intensité I de la lumière absorbée diminue avec l'épaisseur l du milieux. La relation entre I et l est donnée par la loi *Beer-Lambert*.

Sous sa forme différentielle, cette loi prédit que la quantité de la lumière absorbée $\mathrm{d}I$ par l'épaisseur $\mathrm{d}l$ est

$$\mathrm{d}I = I(l+\mathrm{d}l) - I(l) = -\varepsilon\, C\, I(l)\, \mathrm{d}l\ . \tag{4.1a}$$

On constate que l'absorption est un phénomène forcé, provoqué, induit par l'intensité I de la lumière traversant le milieux : plus on a de la lumière, plus il y a de la lumière $\mathrm{d}I$ absorbée. Le coefficient de proportionnalité comprend naturellement la concentration C [Mol/L] des molécules actives et leur caractéristique individuelle ε [L/Mol/cm] appelée le *coefficient d'absorption* ou l'*absorbance*[26]. La forme (4.1a) s'applique aux petites épaisseurs $\mathrm{d}L \approx \Delta l$. Sinon, on doit utiliser la loi intégrale

$$\log I(l) - \log I(0) = -\varepsilon\, C\, l \quad \Rightarrow \quad I(l) = I_0 \exp(-\varepsilon\, C\, l)\ . \tag{4.1b}$$

[26] selon les domaines d'applications spécifiques, on rencontre aussi l'*absorptivité*, la densité optique du milieu, l'opacité ou l'extinction.

Cependant, pour des petites variations de $x = l\,C$, la forme (4.1b) est excessive et difficilement applicable, surtout en présence d'erreurs de mesure. Ainsi, si x varie entre $x_{\min}$ et $x_{\max}$, nous utiliserons plutôt une *linéarisation* de (4.1b)

$$I(l) \approx I_0\,\exp(-\varepsilon\,\bar{x})\big(1-\varepsilon\,(x-\bar{x})+\dots\big) \quad \text{avec } \bar{x}=(x_{\max}-x_{\min})/2. \tag{4.1c}$$

Enfin, dans une étude avec une épaisseur fixe et une concentration C qui est fonction de I, on réécrit (4.1c) sous forme

$$C(I) = a_0 + a_1\,I \tag{4.1d}$$

4.1.2 Régression linéaire

Considérons les mesures (x_i, y_i) avec $i = 1, \dots, N$ pour chaque valeur de x. Dans notre cas (sec. 4.1.1), y et x représentent l'intensité I et la concentration C. On estime les incertitudes de y_i égales à Δy_i (ou à ces écart types σ_i), et on suppose dans le cas de la *régression*, que les valeurs de x sont "exactes". Ainsi x est appelée *variable de contrôle*. On étude le phénomène décrit par la loi théorique $y = f(x;\alpha)$ avec des paramètres phénoménologiques à déterminer $\alpha = \alpha_1, \alpha_2, \dots$. Dans notre cas, il s'agit de l'approximation linéaire (4.1d) avec ses deux paramètres $\alpha = (a_0, a_1)$. En d'autres termes, nous cherchons à faire passer une *ligne droite* appelée aussi «courbe de calibration» par les points $(x_i, y_i) = (C_i, I_i)$ à partir de mesures de I_i pour des échantillons de concentration C_i connue (voir l'appendice F). Ces échantillons de concentration connue sont appelés les «standards». Bien entendu, leur nombre N doit être égale ou être supérieur au nombre des paramètres α (donc pour nous $N \geq 2$). Par la suite, en connaissant les α, nous pourrons déterminer la valeur de C pour toute valeur de I dans l'intervalle $I_{\min} \dots I_{\max}$ couvert par (4.1d).

4.1.3 Colorimètre avec ARDUINO

Nous utilisons une photo-résistance (sec. 3.2), le branchement et les contrôles de la LED (sec. 2.2.1), et les principes de l'échantillonnage ADC rapide aux intervalles de temps fixes (sec. 2.5) pour construire un colorimètre. Ce dispositif sera capable de mesurer des concentrations avec les périodes de 300 μsec (plus prudemment 500 μsec, voir sec. 2.5) et donc au taux maximal de 2-3 kHz. Les formules (F.1b) seront implémentées dans le code de calibration de colorimètre.

Matériel : une carte ARDUINO UNO et un ordinateur pour lui communiquer en TTY, une LED rouge générique (de λ approximativement 630 ou 660 nm), des résistances de 220Ω et $R' = 10\text{k}\Omega$, une photo-résistance de $R_{\min} = 3.1\text{k}\Omega$ exposée par la LED rouge, voir la fig. 13, et $R_{\max} \approx 0.35\text{M}\Omega$ sans lumière, un potentiomètre de 10kΩ en option. Carton, colle, scotch, feutre noir ...

Le principe : La LED et la photo-résistance sont montées sur le breadbord à une distance $\approx$ 15 mm (fig. 16). Elles sont branchés selon les schémas dans la fig. 2a et 13 avec VCC de 3.3V. Pour pouvoir commuter les +5V de la LED, on utilise la pin digitale 10 (voir `LED_PIN` dans le code). Le potentiomètre peut servir à adapter la valeur de la tension de référence ($V_{\max}$ anticipée sur la R) inférieure à 3.3V. La LED envoie sa lumière vers la R, éventuellement, à travers une cuvette. On évite toutes lumières parasites (caches).

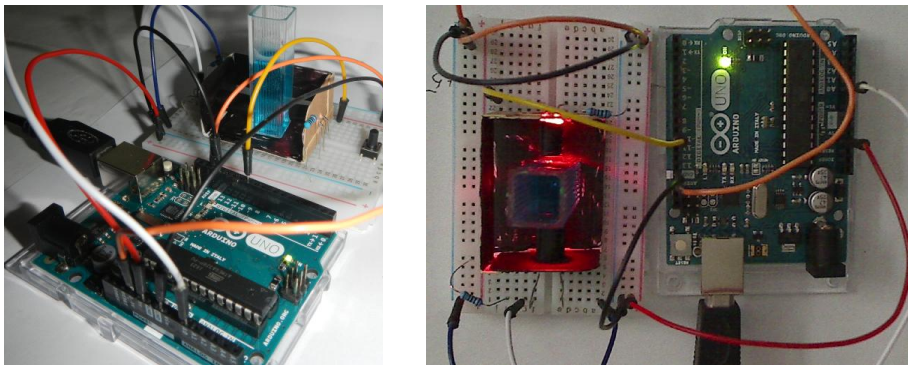


FIG. 16 – Montage du colorimètre avec $CuSO_4$ dans une cuvette.

Le code et l'interface : Une fois téléchargé dans le micro-contrôleur, le programme `~/Arduino/cmeter/cmeter.ino` (voir l'appendice D) permet d'opérer et de communiquer avec le colorimètre (voir également la sec. 2.5 et le code). Ainsi `A5N10` préconise d'afficher deux mesures de 5 échantillons moyennés chaque (lisez le code !). Le taux de l'échantillonnage est réglé par la période `T` en μsec. Les mesures sont démarrées par `S`. On allume/éteint la LED avec `X`. Les points de calibration successifs sont introduits par `d`, et on peut réinitier l'ensemble de ces points avec `D`. Pour effectuer la calibration et déterminer les valeurs de a_0 et a_1 on utilise `C`. Par exemple, la commande `d500S` ajoute le point dont la valeur de concentration est 500. A noter, qu'on cible l'intervalle d'affichage à 4 chiffres. Pour cela on implémente des facteurs dans l'échelle de C. Ainsi 0.5M devient 500 mM ou même 5000 10^{-4}M. Par défaut, les valeurs 0 ... 1023 de l'ADC sont affichées. Pour les convertir en concentration C, utilisez `c`. Par ailleurs, la commande `r` permet d'afficher la valeur de la résistance de R.

4.1.4 TP Mesures de concentration

Objectif : examiner la relation entre l'absorbance et la concentration du sulfate de cuivre (II) $CuSO_4$ dans une solution aqueuse (la loi de *Beer*) et déterminer la concentration dans des échantillons de concentration inconnue. Sur le spectre d'absorption d'une solution de 0,5M de $CuSO_4$ dans l'eau à température ambiante, on peut voir que l'absorption est maximale à la longueur d'onde $\lambda_{max} = 780$ nm, avec un coefficient d'absorption molaire ε de ≈ 12.5 mol^{-1}cm^{-1} ; à 635 nm il est de 2.8 mol^{-1}cm^{-1}.

Matériel : des petites cuvettes plastiques génériques pour les colorimètres et spectrophotomètres (4 ml, $12.55\times12.65\times44.55$ mm) avec couvercles hermétiques pour les solutions standardisés de $CuSO_4$ de la concentration n [10^{-1} M] avec $n = 0, 1, 2, 3, 4, 5$ et 2-3 solutions de contrôle.

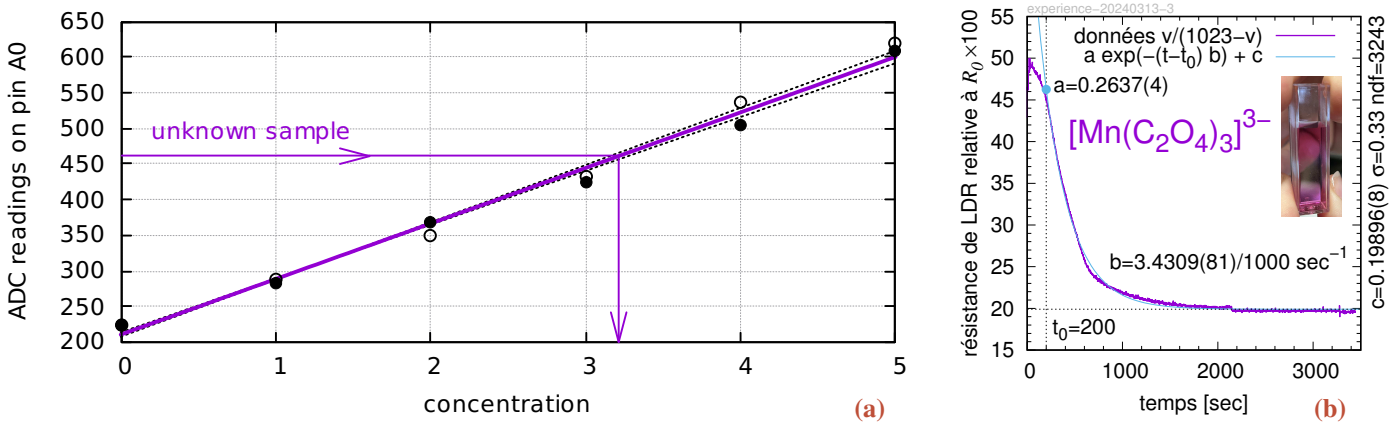


FIG. 17 – Etudes colorimétriques : (a) courbe de calibration de colorimètre ; (b) observations en temps réel.

Mode opératoire : Calibrez votre appareil avec les solutions standards. Représentez les données de calibration et la courbe sur un graph (tableur ou papier millimétrée, voir la fig. 17a) et confirmez la linéarité (cf. appendice F) et le fonctionnement du code `linreg()`. Mesurez la solution inconnue et en déduisez sa concentration.

4.1.5 TP Mesures de la constante cinétique

Objectif : utiliser le mini-colorimètre à la base de ARDUINO UNO dans la sec. 4.1.3 pour étudier *en temps réel* la dissociation du Mn(III) *tris-oxalate* (avec une durée des observations entre 10 min et 1 h) et déterminer la constante de la vitesse de cette réction ainsi que, optionnellement, modéliser sa dépendance en température (pour différentes agitations et intensités de la lumière ambiante).

Exposé : Le Mn(III) *tris-oxalate* $[Mn(C_2O_4)_3]^{3-}$ est un anion complexe unstable, qu'on peut obtenir[27] en ajoutant la solution du permanganate de potassium (violet) à une solution aquatique incolorée du sulfate de manganèse et de l'acide oxalique :

$$4\,MnSO_4 + 15\,HOOC{-}COOH + KMnO_4 \rightleftharpoons 5\,H_3[Mn(C_2O_4)_3]^{3-} + 3\,H_2SO_4 + 4\,H_2O, \tag{4.2a}$$

$$4\,Mn^{2+}(aq) + 15\,(COO^-)_2(aq) + MnO_4^-(aq) \rightleftharpoons 5\,[Mn(C_2O_4)_3]^{3-}(aq) + 4\,H_2O. \tag{4.2b}$$

Cet anion est photosensible, et étant exposé à la lumière du jour ($h\nu$), il se décompose rapidement même aux temperatures ambiantes

$$[Mn(C_2O_4)_3]^{3-}(aq) \xrightarrow{h\nu} 2\,MnC_2O_4(aq) + 3\,H_2C_2O_4(aq) + 2\,CO_2\uparrow.$$

Bien que son mécanisme précis est assez compliqué, dans une première approximation, la dissociation suit la loi cinétique des réactions d'ordre 1. Elle est relativement lente, parce que une énergie supplémentaire (lumière) est requise pour exciter son état de transition. En plus, la stabilité thermique de $[Mn(C_2O_4)_3]^{3-}$ est baisse, et la réaction est affectée par la température de façon très significative.

Mode opératoire : on suive le mode opérationnel simplifié[28] Deux solutions, acide oxalique (sans additif de $MnSO_4$) et permanganate de potassium à 0.02 M, 1.5 mL chaque, sont ajoutées (de préférence, sans exposer excessivement à la lumière) dans une cellule de photométrie (acrylique, $1\times1\times4$ cm, voir la fig. 17b), laquelle est secouée, et placée dans le colorimètre. Une solution marronne contenant le Mn(III) *tris-oxalate* se forme. Notez que le maximum d'absorption de $KMnO_4$ est à 450–470 nm. Donc pour la colorimétrie, on

[27] P. A. Nikolaychuk et M. M. Vayner, The decomposition of Tris-(Oxalato)-Manganate(III) complex ion as the reaction suitable for the laboratory practice on chemical kinetics, *Int. J. Therm. Chem. Kin.* **1**(2) 50–9(2016) ; Practical exercises in physical chemistry. Photometry. I. Shenderovich, ed, Freie Universität Berlin (2006).

[28] Pour tout réalisations de ces réactions, se rapprocher à Mme *Caroline Duhr*, responsable des laboratoires de chimie organique et biochimie à ULCO, Calais, qui les a testé et simplifié en 2024 (projets L2 S4). Cependant, on note que l'ajout de $MnSO_4$ comme une source des cations de Mn^{2+}(aq) reste indispensable pour former le *tris-oxalate* selon (4.2). En absence de $MnSO_4$ l'interpretation des résultats est plus complexe.

doit utiliser soit une LED bleue (assez rare), soit une blanche avec un filtre bleu, comme on fait pour la réaction oscillant. Par la suite, les mesures (avec un taux de 1 ou 0.5 Hz) démarrent immédiatement. La solution devient graduellement (15 à 45 min) incolorée. Optionnellement, plaçez votre colorimètre et les solutions dans une étuve thermostatique (sur une plaque chauffante du labo) pour contrôler et varier leur températures.

Exploitation des mesures : Selon la loi d'absorption sous sa forme linéarisée (sec. 4.1.1, confirmer les conditions de linéarité), la concentration $c(t)$ de Mn(III) *tris-oxalate* et proportionnelle à la résistance $R(t)$ de la photorésistance (LDR). Cette dernière est donnée par la formule $v/(1023-v)$ de diviseur de tension dans la fig. 13b (cf la sec. 3.1, le cas d'une large $\Delta R/R$), où v est le signal sur le pin A0 digitalisé par l'ADC de votre ARDUINO. On ne s'interesse qu'au taux de décroissance b [sec^{-1}] de $c(t) \propto R(t)$ (surtout que la concentration initiale est difficilement contrôlable, et la constante R' peut être ajustée afin de maximiser la plage de réponse de colorimètre). Pour déterminer b, modélisez $R(t)$ par la fonction $a\exp(-(t-t_0)\,b)+c$, où $t_0 \geq 0$ représentent, respectivement, la période initiale à ignorer et la «ligne de base», i.e., la limite $\lim R(t)$ pour $t \to \infty$ (cf la fig. 17b). Au premier temps, estimez t_0 et c en traçant le graphe de $R(t)$. Par la suite, tracez $\log(R(t)-c)$ pour $t \geq t_0$ et trovez son coefficient directeur.

4.1.6 Développement

On peut envisager des projets d'amélioration de certains éléments de notre petit appareil, notamment

Op Amp Afin d'exploiter mieux la plage de résolution de l'ADC de μCU et d'augmenter ainsi la précision, insérer un ampli opérationnel pour (a) déduire la tension de base $V_{\min}$ (aka *debias*) et (b) amplifier le signal par facteur de $\times 2$ ou même 3.

EEPROM Sauvegarder les résultats de calibration (et certains autres réglages) dans la mémoire non-volatile de ARDUINO pour éviter leur perte. Au présent, l'appareil doit être recalibré chaque fois que il est rebranché (mis sous tension) !

Ecran Ajouter un écran (et quelques boutons de contrôle) pour afficher la concentration instantanée en toute autonomie.

GUI Développer une application sur PC pour communiquer avec l'appareil en utilisant Java ou Tcl/Tk.

Bluetooth Installer une carte («module») bluetooth et développer la même application pour ANDROID.

Température Ajouter une thermistance (sec. 3.1) pour contrôler la température des échantillons.

Spectre Étudier les possibilités d'utiliser des LED multicolores RGB et obtenir ainsi un «spectrophotomètre» à trois λ.

Enregistrement en temps réel afin d'étudier, par exemple, la cinétique des réactions chimiques, voir la sec. 4.1.5.

4.2 Spectrophotomètre (DS)

En comparaison avec le colorimètre (sec. 4.1.3), un spectrophotomètre possède un élément optique dispersif, spécifiquement un réseau ou un prisme, qui sert à séparer le faisceau de lumière sortant de la cuvette d'échantillon selon la longueur d'onde (couleur). Ainsi, dans les anciens spectrophotomètres, on trouve un *monochromateur* constitué d'un réseau qui turne pendant l'enregistrement du spectre ; certains possèdent également un dispositif mécanique pour effectuer des mesures automatisées de plusieurs échantillons. On peut envisager de connecter ARDUINO à un tel appareil. Comme alternative, on peut utiliser un PDA, tel qu'un «vieux» TSL1402R[29] à 256 diodes, un IC-LFL1402, ou un réseau moderne TCD1304DG à haute résolution. Tout ces ICs mesurent une tension analgoque sur chaque diode (= pixel) qui peut être accédée et mesurée rapidement en série par le μCU[30]. L'utilité de ces systèmes est dans la possibilité d'observation le spectre en temps réel.

4.3 Mesure de la chaleur massique de l'eau et de la chaleur latente de fusion de la glace (DS)

On se propose d'informatiser le TP mesures calorimétriques de L2 physique-chimie. Les mésures de la température sont détaillées dans la sec. 3.1. L'échantillonnage en temps réel est décrit dans la sec. 2.5, d'où on récupère le code pour prendre les mesures de température. Le courant I (on se renseigne sur le IC ACS758 et/ou les mesures ampèremétriques avec ARDUINO) et la tension U_{CC} (avec le ADC) de chauffage sont stabilisés et peuvent être mésurés juste une fois au départ et une fois à la fin de l'intervalle de temps des mesures de la température. On improvise un petit mélangeur (un électro-aimant vertical reglée par le μCU en PWM à travers certainement d'un mosfet). Finalement, on mésure le poids avec un jauge de contrainte (pont de Wheatstone) et la carte HX711 (voir sec. 3.3). A noter que cette carte possède deux entrées A et B, dont une peut servir pour les mésures de température et l'autre pour le poids. Pour l'interaction avec un ordi (la RASPBERRY 3 sous LINUX dans la classe que vous utilisez pour vos études de ARDUINO à Calais) on s'inspire du scripte pour `bash` dans le TP «réaction oscillante» et on traite les données par l'informatique (MAXIMA, GNUPLOT).

4.4 Expérience Clément–Désormes (DS)

Il s'agit d'un autre TP de L2 physique-chimie qui a pour but la détermination de la constante adiabatique γ de l'air. Dans ce cas, l'utilisation de l'μCU ARDUINO est même plus interessant que dans la sec. 4.3, car elle permet des observations du procès en fonction de temps. Ce qu'est impossible autrement. On équipe la bonbonne Clément–Désormes avec les capteurs (différentiels et totales) de la

[29] voir *Marian Janek*, Refraction, diffraction and energy losses studies using Arduino and Raspberry Pi, J. Phys. Conf. Ser. **2339**(1) :012004 (2022).
[30] voir, par exemple, *Daniele Murra et al*, Interfacing Arduino boards with optical sensor arrays, Sensors **23**(12)9787 (2023).

pression p et de la température T (sec. 3.1) et on récupère leur données en temps réel (sec. 2.5) pendant la période de l'établissement de la température après la détente. Pour mesurer précisément les petites variations de T, on utilise le montage en pont de Wheatstone (fig. 11d, cf. sec. 3.3). On utilise également un capteur de pression différentielle MPXV5010DP, MPXV7002, ou encore un simple MPX53DP pour observer directement la surpression Δp dans la bonbonne.

4.5 Pèse-ruches à distance

On se propose d'échantillonner l'évolution de poids d'une ruche en temps réel. Ceci serve aux apiculteurs pour déterminer si les abeilles sont en train de produire le miel (et donc tout va bien) ou de le consommer (et il faut les alimenter en supplément avec un sirop du sucre). Pour une ruche éloignée (en montagne...), les échantillons peuvent être envoyées par SMS en permettant ainsi à l'apiculteur de ne pas se déplacer inutilement. Une ruche pèse entre 10 et 50 kg ; son poids varie en fonction de la présence des abeilles dont le poids peut aller jusqu'à 3 kg (soit environ une colonie de 30 000 abeilles). Les changements nets journaliers aillent de 100 g à 1 kg. Ceci suggère un échantillonnage une fois par heure, cumulé en 24 h, référencé une fois, par exemple au minuit (quand tout les abeilles dorment dans la ruche), et reporté par SMS. On utilise, évidemment, les capteurs de force. La solution économique serait d'employer les capteurs à 3 fils (formant un démi-pont de Wheatstone, sec. 3.3) qu'on trouve couramment dans les pieds des pèse-personnes de commerce. Alimenté par un accu au Pb 12 ou 6 V ou un accu LiPo de 4.2 V, avec, si on veut, un système de recharge solaire, l'appareil doit économiser sa consommation. Ceci implique le mode «POWER DOWN» interrompu par un horloge RTC (sec. 2.4.2). On y ajoute un bouton, une carte GSM style SIM800L, et la gestion d'alimentation (sec. 2.7) des capteurs activée elle aussi que pour les courts moments des mésures. Notons que la carte GSM permets de déterminer l'heure et la date au démarrage et que la carte RTC possède un thermomètre qui peut être utile pour compenser la variation de la sensibilité des capteurs en fonction de °C. En option, on ajoute des autres capteurs environnementaux.

4.6 Capteur de vitesse de vent par sonde Pitot (DS)

On propose de fabriquer un capteur autonome de vitesse relative (résistance) du vent sur la base de ARDUINO et d'une sonde (tube) Pitot (Henri Pitot en 1732, voir le cours de hydrodynamique). Ce capteur peut être déployé dans une voiture (les voitures de F1 possèdent une sonde de ce type), drone, etc. L'objectif minimum sera d'avoir un prototype capable de mesurer la vitesse avec une précision et une stabilité raisonnables et les renvoyer par la connexion série/usb et/ou sur un écran lcd. En option, on peut envisager une connexion via bluetooth vers des smartphones et une application pour piloter le capteur. La sonde peut être trouvée d'occasion ou fabriquée au laboratoire. Pour les mesures, il s'agit de piloter plusieurs capteurs de pression (statique, dynamique) et de température, utiliser leur données pour calculer la vitesse, puis tester et calibrer l'appareil. Pour les plus avancés, une correction pour la vitesse réelle mesurée par un GPS peut y être ajoutée.

4.7 Capteurs environnementaux

Dans différentes applications d'évaluation de la pollution, d d'installations chimiques (fuites), de combustion ou autres, on cherche à mesurer les concentrations des gaz (NO_2, CO, CO_2, O_3, SO_2 ...) ainsi que la pression, la température et les particules fines. On propose de développer un appareil portatif capable d'effectuer et d'enregistrer ces mesures (mode traceur) en temps réel et en autonomie pendant une certaine période du temps. Ce genre d'appareil peut être monté sur un drone ou un robot pour travailler dans les zones dangereuses, voir inaccessibles aux hommes. Ce projet est naturellement d'importance pour les sites de Calais et Dunkerque.

Ce projet ainsi que le projet Pitot fait partie du projet initial de BQE 2017. On envisage de piloter plusieurs capteurs avec un enregistrement des données sur une carte CF en temps réel, par exemple chaque seconde. En option, on peut envisager une communication wifi ou RF, ou même GSM.

Si la partie électronique n'est pas trop lourde, une calibration des capteurs peut être envisagée (en collaboration avec les chimistes ? sur Dk ? chambre des essays). On peut avoir plusieurs binômes faisant différents capteurs et autres parties de l'appareil, notamment un système d'enregistrement universel (avec aussi données GPS) à déployer ici et dans le capteur Pitot.

4.8 Viscosimètre avec un capteur inductif de déplacement/vitesse (DS)

Pour mesurer la viscosité d'un liquide (glycerol) on utilise une petite bille en acier du rayon r descendant le long de l'axe d'un cylindre en plexiglas de rayon $\rho \gg r$ et de longueur $l = 50$ cm rempli du liquide et placé verticalement. Ce viscosimètre fait parti des TP en L1 et L3. Le régime stationnaire (vitesse constante) s'installe après 10 premiers centimètres de descente. On détecte le passage de la bille à 10, 20, 30, et 40 cm ; le temps de passage entre les points du repère donne la vitesse stationnaire qui dépend de la viscosité du liquide (loi de Stokes). A noter que le même dispositif peut être testé (sans liquide) pour mesurer la constante gravitationnelle g.

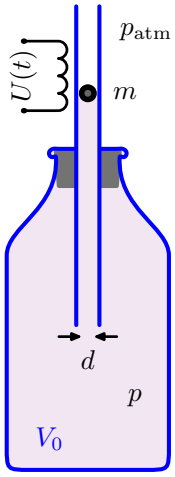
La détection utilise le principe d'un détecteur de métal. On entoure le cylindre par un câble fin isolé en faisant 8 tours à chaque point de repère (donc $N = 32$ en tout) et on mesure le changement d'inductance L de l'ensemble provoqué par le passage de la bille. Pour $N = 32$ tours de câble et $\rho = 25$ mm (5m de câble) on obtient

$$L \approx 5\, N^2\, \rho = 128\, \mu\,\text{H}.$$

Pour mesurer L en temps réel on passe des impulsions de courte durée (quelques μsec). On peut utiliser Arduino lui même pour donner les impulsions, voir aussi d'autres projets, et les capteurs industriels. Dans le circuit, le câble fait partie d'un filtre LR, où on doit avoir

au moins $R = 220\Omega$ pour protéger le circuit d'Arduino de surcharge. La résistance du câble étant de quelques Ω, donc bien inférieure à R. On peut ainsi anticiper le temps caractéristique du filtre $\tau = L/R = 0.58\mu$sec qui limite la durée utile des impulsions. On note que la fréquence du contrôleur ATmega326 est de 16MHz et son cycle de 0.06μsec est seulement 10 fois inférieur à τ. Ainsi pour former les impulsions de quelques μsec il est préférable d'utiliser l'accès direct aux registres au lieu de la procédure `digitalWrite` de la bibliothèque, trop gourmande en opérations. Le délai de lecture de son ADC avec `analogRead` est de 100μsec peut aussi être diminué. Autrement, pour le même $L = 100\mu$H il existe aussi un circuit intégré CS209A avec son propre oscillateur dont le fonctionnement reste à étudier.

4.9 Expérience de Rüchardt



Une bouteille de volume V_0 est munie d'un tube de petit diamètre interne d. Une balle en acier de la masse m et de même diamètre glisse facilement dans le tube en faisant des petits déplacements Δy tout en gardant la bouteille enfermée hermétiquement. Le volume V_0 est tel que ces déplacements ne l'affectent pratiquement pas. La pression p dans la bouteille dévie légèrement de la pression atmosphérique $p_{\rm atm}$. Ce dispositif permet de mesurer la valeur de la constante adiabatique γ (coefficient de Laplace) du gaz dans la bouteille. Une bobine placée en haut du tube et alimentée par une source de la tension $U(t)$ sert à ajuster la position de la balle ou à forcer ses oscillations. Dans le but de développer un TP moderne pour les L2, L3 et Master, on propose d'utiliser un microcontrôleur Arduino pour échantillonner en temps réel (sec. 2.5) les mesures simultanées de déplacement $\Delta y(t)$, de pression $p(t)$, de température du gaz $T(t)$, et de tension $U(t)$. En particulier, les déplacements sont mesurés avec un capteur laser 850nm *VL6180X/VL53L0X* (Time of Flight Micro-LIDAR Distance Sensor) capable d'un taux d'échantillonnage maximal de $\approx 200 Hz$ (voir sec. 2.7.1 de sa notice).

Pour une pression atmosphérique ($p_{\rm atm} \approx 100$ kPa), une masse $m = 10$ de 20 g, $V_0 = 10$ L, $d = 10$ à 20 mm, $\gamma = 7/5$ (gaz diatomique) à la température 25°C, les fréquences propres sont de l'ordre de 1 à 10 Hz.

4.10 Magnétomètre

On propose d'utiliser les modules «breakout» de SparkFun (le circuit intégré de HMC5883L de Honeywell, ±1 à ±8 G, 12-bit, LSB 1 à 4 mG, à 75 Hz), de Pololu 2737 (LIS3MDL par ST ±4 à ±16 G, 16-bit, LSB 0.5 mG, RMS 4 mG, 150 Hz, 10×23 mm), de Sparkfun 19921 (MMC5983, FSR ±8 G, 18-bit, 0.06 mG LSB, bruit RMS 0.4 mG, 8×19 mm), de Adafruit 5579 (MMC5603 ±30 G, 20-bit, LSB 0.06 mG, jusqu'à 1 kHz), 4488 (LIS2MDL, jusqu'à ±50 G), tout avec un interface I^{2}C, ou, enfin, un capteur simple analogique lineaire à effet Hall KY-035 (circuit AH49E) pour mesurer le champ magnétique en temps réel. Par la suite, on peut utiliser une imprimante 3D pour scanner le champ dans un volume, ou avoir un système de moteurs. Applications : TP physique, entre autres. Une autre application fortement intéressante est de construire un pendule afin de observer et mesurer la vitesse de rotation de la Terre[31].

Notez que la sensibilité du capteur ne doit pas être trop élevée si les champs magnétiques B visés sont plus forts que celui de la Terre $B_{\oplus} \leq 60\,\mu$T ou 0.6 G[32]. Estimons nos besoins. Un courant continu circulaire I de rayon r autour de 0 dans le plan xy, crée le champ

$$B_r(z) = \mu_0 \frac{I}{2} \frac{r^2}{(r^2+z^2)^{3/2}} \quad \text{avec } \mu_0 = 4\,\pi\times10^{-7}\ \text{H/m} \quad \Rightarrow \quad \mu_0 \frac{I}{2\,r} \text{ au centre à } z = 0.$$

Pour N spires de même rayon on compte $N B_r(z)$. Par ailleurs, le champ sur l'axe d'un solénoïde infinie ($l \to \infty$) est

$$B = \mu_0\, n\, I, \quad \text{avec } n = N\, l^{-1} \text{ le nombre des spires par mètre.}$$

Ainsi, en TP L2, à l'intensité de courant de $I = 5$ A (maximale), le champ varie entre 1 et 0.5 G dans les anneaux simples de $r = 3$ et 6 cm, et monte jusqu'à 63 G dans le solénoïde de $n = 10^3$ qu'on dispose. En même temps, le capteur linéaire à effet Hall AH49E sorte un signal analogique de 1 à 4 V pour -1000 à $+1000$ G et $V_{\rm CC} = 5$ V (recommendée). Par conséquent, la sensibilité typique du AH49E égale 1.5 mv/G tandis que son bruit annoncé reste au dessous de 0.1 mV, soit 66 mG. Pour les valeurs maximales de ±100 G attendues, le signal sera dans l'intervalle de 2350 à 2650 mV On peut tenter d'utiliser ce capteur au condition de pouvoir bien mesurer les 0.1 mV. Pour cela, il vaut pré-amplifier $\times3$ et soustraire la base de 2350 mV avec un ampli-op et atteindre 0.5 G/mV et le bruit de 0.3 mV. L'ADC de ARDUINO de 10 bit dans le diapason $[0, 1]$ V (référence interne AREF = 1 V) nous assure le LSB = 1 mV. En cumulant une cetaine de mesures, on peut espèrer de poivoir mesurer B avec une incertitude de 0.1 G, ce que suffit marginalement nos besoins en TP L2.

4.11 Projets autour de accéléromètre et gyroscope

On considère les différents applications des cartes accéléromètre–gyroscope (sec. 3.5).

[31] D. B. Plewes, Magnetic monitoring of a small Foucault pendulum, *Rev. Sci. Instr.*, **89**(6) :065112 (2018)

[32] 1 gauss (G) = 0.1 mT, l'unité du système CGS, est utilisée couramment pour les champs magnétiques faibles.

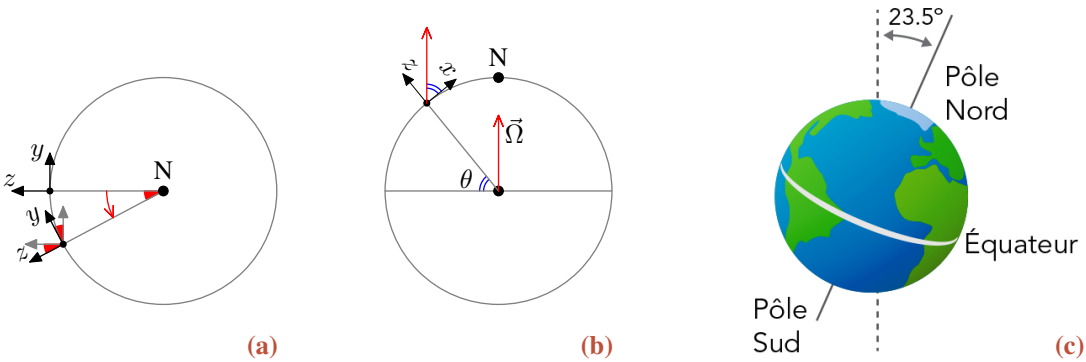


FIG. 18 – Rotation de la Terre et du gyroscope. Le vecteur $\vec{\Omega}$ représente la vitesse de rotation de la Terre ; le pesenteur $\vec{g}$ pointe vers le centre ; (x, y, z) marquent les axes du gyroscope. (a) La hémisphère nord avec deux positions successives du gyroscope sur l'équateur ; z est aligné avec $-\vec{g}$ et pointe «verticalement vers le haut», y donne le ouest, tandis que $\vec{\Omega}$ et x (orienté N) sont pointés vers nous. (b) Le gyro à la lattitude $\theta = 50.7°$ avec la même orientation des axes. (c) L'orientation de $\vec{\Omega}$ par rapport à l'orbite de la Terre autour du Soleil (axe horizontal).

4.11.1 Rotation de la Terre

Les MPU6050 et L3GD20H peuvent servir à mesurer directement la rotation de la Terre (voir la fig. 18 et cf la note 31). On pointe l'axe z du gyroscope verticalement vers le haut, et on oriente son axe x vers le nord (N, voir la fig. 18a avec x et $\vec{\Omega}$ alignés) ou sud (S). Pour passer de N à S, le gyro est retourné de 180° autour de la verticale : N $\to$ S : $(x, y, z) \mapsto (-x, -y, z)$. Pour chaque orientation, les données [degrés/s] sont collectées pendant *une minute* à ≈100 échantillons/s dans la plage de 250°/s (la plus lente et sensible). Chaque mesure représentera donc une moyenne de $N = 6000$ échantillons, et leur différence égale à

$$\Delta = \omega_N - \omega_S = 2\,\Omega\,\cos\theta = 2\,(360°\,T^{-1})\,\cos\theta, \text{ avec la période } T = 24 \times 60^2 = 86\,400\,\text{s et } \Omega = 0.0042°/\text{s, ou } 1° \text{ en } 240 \text{ secondes.}$$

Ici, l'origine du facteur $\cos\theta \leq 1$ parvient du fait, qu'à la latitude θ, le vecteur $\vec{\Omega}$ est incliné par rapport à la verticale : dans l'hémisphère N, l'angle entre $-\vec{g}$ et $\vec{\Omega}$ égale à $\frac{1}{2}\pi - \theta$, voir la fig. 18b. Ainsi $\Delta = 0.0053°/\text{s}$ à notre latitude $\theta = 50.768°$. Afin d'éviter ce facteur et de maximiser Δ nous pouvons soit (a) monter le gyro «en pente» d'angle θ orientée N-S et aligner son axe x avec $\vec{\Omega}$, soit (b) combiner les axes x et z. Dans le cas (b), on doit passer de N à S en tournant autour de l'axe y : N $\to$ S : $(x, y, z) \mapsto (-x, y, -z)$, et on mesure

$$\Omega = |\vec{\Omega}| = \sqrt{\Omega_x^2 + \Omega_z^2}, \quad \text{où } 2\,(\Omega_x, \Omega_y, \Omega_z) = \big(\omega_{Nx} - \omega_{Sx},\ 0,\ \omega_{Nz} - \omega_{Sz}\big).$$

Notez que le bruit de gyroscopes MPU6050 et L3GD20H est estimé à 10 LSB, et nous devrons faire avec une incertitude $\sigma = 0.08°/\text{s}$ de chaque échantillon. Par conséquent, selon la «loi des grands nombres», nous obtendrons une $\sigma/\sqrt{N}$ pour chaque mesure ω, et $2\,\sigma/\sqrt{N} = 0.0002$ pour Δ. Ceci suffira clairement pour détecter sa valeur anticipée de 0.005, à condition, bien sûr, que la ligne de base de gyroscope n'évolue guère durant les 2 minutes d'enregistrement.

4.11.2 Indicateur de freinage style F1

On se propose d'utiliser un accéléromètre + gyroscope (avec, en option, un GPS) pour détecter, enregistrer et réagir à toutes accélérations du véhicule. Ainsi un clignotant rouge feux arrière peut être activé quand le véhicule décélère au frein moteur. Une autre application est un indicateur automatique des virages dans une moto où le pilote n'a pas le temps d'activer/désactiver le clignotant.

4.11.3 Pendule de *Pohl*

Le pendule de *Pohl* est un oscillateur de torsion (voir la fig. 20 et la sec. 4.3.1 du cours de vibrations) constitué de : (a) un disque assez massif en cuivre qui peut tourner autour de son centre par un angle θ avec $|\theta| \leq \theta_{max}$; (b) un pointeur (une flèche blanche) placé sur le disque permettant de repérer l'angle de rotation θ sur le rapporteur ; (c) un ressort spiral, qui exerce un couple mécanique avec la tendance à ramener le disque vers sa position d'équilibre à $\theta = 0$; (d) un système de freinage électromagnétique (par les courants de *Foucault* et la force de *Laplace*) qui permet d'amortir les oscillations de façon douce et contrôlée sans aucuns frottements mécaniques ; (e) un moteur, relié au ressort par un levier. Lorsque le disque tourne d'un angle θ, le ressort s'enroule (ou se déroule), ce qui fait apparaître une force de rappel dont le moment mécanique s'oppose au moment du disque. Si on lâche le disque, le système retrouve son état d'équilibre en oscillant : c'est le régime d'oscillations *libres*. Le moteur permet d'exercer une force excitatrice sinusoïdale sur le pendule. dont la fréquence $\Omega/(2\pi)$ corresponde à la fréquence de rotation du moteur et peut être ajustée. Le système se met au régime sinusoïdal *forcé* avec la même fréquence.

Les oscillations libres et forcées du pendule de *Pohl* peuvent être étudiées en effectuant *deux* enregistrements simultanés : l'un pour les mouvements rotatifs du pendule et l'autre pour les mouvements de la poussée produite par le moteur (force externe). Au lieu des capteurs pour les déplacements angulaires θ, on utilise deux *gyroscopes* L3GD20H (*Pololu* 0J7997, sec. 3.5) pour mesurer $\dot{\theta}$. On les installe sur le dispositif TP PHYWE ref. P2132705. On y ajoute un *détecteur de ligne* QRE1113 (sec. 2.4.3) pour enregistrer les moments de passage du pendule à son point d'équilibre, et, en option, pour commencer l'enregistrement à ce moment precis (donc avec une phase

initiale φ_0 de 0 ou π). Il est possible également de mesurer le courant dans la bobine de freinage Faucault avec un détecteur de courant DC à effet Hall ACS712, un plus récent ACS723, ou même un LTSR-6-NP. Enfin, on peut piloter le moteur électique[33] en PWM via un transisteur NPN ou un MOSFET de puissance capables de commuter 1 A, dont le cirquit est découplé, pour plus de sécurité, à l'aide d'un optocoupleur (voir la sec. 2.7 ainsi qu'un cirquit tout fait par RobotDYN[34]). Les enregistrements peuvent être traités sous LINUX (RASPBERRY) avec un script en `gnuplot` et `gawk`, et possiblement d'autres utilités standards, tels que `ps2pdf`.

TP pendule de *Pohl* Matériel requis :

1. le pendule PHYWE ref. P2132705 soit (a) nu, (b) amenagé avec capteurs pour ARDUINO, (c) amenagé avec capteurs et drivers PWM (du moteur et de la bobine de freinage, voir la sec. 2.7, fig. 9e) pour ARDUINO, ou (d) muni d'une grille pour la photocellule ;
2. les étudiants ramènent leur propre clé USB pour sauvegarder leur résultats dans les cas (b), (c), et (d) ;
3. (a), (b), et (d) alimentation fixe 24 V/1 A DC ou AC pour le moteur[33] ;
4. (c) alimentation fixe 10 V/1 A DC pour le driveur PWM du moteur[33] ;
5. alimentation DC stabilisée $\leq$1 A et 0−10 V pour la bobine de freinage *Foucault* ;
6. ampèremètre 0−1 A DC (bobine) et voltmètre 0−24 V (moteur) ;
7. (a) chronomètre pour les études manuels
8. (b) carte ARDUINO UNO pré-chargé avec le programme `gyrosampler` et fils connecteurs M/F ;
   (b) ordinateur-écran RASPBERRY PI3 muni de cables pour ARDUINO (USB-B) et la clé (USB-A) ;
9. (d) module GTI, photo-cellule, et un PC sous WINDOWS XP avec logiciels GTI et RÉGRESSI.

Montage :
mettre les sources d'alimentation à 0 V/0 A et les connecter ; brancher l'ampèremètre en série avec la bobine de freinage. Selon le dispositif : (a), (b), et (d) brancher le voltmètre (sortie UX) ; (c) brancher l'alimentation DC à l'entrée de carte driver PWM (liée à ARDUINO, DP_9, V_{CC} et GND) ; (b) Brancher ARDUINO à PI3 avec le câble USB B ; soit (verion moderne) brancher la petite carte d'extention («shield») sur les DPs de ARDUINO, soit (verion ancienne) repérer «à l'arrière» du pendule PHYWE une petite plaque portant 5 connecteurs *Dupont* M avec fils *jaune*, *vert*, *noir*, *rouge*, et *blanc* ; câbler le pin digitale DP_2 au fil *blanc*, pins SCL et SDA (à l'aurte extremité de la même ligne de pins digitaux de la carte UNO, face aux LEDs) au fils *jaune* et *vert*, GND et V_{CC} 5V (ligne des connecteurs opposée aux DPs) aux fils *noir* et *rouge*. Vérifier que rien n'empêche les mouvements du pendule ; (d) voir avec le prof.
Expérience :

1. Observer les oscillations libres (sans amortissement) et déterminer la pulsation propre ω_0 du pendule.
   (a) Lancer le pendule, compter une dizaine de démi-périodes (passage de la flèche à 0 du rapporteur qui marque la position verticale d'équilibre) et mesurer le temps en lançant le chronomètre au premier passage à 0. Par la suite, selon le dispositif :
   (b) Sur le compte `arduino`, ovrir le «terminal» du logiciel `arduino` à la vitesse 115200 bps. Le μCU répond avec des informations sur son état. Envoyer la commande `W`. Décaler et lancer le pendule. La LED sur la carte marque chaque démi-période. Envoyer la commande `Z` et noter les informations.
   (d) Lancer le pendule et enregistrer ces mouvements avec le logiciel `GTI`. Exporter vers `Régressi` et réproduire les données avec la fonction théorique $\theta(t) = A\,\exp(-\gamma\,t)\,\sin(\omega_0\,t+\varphi)+\theta_0$ en ajustant ses paramètres. À voir avec le prof. et les TP L1.
2. Étudier les oscillations libres amorties et déterminer le coefficient γ pour différents courants I=0.25 à 0.5 A dans la bobine.
   (a) Lancer le pendule, repérer sur le rapporteur les amlitudes max/min A_i décroissantes par leur valeur absolue à chaque démi-période. Pour convertir les graduations du rapporteur en degrés, se référer à l'appendice 4.11.3A. Tracer une ligne droite dans les coordonnés $(i, \log(|A_i|))$ et déterminer son coeff. directeur. En trouver γ en [1/sec].
   (b) En sa version avancé, le programme `gyrosampler` détermine la quasi-période T, l'amplitude initiale A, et le coefficient d'amortissement γ après avoir échantillonné $\dot\theta(t)$ pendant au moins 2−3 démi-périodes, voir 1(b). Cependant, la précision de ces estimations dépend du rapport de taux de l'échantillonnage et T. Pour obtenir les valeurs plus précises, et pour estimer les incertitudes, enregistrer et traiter la courbe $\dot\theta(t)$ comme expliqué dans l'appendice 4.11.3C.
   (d) suivre la même approche que dans même que 1(d)
3. Étudier les oscillations forcées. Allumer l'alimentation du moteur. Dans le cas «manuel» (a), (b), et (d) : régler la sortie à 24 V fixe, régler la tension U_x aux bornes du moteur à l'aide de deux potentiomètres rotatifs «GROB» (approximatif) et «FEIN» (fin). Contrôler U_x au voltmètre. Dans le cas «PWM» (c) : régler la sortie à 10 V DC fixe, utiliser la commande `V`n avec $n = 0\ldots1000\,‰$ (approx. 0 à 10 V) pour regler la vitesse du moteur. La pulsation des oscillations égale à $\Omega = 2\,\pi/T$, où T est la période de rotation du moteur. Pour faciliter les manipulations, déduire la dépendance $T(U_x)$ ou $T(n)$ et prévoir le réglage nécessaire pour atteindre le pic de la courbe de résonance avec $\Omega \approx \omega_0$. (Selon PHYWE, la résonance correspond à $U_x \approx 8$ V.) Mesurer la pulsation Ω, l'amplitude $A(\Omega)$, et le déphasage $\varphi(\Omega)$ pour différents valeurs de $\Omega(U_x)$ dans l'intervalle $[0, 4\,\omega_0]$ et pour 2−3 valeurs de $\gamma(I)$.
   (a) Estimer Ω en suivant l'approche de 1(a) ; mesurer T en chronométrant les tours du moteur (encodeur blanc sur sa roue, face «arrière», compter plusieurs tours) ; confirmer la relation entre T et Ω. Sur le rapporteur noir, repérer les points de retour

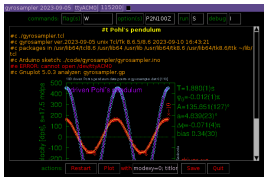
[33] FAULHABER *Schöneich* 2230V024S (148) avec un reducteur 81:1 22B (298) à engrenage droit. Selon l'étiquette de PHYWE, le moteur requiert Antrieb 24 V 650 mA, Motorspannung an Prüfbuchsen 2−16 V, in Resonanznähe 8 V.
[34] à la base d'un N-channel MOS-FET de puissance IRF540N (4.5-24 V (peak 36 V) 30 A 50 W, une diode Schottky intégrée) et le photocoupleur EL817.

$\theta_{\max} = -\theta_{\min}$ (plusieurs fois, prendre le moyen, estimer l'erreur) ; en déduire l'amplitude A (en degrés, voir 2(a)) correspondant à la fréquence Ω. Pour mésurer le déphasage, chronométrer le temps entre les passages consécutifs du pendule et de la roue encodeur du moteur à leurs repères respectifs (l'équilibre pour le pendule, où sa phase égale 0 ou π), et convertir en fraction de T.

(b) et (c) L'estimation de T est accessible directement par la commande `Z`. Pour mesurer A et avoir plus de précision, enregistrer simultanément et traiter les données $\dot\theta(t)$ (pour les mouvements du pendule) et $\dot f(t)$ (pour la poussée du moteur) avec le «flag» `F` (voir l'appendice 4.11.3C). Tracer les courbes paramétriques de *Lissajous* $t \to \big(\dot\theta(t), \dot f(t)\big)$ et retrouver le déphasage φ par la méthode des «ellipses»[35], employée couramment dans les études des circuits éléctriques aux courants/tensions sinusoïdales.

(d) avec GTI, l'impossibilité d'enregistrer $\theta(t)$ et $f(t)$ simultanément fait la mesure de déphasage irréalisable.

4. Représenter les résultats comme courbes de résonance[36] $A(\Omega/\omega_0)$ et de déphasage $\varphi(\Omega/\omega_0)$ pour différents ammortissements γ.



Appendice 4.11.3C : logiciels `gyrosampler` On utilise trois logiciels `gyrosampler` avec extentions `.ino`, `.tcl`, et `.gp` pour, respectivement, échantillonner (acquérir), collecter, et traiter les données. Par manque de dévélopeurs et testeurs, les deux derniers (faisant appel aux autres utilitaires, tels que `awk`, `netpbm`, et `gs`) ne sont opérationnels en ce moment que sous LINUX. Facilement adaptatables pour MACOS, ou autres UNIX'es, leur exploitation sous WINDOWS reste problématique mais possible en principe.

`gyrosampler.ino` est un sketch en `c++` pour ARDUINO (voir la sec. G). Au plus simple, on peut l'installer dans le μCU et l'interroger avec l'outil habituel ARDUINO IDE (sec. 2.1.1) disponible sur tout plateformes. Son fonctionnement ressemble à celui de `adcsampler` (cf. sec. 2.5 et 4.1.3) : ouvrir et configurer le moniteur à 115200 bauds, envoyer des commandes CLI au sketch, copier avec `Ctrl-A` les mesures qui apparaissent en réponse, et les coller (`Ctrl-V` ou souris) dans un fichier text `gyrosampler.dat` en utilisant tout éditeur de texte. Par la suite, ces données peuvent être traitées dans un tableur (tel que EXCEL ou RÉGRÉSSI sous WINDOWS) ou avec `.gp`. Autrement, on s'en sert de `.tcl` pour tout faire. Voici les commandes CLI.

Les cavaliers (flags) agissent comme des boutons d'un interrupteur on/off qui basculent leur paramètre $1/0$ à chaque fois :

`F` distingue entre les régimes *forcé* et *libre* avec l'acquisition des données de deux et un gyroscope(s), respectivement ;

`W` distingue le déclenchement immédiate et au premier passage à l'équilibre (voir l'action `S`) ;

Les options sont suivis par des valeurs numériques x (un nombre entier positif à plusieurs chiffres décimaux), qu'elles passent aux paramètres du scripte. Leur appel répétitive (avec les mêmes valeurs) n'affecte guère le fonctionnement du scripte :

`T`x définit l'intervalle `READ_PERIOD` entre les échantillons en μsec. Par default, `READ_PERIOD` est fixé à 20000, donnant le taux de 50 Hz, soit approximativement 100 points par la période du pendule ;

`N`x contrôle le nombre maximal des échantillons `npts` ;

`P`x définit le nombre maximal `pcnt` des démi-périodes à observer (si $x > 0$, voir l'action `S`) ;

`V`x avec $x = 0 \ldots 1000$ contrôle la vitesse du moteur par le ‰ PWM duty cycle.

Les actions sont des boutons-poussoir déclenchant une réponse :

`Z` affiche le compteur de passages à l'équilibre (c.p.é.), voir la sec. 2.4.3, estime la démi-période T, et remet le c.p.é. à zéro ;

`S` demarre l'acquisition après avoir mis le c.p.é. à zéro ; L'enregistrement commence soit immédiatement, soit au premier passage à l'équilibre (selon le flag `W`), et procède jusqu'à l'épuisement de `npts` (voir option `N`) ou `pcnt`, si ce dernier est positif (option `P`). Pour chaque échantillon, la/les valeur(s) de la vitesse angulaire reportée(s) par le/les gyroscope(s) en mdps est/sont affichées (option `F`) sous forme de nombres entiers avec le signe, suivie(s) par la valeur momentanée du c.p.é.

`I` affiche des informations sur la carte, le sketch, et ses paramètres actuels.

`J` affiche le courant dans la bobine de freinage

`gyrosampler.gp` est un scripte pour le logiciel graphique GNUPLOT. Pour l'executer, il souffit de saisir un terminal et lancer la commande `gyrosampler.gp`. Il analyse les données dans `.dat` et produit le fichier graphique `.pdf`. Les exemples de ce dernier sont illustrés dans la la fig. 19. La commande reconnaît quelques options `nom` sous forme `nom=val`, spécifiquement

titlon avec `val` = 0 ou 1 gère l'apparition du title sur le graphique ;

modexy avec `val` = 0 ou 1 sélectionne le mode d'affichage classique (avec l'axe de temps t, fig. 19a et 19b) ou, en régime forcé, le mode paramètrique dit «x-y», voir la fig. 19c ;

dset avec valeurs 0, 1, etc définit l'indice du jeu des données à analyser, les jeux sont separés par un double changement du ligne ;

scl (`.75`) ajuste l'hauteur du graphe dans le `.pdf` en unités de la taille de réference de 3 in sans contrôler les polices.

Ses options sont soumises au scripte avec `gyrosampler.gp -e "nom1=val1; nom2=val2"`.

`gyrosampler.awk` est un scripte auxiliaire en langage `awk` qui permet à `gyrosampler.gp` d'extraire des informations du fichier `.dat` issu typiquement de l'interaction avec `gyrosampler.ino`. Il est possible d'engager ce scripte (dans un terminal) par la commande `gyrosampler.awk gyrosampler.dat` et vérifier ainsi toutes ces informations.

[35] Pour une ellipse $t \to \big(a\sin(\omega t),\ b\sin(\omega t + \varphi)\big) = (x, y)$ avec $a > 0$ et $b > 0$ les amplitudes de $x(t)$ et $y(t)$, on trouve $|\sin(\varphi)| = h/B$, où h donne son intersection supérieure avec l'axe y (vertical). Cette méthode *ne permet pas de déterminer le signe* de φ. Pour cela, il faut connaître la direction de parcours.

[36] voir l'exercice 4.3 de la section 4 du cours «Vibrations et ondes»

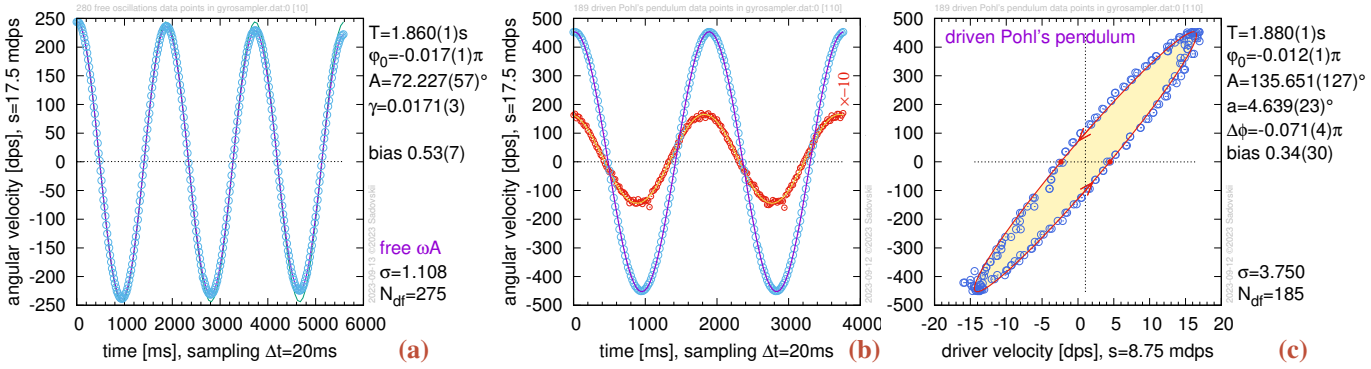


FIG. 19 – Pendule de *Pohl* : la présentation et l'analyse des mesures avec l'utilitaire `gyrosampler.gp` (scripte GNUPLOT) des oscillations (a) libres, (b) forcées, et (c) forcées sous forme de l'ellipse de Lissajous, dit «mode x-y». Les graphes représentent les données de gyroscopes `0x6a` et `0x6b` sur les vitesses $\mathrm{d}\theta/\mathrm{d}t$ du pendule et de la force avec les amplitudes respectives sont ωA et $\omega\, a$.

`gyrosampler.tcl` est une appli TCL/TK assurant une interface graphique (GUI, voir l'entête de cet appendice) et une alternative à ARDUINO IDE (pour `.ino`) et `gyrosampler.gp`. Son utilisation est assez intuitive. Saisir une fenêtre de terminal et lancer la commande `gyrosampler.tcl`. Au démarrage, l'appli tente de connecter à la carte ARDUINO. Si la connexion est établie, les «commandes» à `.ino` peuvent être soumises, et les réponses sont montrées dans la fenêtre principale de l'appli. Par la suite, les «actions» sont proposées. Ainsi «Plot» engage `gyrosampler.ino` et «Save» permet de sauvegarder les fichiers `.dat` et `.pdf` actuels sur une clé usb (si présente) ou dans le répertoire courant (`./`) avec leur nom prolongé par une indice unique. A noter, que s'ils ne sont pas effacés explicitement avec «Clear», les résultats des appels successives à `.ino` sont cumulés et sont exploités ensemble par «Plot» avec l'indice défaut de jeu des données (dataset `dset`) correspondant au résultat le plus récent.

Appendice 4.11.3A : calibrer la plage de sensibilité de gyroscopes Le gyroscope L3GD20H possède *trois* plages de sensibilité (*full scale* ou FS) : ±250 (défaut L3G), ±500, et ±2000 dps (degrés/sec). Les amplitudes de la poussée de moteur sont faibles et on garde ±250 pour le gyroscope `0x6b`. L'amplitude maximale $A_{\max} = \theta_{\max}$ du pendule même (gyroscope `0x6a`) et limitée à 20 grandes divisions (g.d.) de circle rapporteur du dispositif PHYWE P2132705. On remarque que 10 g.d. (l'angle entre deux rayons du corps de pendule en cuivre) correspondent à $1/5$ me du cercle complet. Par consequent,

$$\frac{2\,\pi}{5}\,\mathrm{rad} = 10\,\text{unités g.d.} \Rightarrow \frac{\pi}{25}\,\mathrm{rad} = 7.2^\circ = 1\,\text{g.d.}, \quad \frac{\pi}{5}\,\mathrm{rad} = 36^\circ = 5\,\text{g.d.}, \quad \text{et} \quad A_{\max} = 20\,\text{g.d.} = 144^\circ.$$

La periode T_0 des oscillations libres du pendule et sa pulsation propre ω_0 sont, respectivement, $\approx$1.8 s et $2\,\pi/1.8$. C'est autour de la fréquence $1/T_0$, dans les conditions de résonance, qu'on anticipe les grosses amplitudes de θ et $\dot\theta$. Ainsi pour $\dot\theta$ maximale on estime

$$\dot\theta_{\max} = A_{\max}\,\omega_0 = 144\,\frac{2\,\pi}{1.8} = 160\,\pi = 502.65\,\text{dps}.$$

Par conséquent, il faut choisir la plage FS $= \pm500$ dps pour le gyroscope «avant» `0x6a` (indice `0` dans `gyro[]`) et les deux bits FS1-FS0 de son registre REG4 `0x23` (sec. 7.5, pp. 39-40 du datasheet de L3GD20H) doivent être mis à 0-1 (au lieux de 0-0 par défaut).

```
gyro[0].writeReg(L3G::CTRL_REG4,0x10); // FS=500
```

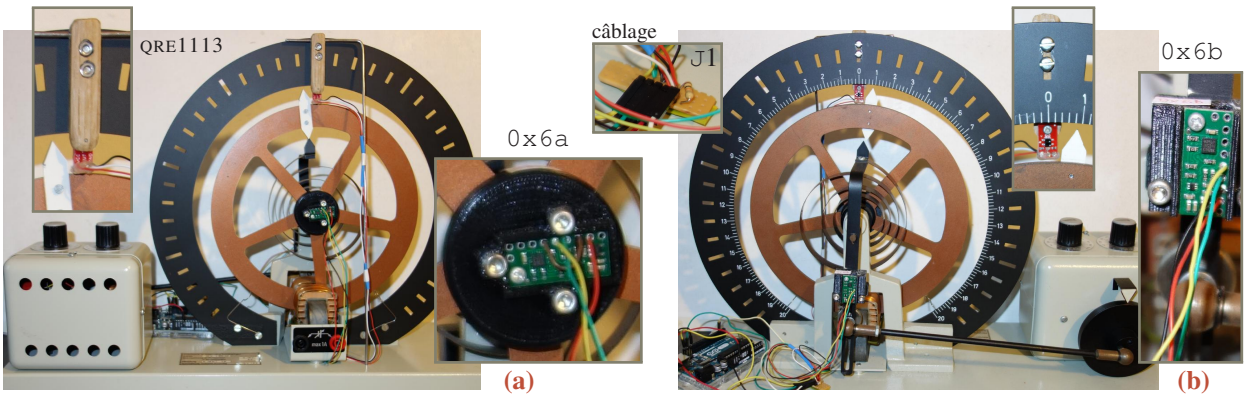


FIG. 20 – Pendule de *Pohl : montage des capteurs à l'avant (a) et à l'arrière (b) du dispositif* PHYWE P2132705, voir l'appendice 4.11.3B.

Appendice 4.11.3B : montage des capteurs

- Souder quatre fils (AWG30 souples multibrins 0.05 mm^2 avec isolant en silicone) de longueur 25 à 30 cm à chaque carte gyroscope L3GD20H : *jaune* à SCL (horloge du bus I^{2}C), *vert* à SDA (données I^{2}C), *noir* à GND (masse), et *rouge* à VIN (tension d'entrée V_{CC}).
- Les fils doivent rester libre afin de ne pas empecher les mouvements du pandule et de la poussée. Pour organiser le passage des fils et les rassembler «à l'arrière», où on placera une petite plaque de montage, mettre une gaine 3/32 thermorétractable 2.4 → 1.2 mm de 3 cm.
- Sur une des cartes L3GD20H, mettre (souder) un cavalier entre SDO et GND pour lui attribuer l'adresse I^{2}C `0x6a`. Cette carte sera destinée à mesurer les mouvements du pendule et sera dénommée «avant» (AV) et/ou par son adresse. L'autre carte sera destinée à mesurer les (petites) poussées du moteur et sera dénommée «arrière» (AR) et/ou par son adresse `0x6b` (voir la fig. 20).
- Souder trois fils de ≈45 cm à la carte QRE1113 (détecteur IR d'obstacles) : *noire* à GND, *blanc* à OUT, et *rouge* à VCC. Protéger ces soudures par la colle thermique pour éviter un court-circuit accidentel provoqué par des contactes de la carte avec les parties métalliques du pendule.
- Sur une petite plaque (breadboard) de montage de 3×4 cm (4×12 trous, la plaque peut être élargie de 1 cm de chaque côté pour y accommoder deux trous de fixation de diamètre 2 mm), souder 5 connecteurs Dupont coudé males pour SCL, SDA, GND, VCC, et VOUT (sortie QRE1113). Y ramener, respectivement, deux fils jaunes, deux verts, trois noires, trois rouges, et le fil blanc. Noter que 2 fils AWG30 peuvent être insérés et soudés ensemble dans un seul trou de la plaque. Souder une résistance de 10 K entre VCC et VOUT. Protéger les soudures par la colle thermique.
- Descendre les fils de QRE1113 le long du rapporteur côte «avant» (serres câbles nylon) ; ranger ces fils et les fils du gyroscope AV `0x6a` au-dessous de la bobine de l'aimant éléctrique (gaine) avant de les ramener tous vers «l'arrière» à la plaque de montage.
- Imprimer les supports de deux gyroscopes «avant» et «arriere» avec sa couverture (fichiers `stl` par *Pierre Kulinski* et *Pascal Masselin*). Pour leur fixation, utiliser 3 vis M3.20 (av) et 2 mêmes vis avec 2 écrous M3 (ar). Pour installer les cartes L3GD20H dans leur supports, utiliser 2 vis M2.5 avec leurs ecrous. Alternativement, remplir les trous avec la colle thermique et utiliser 2 vis bois VB2.5 (diamètre 2 mm, longueur 5 mm).
- Fabriquer (imprimer) le petit support de la carte QRE1113. Utiliser 2 vis M3.10 avec ecrous pour l'installer dans la fente du rapporteur noir au point 0 g.d. (l'équilibre en haut) de côté «avant» (côté flèche blanche sur la fig. 20). La carte même s'y fixe avec une vis VB2.5. Vérifier que la distance entre QRE1113 et la flèche reste de 2−3 mm, et que le pendule peut osciller sans empêchement (voir la fig. 20).
- Installer un «étai» du rapporteur de côté «avant» afin de gâcher tout ses vibrations latéraux (devenant très nuisibles et provocant fausses signaux de détecteur QRE1113, surtout pour les grandes amplitudes en régime forcé proche de la résonance). L'étai constitue l'hypoténuse de ≈28.5 cm d'un triangle de 5 × 28 cm avec le rapporteur. Il fait l'angle de 80° avec la base, et se fixe par insertion (eventuellement collée) en haut dans le trou horizontal du support de QRE1113, et en bas, dans le trou percé verticalement dans la plateforme-base (en bois MDF) du pendule PHYWE. Son bout inférieur de 15 mm est incliné donc legérement de 10°, tandis que son bout supérieur de 5-6 cm est horizontal et perpendiculaire au triangle. Dans notre prototype, l'étai a été fabriqué en fil d'alliage alu assez rigide de diamètre 2.5 mm et de longueur ≈36 cm. Après son insertion, il est déformé légèrement afin d'obtenir la distance optimale de 2−3 mm entre le détécteur QRE1113 et la flèche blanche.

4.12 PCR et qPCR thermal cycler (DS)

De nombreux tests modernes de diagnostic moléculaire ciblant les acides nucléiques sont généralement limités aux pays développés ou aux laboratoires de référence nationaux des pays en développement. La capacité de rendre les technologies de diagnostic rapide des maladies infectieuses dans un format portable et peu coûteux constituerait une avancée révolutionnaire dans le domaine de la santé mondiale. De nombreux tests moléculaires moléculaires sont également élaborés sur la base de réactions en chaîne par polymérase (PCR), qui nécessitent des thermocycleurs relativement lourds (de l'ordre de 10 à 20 kg) et nécessitant une une alimentation électrique continue. La vitesse de montée en température de la plupart des thermocycleurs les plus économiques est relativement lente (2 à 3 °C/s), de sorte qu'une réaction en chaîne par polymérase peut prendre 1 à 2 heures. Par-dessus tout, ces thermocycleurs sont encore trop chers (de 2 000 à 4 000 dollars) pour être utilisés dans des environnements à faibles ressources.

Les projets OpenPCR et PS-PCR sont déjà très développés et testés, il y a une doc très détaillée pour les reproduire et tout le logiciel nécessaire (y comprit côté PC où on les branche via port usb). Comme cerise sur le gâteau, on pourrait peut être y ajouter un pilote par iPhone... évolution vers la détection en temps réel avec fluorescence voir Open qPCR alternative robotique à l'élément Peltier, voir rapid and low-cost PCR thermal cycler for low resource settings

En bref : il s'agit de réplication de gènes (DNA). À la base, c'est un bon élément Peltier piloté par Arduino. On mesure/contrôle aussi la température et (en option) la fluorescence. Il y a une grande partie biologique et un peu de cinétique chimique. Les tests sont à faire en collaboration avec les biologistes (Jean-Christophe Devedjan)

4.13 Capteur déplacement/vitesse (DS)

On récupère une vieille souris pour connecter ces capteurs au microcontrôleur Arduino et en faire un capteur de déplacement linéaire (1D et/ou 2D), angulaire (rotation), vitesse, et accélération. Autrement, on utilise le détecteur infrarouge QRE1113 ou une photocellule décrits dans la sec. 2.4.3. Dans le cas d'une photocellule ou «photogate», l'idée est de reproduire le système «picket fence». Ceci est utilisé pour enregistrer les déplacements linéaires dans les anciens TP oscillations (GTI+RÉGRESSI) Voir le prototype et la théorie. Il y a des nombreuses applications dans les labos de TP, notamment oscillations et autres TP L1 de mécanique, pendule de Pohl (ancien TP L3, sec. 4.11.3), goniomètre (L1 optique), mesures en 2D, robotique.

4.14 Anémomètre et girouette à effet Doppler (R. Bocquet)

Le but du projet est de réaliser un anémomètre à effet Doppler. Il s'agit d'utiliser cet effet bien connu, dans la gamme des ultrasons, pour mesurer la vitesse du vent ainsi que sa direction. Cette gamme de sons se situe à des fréquences au-dessus des 20 kHz, limite audible

de tout être humain normalement constitué. Vous aurez à votre disposition des émetteurs et des récepteurs accordés certainement à 30 kHz mais qu'il faudra vérifier. L'idée de la mesure est assez simple : l'onde sonore est une onde mécanique portée par l'air, la vitesse de propagation dépendra donc de la vitesse de l'air. Vous devrez :

- comprendre le principe physique de l'effet Doppler dans l'air
- mettre en évidence cet effet dans une expérience
- proposer un montage permettant de donner également la direction
- proposer un montage (électronique + ARDUINO) pour mesurer des vents allant de 0,1 noeud à 50 noeuds[37]
- réaliser le montage si vous avez le temps et le tester.

Ne négligez pas le travail préparatoire dans ce projet.

4.15 Cinémomètre à GPS (R. Bocquet)

Le but du projet est de réaliser un instrument de la taille d'une grosse montre pour mesurer la vitesse et la direction de déplacement d'un bateau à voile ou d'une bicyclette. Pour cela on vous propose d'utiliser un composant (u-blox 6 GLONASS GPS) qui n'est autre qu'un GPS et un tout petit écran de visualisation de 4 caractères en bus I2C. On utilisera le format de données NMEA, très utilisé dans l'industrie du nautisme et disponible sur le GPS que vous avez à votre disposition. Vous devrez :

- réaliser le montage GPS et visualisation avec carte ARDUINO
- réaliser le montage GPS et visualisation sur écran
- réaliser le montage avec un micro-contrôleur Atiny 45 ou 85

* développer une carte électronique autonome (pile 3,3 V bouton) et son boitier

4.16 Optimisation de la consommation d'un micro-contrôleur ATtiny 85 (R. Bocquet)

Les microcontrôleurs que vous utilisez ont la possibilité de fonctionner avec de très faibles consommations et sous des tensions de 3,3 V. Il s'agit dans ce projet de mettre en place une programmation d'un micro-contrôleur ATtiny 85 qui est un composant électronique enfichable sur la plaquette d'essais et de faire un montage permettant de mesurer la consommation du composant. Ce projet demande des notions d'électronique et de très bonnes connaissances de la langue anglaise. En effet vous devrez lire la notice du micro-contrôleur et programmer directement les ports du micro-contrôleur.

4.17 Centrale météorologique – 2 groupes de travail (R. Bocquet)

Il s'agit de mettre en place une centrale de mesure météorologique avec un boîtier extérieur embarquant les capteurs et un boîtier intérieur pour réception, traitement et stockage des données. Les deux boîtiers seront reliés par une transmission radiofréquence à 433 MHz. Dans le cas où un seul groupe est constitué, chaque partie peut -être traitée séparément.

Groupe 1 : capteurs et émetteur. réaliser et mettre au point un montage permettant à minima de mesurer la température, la pression, l'humidité relative et la luminosité. Vous définirez un protocole de données et mettrez en place une liaison RF pour transmettre les données. Vous étudierez les possibilités de transfert sur carte de votre montage en utilisant un micro-contrôleur ATtiny 45 ou 85.

Groupe 2 : récepteur. réaliser et mettre au point un récepteur des données météo du groupe 1 avec un moyen de sauvegarde. Vous programmerez une carte ARDUINO pour visualiser sur un écran LCD graphique les données et développerez un code pour réaliser une prévision météo, fonction des données enregistrées.

Groupes 1 et 2 : liaison RF. établir la liaison RF entre les 2 boîtiers et tester les distances possibles de transmission, ainsi que les difficultés qui pourraient nuire à la qualité de la transmission.

Appendices et sketches dans le répertoire `~/Arduino/`

License : Sauf noté autrement, tout code pour ARDUINO dans les appendices ci-dessous est ©2025 D. A. Sadovskií (`sadovski@univ-littoral.fr`) ; la Licence Publique Générale GNU (GNU GPL) ver. 3 de la Free Software Foundation, Inc. (FSF), 51 Franklin St., 5th Floor, Boston, M A 02110–1301, Etats Unis, est impliquée. Ce code est un logiciel libre ; vous pouvez le redistribuer et/ou le modifier au titre des clauses de la version 3 de la GNU GPL, telle qu'est publiée par la FSF. Le code est distribué dans l'espoir qu'il sera utile, mais sans aucune garantie, ni même une garantie implicite de commerciabilité ou de conformité à une utilisation particulière. Vous devriez avoir reçu et consulté un exemplaire de la GNU GPL avec ce code ; si ce n'est pas le cas, demandez le à la FSF.

[37] 1 noeud = 1 mile nautique par heure

A Branchement de LEDs et barographe ~/Arduino/BlinkSOS/BlinkSOS.ino

```
/*
  SOS signal Blink (a derivative of Blink in the Arduino example library)
  Repeatedly turn the onboard LED on/off for 200 msec three times,
  then for 500 msec three times, and then again 200 msec for three times.

  Most Arduinos have an on-board LED you can control. On the UNO, MEGA and ZERO
  it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is set to
  the correct LED pin independent of which board is used.
  If you want to know what pin the on-board LED is connected to on your Arduino model, check
  the Technical Specs of your board  at https://www.arduino.cc/en/Main/Products

  This example code is in the public domain.

  modified 8 May 2014  by Scott Fitzgerald
  modified 2 Sep 2016  by Arturo Guadalupi
  modified 8 Sep 2016  by Colby Newman

  modified  3 Aug 2017  by Dima Sadovskii
  modified 18 Nov 2021  by Dima Sadovskii
*/

// NB: use a PWM (~) pin to see analog fading effect, pin 8 is not suitable
#define PIN_BASE 9  // starting digital pin number for 4 external LED's
#define PIN_ANLG A0 // pin number for digital entry
// the setup function runs once when you press reset or power the board
void setup() {
  int j,i;
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
  // initialize 4 external digital pins starting in sequence from PIN_BASE as output
  for(j=PIN_BASE,i=4; i--; pinMode(j++, OUTPUT));
  // light the LED's in a sequence
  for(j=PIN_BASE,i=4; i--; j++) {
    digitalWrite(j, HIGH);   // turn the LED on (HIGH is the voltage level)
    delay(200);              // wait for a 200 milliseconds
    digitalWrite(j, LOW);    // turn the LED off by making the voltage LOW
    delay(200);
  }
  // Open serial communications and wait for port to open:
  Serial.begin(9600);
  while (!Serial) {
    ; // wait for serial port to connect. Needed for native USB port only
  }
  // send an intro:
  Serial.println("\nSOS blinking and Serial interaction via keys");
  // Serial.println();
}

// parse a single byte character key as a hexadecimal digit
unsigned int hexToByte (char c) {
  if ( (c >= '0') && (c <= '9') ) return (int)(c - '0');
  if ( (c >= 'A') && (c <= 'F') ) return (int)(c - 'A')+10;
  if ( (c >= 'a') && (c <= 'f') ) return (int)(c - 'a')+10;
}

// issue a triple blink on pin unless the analog input on PIN_ANLG is non-zero
// or there is a byte waiting on the serial input line to be examined
void blink3(int d, int dd, int pin) {
  int i;
  for(i=3; i; i--) {           // repeat three times
    if(analogRead(PIN_ANLG) || Serial.available()) return;
    digitalWrite(pin, HIGH);   // turn the LED on (HIGH is the voltage level)
    delay(d);                  // wait for d milliseconds
    digitalWrite(pin,  LOW);   // turn the LED off by making the voltage LOW
    delay(d);
  }
  delay(dd);                   // wait for extra dd msec
}

int aold=-1; // last read analog input

// Arduino runs this loop function over and over again forever
void loop() {
  int i,j,hex,aval=0;
```

```
  char key;
  aval = analogRead(PIN_ANLG);
  if(aval>0) {                          // deal with nonzero analog input
    Serial.print(aold);
    Serial.print(" V=");
    Serial.println(aval);
    aold = aval;                        // keep track of last analog input
    for(i=4; i--; digitalWrite(i+PIN_BASE,(aval>=i*256)?HIGH:LOW));
  }
  else { // show SOS signal
    if(aold>0) {
      aold=0;
      Serial.println("stop analog entry");
      for (i=255 ; i >= 0; i -= 5) {    // fade out from max to min in increments of 5 points (range from 0 to
          255):
        analogWrite(PIN_BASE, i);       // NB: only possible for a PWM pin, e.g. 9 (but not 8!)
        delay(20);                      // wait for 20 msec to see the dimming effect
      }
      //digitalWrite(PIN_BASE,LOW);
    }
    blink3(200, 250, LED_BUILTIN);  // letter S
    blink3(500, 250, LED_BUILTIN);  // letter O
    blink3(200, 500, LED_BUILTIN);  // letter S and final pause
  }
  // Read serial input:
  if(Serial.available() > 0) {
    key = Serial.read();
    Serial.print("key ");
    Serial.print(key);
    Serial.print(" with ASCII ");
    Serial.print(int(key));
    Serial.print("=0x");
    Serial.print(key,HEX);
    if(isHexadecimalDigit(key)) {     // use isDigit for decimal input
      //if Hexadecimal display the numerical value of the key
      Serial.print(" gives HEX value ");
      Serial.print(hex=hexToByte(key));
      Serial.print(" and BIN code ");
      for(i=8,j=PIN_BASE+4;i>0;i=i>>1) {
        Serial.print(((hex&i)?1:0));
        digitalWrite(--j,((hex&i)?HIGH:LOW));
      }
    }
    Serial.println();
  }
}
```

B Échantillonnage de ADC ~/Arduino/adcsampler/adcsampler.ino

Programme pour échantillonner l'ADC en temps réel sans interruptions horloge ni bufferisation (envoi direct sur le port série)

```
/* -*- mode: c++; coding: latin-1; current-input-method: latin-1-prefix; ispell-local-dictionary: "english";
    fill-column: 79; comment-column: 0; eval: (local-set-key "\C-c\C-\M-u" 'browse-url-firefox); -*-
  This simple precise fixed time interval ADC sampler
  can be used for periods greater than 300 usec (see code, safer 500 usec)
  and thus has maximal theoretical sampling rate of 3kHz (more like 2kHz)
  see https://playground.arduino.cc/Interfacing/LinuxTTY on interfacing
  with linux serial port, which is normally /dev/ttyACM0
  stty -F /dev/ttyACM0 cs8 115200 ignbrk -brkint -icrnl -imaxbel -opost -onlcr -isig -icanon -iexten -echo -
      echoe -echok -echoctl -echoke noflsh -ixon -crtscts
  stty -F /dev/ttyACM0 115200 cs8 cread clocal
  screen /dev/ttyACM0 115200
*/
#define PIN_ANLG A0 /* pin A0..A5 for analog 5V max 10-bit ADC entry */
unsigned long READ_PERIOD = 4000;  // 4000 us gives 250 Hz sampling rate
unsigned long lastRead=0;
unsigned int npts=0; // number of samples

void setup() {
  /* Open faster serial communication (instead of usual speed 9600bps) */
  Serial.begin(115200); // 115200 bps = 14400 bytes/sec, 70 usec/byte
  while (!Serial) {
    ; /* wait for serial port to connect (for native USB port only) */
  }
```

```
  Serial.println("# fixed time interval ADC sampler");
}

void loop() {
  static unsigned int ncnt=0;
  static char key='Q';
  static long val=0;
  static char hex_format=0;
  char r;
  while(Serial.available()) { // read settings
    r = Serial.read();
    if(isDigit(r)) {
      val*=10;
      val+= r-'0';
    }
    else {
      switch(key) {  // keys that preceed numerical values
        case 'T':    // set sampling period (usec)
          if(val) READ_PERIOD = val;
          break;
        case 'N':    // set number of samples
          if(val) npts=val;
      }
      val=0;
      key=r;
      switch(key) {  // switches
        case 'H':    // hex format toggle
          hex_format^=1;
          break;
        case 'S':    // start sampling
          ncnt=npts;
        default:
          Serial.print("# period=");
          Serial.print(READ_PERIOD);
          Serial.print(" usec, samples=");
          Serial.print(npts);
          Serial.print(" at ");
          Serial.print(1000000.0 / READ_PERIOD);
          Serial.print(" Hz");
          Serial.println();
        case 'N':
        case 'T':
        case 'Q':
          break;
      }
    }
  }
  if(ncnt) { /* on 16MHz boards the time resolution is 4 usec, overrun in approx 70 min */
    for(ncnt=0, lastRead=micros()+8; ncnt<npts; ){  /* acquire and display npts samples */
      while(micros()<lastRead);
      lastRead += READ_PERIOD;
      val=analogRead(PIN_ANLG);  // 0..5V->0..1024 takes about 100 usec (10kHz)
      if(hex_format) Serial.println(val,HEX); // at least 200 usec for 3 bytes
      else Serial.println(val);
      ncnt++;
    }
    ncnt-=npts;
  }
}
```

C Exploration de PWM et IRQ : générateur des signaux créneau avec fréquencemètre

Le sketch ~/Arduino/pwmsg/pwmsg.ino accompagne la sec. 2.4.5, où nous utilisons une LED clignotant rapidement et une photdiode PD (sur le pont diviseur de tension avec une résistance pulldown R de 300Ω, voir le schéma électronique dans la fig. 6b) pour détecter ces clignotements et déterminer leur fréquence. La LED est alimentée par la tension créneau parvenant du pin DP_9 de l'μCU ARDUINO en mode PWM. Les oscillations de l'intensité sont transformées par la PD en courant de quelques dizaines de μA permettant de basculer l'état logique[38] du pin digital DP_3 entre HIGH (PD illuminée) correspondant à une tension $U_H \geq V_{IH} = 3\,V$ et LOW (sans lumière) correspondant à une tension $U_L \leq V_{IL} = 1.5\,V$, et de requérir une interruption externe IRQ. Cette dernière est servie de façon asynchrone par une petite fonction ISR qui avance le compteur f_cnt de (démi)périodes. La ISR est attachée directement au vecteur INT1_vect de IRQ_1 par DP_3.

```
volatile unsigned char f_cnt=0;                    // trigger counter (rapid)
volatile unsigned long int f_carry=0;              // carry on count
```

[38] selon les niveaux logiques de l'MCU ATMEGA328P, voir la fig. 3

```
#define frqcnt_count (f_carry ? ((f_carry-1)*0xFF + f_cnt) - 1 : 0)
ISR(INT1_vect) { if(! ++f_cnt) f_carry++; }       // IRQ handler for DP3
```

On améliore le montage avec un bouton poussoir de marche/arrêt rapide de PWM de style 4c sur le pin DP_2. Le bouton commande une IRQ_0 externe, qui est servie par la fonction `ISR(INT0_vect)`, equivalent à l'ancienne ISR `button_int_handler` (sec. 2.4), mais attachée statiquement au vecteur `INT0` de DP_2. Pour s'en passer de cette option, évitez (commentez) la définition de `BUTTON_PIN` ou mettez DP_2 à la masse.

Le sketch `pwmsg` assume la possibilité de complimenter les IRQs de niveau logiques sur DP_3 par le *comparateur analogique* `AC` qui génére une IRQ lorsque la tension à la sortie de circuit `PD` devient égale à une tension de référence interne $V_{\text{ref}} = 1.1$ V (à l'entrée «+» de l'`AC`). On joint totu simpliment l'entrée «-» de l'`AC` aur la broche DP_7 à la DP_3. Cette configuration nous permet de compter soit les IRQs de comparateur, soit ceux du niveau logique (voir la commande `Z`). Vu que la IRQ par la DP_7 est servie par la méme petite routine, on déclare :

```
ISR(ANALOG_COMP_vect, ISR_ALIASOF(INT1_vect));    // same IRQ handler for AC and DP3
```

Le comparateur `AC` est activé dans le `setup()` par configuration de son registre ACSR, notamment le bit 7 (`ACD` pour activer le `AC`), 6 (`ACBG` pour choisir V_{ref} de 1.1 V à sa borne positive), 3 (`ACIE` pour activer l'interruption), et 0-1 (selon le mode d'interruption `ACIS_SLOPE` égale à 0, 2, ou 3).

```
ACSR = (1<<ACD)|((1<<ACBG)|(1<<ACIE)|ACIS_SLOPE); // mode 0 SWITCH, 2 FALLING, or 3 RISING
```

Une fois le MCU est connecté au port USB, `pwmsg` lance son interface de communication (CLI) sur la ligne série (dans la partie `loop`), et permet de sélectionner le mode et la fréquence du signal PWM, son rapport cyclique[39], et d'afficher l'état de MCU et la réponse du fréquencemètre. Le CLI est actionné par des «clés» à une lettre précèdes dans des certains cas par un «argument» numérique.

```
/*
  simple square wave Vmin=0, Vmax=Vcc signal generator using Arduino DP9 pwm
  make use of an external button on DP2 to turn the pwm on/off
  make use of an external probe on DP3 to measure the frequency f and period T
  (both button an frequency probe are handled through interrupts)

  Alternatively, the frequency meter makes use of the comparator interrupt with
  internal 1.1V standard replacing AIN0 and probe on AIN1. On the UNO board,
  comparator entries AIN0 and AIN1 are connected to DP6 and 7, respectively. Of
  these, only DP7 is required and should be connected to the probe on DP3.

  tested on UNO (Vcc=5V) with 5mm red LED biased by 220R pullup, and SFH-203
  T1-3/4 5mm PD (I_R = 80mA typical, 50mA min, max reverse V=20V, 400-1100nm,
  lambda_max 80-900nm, response 5ns!) reverse biased through 3k pulldown
  @3922Hz observed response time was 20us for going from 4.1->1.6V (3.6->1.5V)
  given that the LED full brightness light-on time is about 10us, this is fine
  the response is symmetric; the meter reports 3921.6Hz (3922 pwm, 3920.4 scope)
  the meter (CHANGE) survives even at 31kHz, barely though, reporting 37k (?!)

  NB: a simple low-pass RC filter can take care of the UNO pwm output HF
  jitter.  Basically, a small bypass cap for the LED biased with the 220R
  pullup to 5V Vcc (i.e., 220R _before_ the LED anode and the LED cathode at
  GND!).  Thus for the 8-bit pwm @3922Hz (cs=2), 220R+100pF work nicely giving
  fc=7MHz.  However, the LED itself has a "parasite" capacitance of about 50pF,
  which combines with the 220R pullup and forms an RC lowpass already, matching
  LED's uptime of about 1us (1MHz), and making the above filtering redundant.

  ATTN: when compiled to trigger solely on RISING or FALLING events instead of
  on CHANGE, the meter counts whole periods and reports the same frequencies,
  up to and except for 31k when the circuit is too slow to follow ...

  NB: the typical values of T and f may come out, respectively, longer and lower
  than those reported by the scope connected on DP3 (ch2) and pwm DP9 (ch1)
  especially so for large periods and lowe frequencies due to timer inaccuracy
  see https://forum.arduino.cc/t/why-is-my-time-slow/1204090/9
  the test results below show no apparent simple correlation
   CS   pwm  meter   scope [Hz]  at reported uC temp 28C
    1 31373 37???   31362
   10  3922  3921.6  3920.33
   11   490   488.5   490.0
  100   123   122.2   122.5
  101    31    30.5    30.6

  DS 2025-05-26 08-28  attn: UNTESTED for Timers 0,2
*/
#define _PROGRAM_ "pwmsg"          // program name
#define DEBUG 1                    // set to 0 to suppress extra msg
#define LIB_DEBUG 2
#define _VER_ "May 26 2025"        // program version
#if defined(__has_include) && __has_include("local_setting.h")
#include "local_setting.h"
#endif
```

[39] *duty cycle* = rapport cyclique ou facteur de forme

```
//- ATTN: on Arduino Uno and other 328p-based boards, digital pins 2 and 3
//- are the only ones that can be used for programmable external interrupts 0,1
#define FRQCNT_PIN 3               // frequency counter IRQ trigger
#define FRQCNT_SLOPE FALLING       // count full periods (slower but symmetric)
#undef  FRQCNT_SLOPE               // default to count 1/2 periods
#define BUTTON_PIN  2              // external button IRQ trigger
#define BUTTON_LED LED_BUILTIN     // used as pwm on/off indicator
#define BUTTON_TYP HIGH            // normal HIGH, i.e. with a pull-up resistor
#define PWM_PIN 9        // PWM @490 Hz (away from DP2 and not used for SPI)

#include <limits.h>
#include <math.h>        // some math constants
#include <avr/sleep.h>   // AVR library for sleep modes
#include <avr/power.h>
#include <avr/wdt.h>     // WDT timer can be used if thigs go south
#include <avr/io.h>      // direct uC register access
#include <avr/interrupt.h>
#include "pwm.h"         // ATTN. must preceed any function declarations

#include "uartout.h"     // for c-style printf and friends from stdio h
#include "printlf.h"     // ad-hoc macro replacing CR+LF of Serial.println()

#ifdef DEBUG
#include "freeRAM.h"     // free RAM information
#include "boardinfo.h"   // extract board information from chip signature
// NB: at 16 samples, sampling is fast, but accuracy is about 10mV
// original version in pwmsg/readVcc.h -> ../intRTC/readVcc.h
#define readVcc_nsamples 16
#include "readVcc.h"     // internal voltage reference
unsigned int Vcc = 0;    // Vcc in [mV]
unsigned int uCtemp = 0; // uC temperature [C]
#endif

#ifdef FRQCNT_PIN        // frequency meter
#include <time.h>        // modified c header for avr-libc and AVR-GCC
#ifndef FRQCNT_SLOPE
// default to trigger on both falling and rising events => count 1/2-periods
#define FRQCNT_SLOPE CHANGE
#endif

volatile unsigned char f_cnt=0;          // trigger counter (fast)
volatile unsigned long int f_carry=0;    // carry on count
volatile time_t frqcnt_timer=0;          // counter start time [us]
volatile time_t frqcnt_span =0;          // last count time
#define frqcnt_count (f_cnt || f_carry ? (f_carry*UCHAR_MAX + f_cnt) - 1 : 0)
#if FRQCNT_SLOPE == CHANGE
#define frqcnt_pnumb (frqcnt_count>>1)
#else
#define frqcnt_pnumb frqcnt_count
#endif
#define frqcnt_mHz(T) (1000000000UL / T) // frequency in Hz/1000

unsigned long int frqcnt_period() {      // period in usec [us]
 unsigned char c = f_cnt;
 time_t t = millis();
#if FRQCNT_SLOPE == CHANGE
 // Serial.print(c,HEX); Serial.print(' ');
 // wait till the next odd trigger event, bailout to safety on long wait (>2s)
 while( (c == f_cnt || (f_cnt&1) != 0) && millis()-t < 2000 ); //  Serial.println(f_cnt,HEX)
 // Serial.print(f_cnt,HEX); Serial.print(' ');
#else
 // wait till the next trigger event, bailout to safety on long wait (>2s)
 while(c == f_cnt && millis()-t < 2000 );
#endif
 return (micros()-frqcnt_timer) / frqcnt_pnumb;
}

void frqcnt_restart() {
   f_cnt = 0;                    // initiate period counter (0..255)
   f_carry = 0;                  // and period counter overloads
 }
// NB: see hardware/arduino/avr/variants/standard/pins_arduino.h for this macro
#define FRQCNT_INT digitalPinToInterrupt(FRQCNT_PIN) /* 0 or 1 */
// bits INT0,1 and INTF0,1 of the External Interrupt Mask & Flag Registers EIMSK & EIFR, p.55
#define FRQCNT_INT_CHECK (1<<FRQCNT_INT)
// External Interrupt Control Register A (EICRA) for sense control, 12.2.1 p.54
```

```
// LSB of ISC10,1 bits (2) Table 12-1 and ISC00,1 bits (0) Table 12-2 p.54
#define FRQCNT_ISC (FRQCNT_INT ? ISC10 : ISC00)
// NB: on UNO, DP 2,3 serve external interrupts x=0,1 with vectrors INTx_vector
//     lazy if does not expand and won't work subsequently in the ISR macro
//     (FRQCNT_PIN == 2 ? INT0_vect : (FRQCNT_PIN == 3 ? INT1_vect : NULL))
#if FRQCNT_PIN == 2
#define FRQCNT_INT_vect INT0_vect
#elif FRQCNT_PIN == 3
#define FRQCNT_INT_vect INT1_vect
#else
#define FRQCNT_INT_vect NULL
#endif
// NB: 0 is a valid value representing low-lever external wake-up-suitable IRQ
#define FRQCNT_SEN ((EICRA&(0x3<<FRQCNT_ISC))>>FRQCNT_ISC)
// (FRQCNT_PIN == 2 ? FRQCNT_SLOPE : (FRQCNT_PIN == 3 ? (FRQCNT_SLOPE<<2) : 0))
#define FRQCNT_INT_SLOPE (FRQCNT_SLOPE<<FRQCNT_ISC)
/* ISR for handling interrupts from the digital waveform probe pin */
ISR(INT1_vect) {                  // void frqcnt_int_handler() {
 if( f_cnt == 0 && f_carry == 0)
   frqcnt_timer = micros();       // beginning time of the period count [us]
 // if(f_cnt++ & 0xFF) f_carry++;
 if(!++f_cnt) f_carry++;          // triggers on 0xFF
 // f_cnt++; if(SREG & 1) f_carry++;
}
/*
   NB: at the CPU level, when uint8_t is autoincremented (or, generally, when
   two uint8_t's are added), the lower 8 bits of the result are stored in the
   destination, and the overload, if it happens, is signalled out in the form
   of a "carry flag" aka "carry bit" in the uC status register SREG (bit 0 on
   AVR architecture, see ATmega datasheet, p.10).  Unfortunately, the
   c-compiler's optimizer might reorder (or even remove) operations and there
   is no way to guarantee that the SREG check occurs immediately after the
   targeted operation [https://forum.arduino.cc/t/addition-overflow/127070/9].
   https://ucexperiment.wordpress.com/2016/03/13/arduino-status-register-sreg/
*/
/*
  The actions to be taken upon IRQ's from INT1 (DP3 external) and ANALOG_COMP,
  are identical and so a single implementation for their ISR suffices. The code
  is shared using the ISR_ALIASOF() attribute to the ISR macro, see
  https://avrdudes.github.io/avr-libc/avr-libc-user-manual/group__avr__interrupts.html
*/
ISR(ANALOG_COMP_vect, ISR_ALIASOF(INT1_vect)); //FRQCNT_INT_vector

#define ACIS_SLOPE  (FRQCNT_SLOPE==1 ? 0 : FRQCNT_SLOPE)
#define ACSR_CONFIG ((1<<ACBG)|(1<<ACIE)|ACIS_SLOPE)
/* Analog Comparator (AC) Status Reg (ACSR) value, 22.3 p.203
   ACD        (bit 7) set to 1 to disable AC
   ACBG       1 to have internal 1.1V voltage ref on the (+) input instead of AIN0
   ACO        0 to clear AC Output
   ACI        0 to clear AC Interrupt Flag
   ACIE       1 to Enable AC Interrupt
   ACIC       0 to disable AC Input Capture
   ACIS1,0    Interrupt Mode: 00 toggle, 10 falling, 11 Rising, 01 reserved
   ATTN: hardware/arduino/avr/cores/arduino/Arduino.h defines correctly FALLING
         and RISING as 2 and 3, but, unfortunately, defines CHANGE as 1 (not 0!)
*/

#endif /* FRQCNT_PIN (frequency counter pin) */

unsigned char compound_pwm_toggle() {
#ifdef PWM_TCCRA                  // toggle PWM on/off bit
 bool state = pwm_toggle();
#ifdef BUTTON_LED                 // LED indicator of PWM state
 digitalWrite(BUTTON_LED, (state) ? HIGH : LOW );
#endif // button led
#ifdef FRQCNT_PIN                 // frequency meter:
 if(state) frqcnt_restart();      // restart (half-)period counter
#endif
 return state;
#else                             // paranoia setup with no pwm ...
#ifdef BUTTON_LED                 // toggle button LED indicator
 digitalWrite(BUTTON_LED, !digitalRead(BUTTON_LED));
 return digitalRead(BUTTON_LED);
#else
 return 0;
#endif // button led
```

```
#endif // tccra
}

#ifdef BUTTON_PIN                 // UNO pins 2,3 serve ext. interrupts 0,1
// https://docs.arduino.cc/language-reference/en/functions/external-interrupts/attachInterrupt/
// following modes 0..3 are possible on UNO: LOW, CHANGE, RISING, FALLING
#define BUTTON_MODE_SYMBOLS {'_','x','\\','/'}
// NB: see hardware/arduino/avr/variants/standard/pins_arduino.h for this macro
#define BUTTON_INT digitalPinToInterrupt(BUTTON_PIN) /* 0 or 1 */
// NB: on UNO, DP 2,3 serve external interrupts x=0,1 with vectrors INTx_vector
#define BUTTON_INT_vect (BUTTON_PIN == 2 ? INT0_vect : (BUTTON_PIN == 3 ? INT1_vect : NULL))
// bits INT0,1 and INTF0,1 in the External Interrupt Mask & Flag Registers EIMSK & EIFR, p.55
#define BUTTON_INT_CHECK (1<<BUTTON_INT)
// External Interrupt Control Register A (EICRA) for sense control, 12.2.1 p.54
#if BUTTON_INT == 1
#define BUTTON_ISC ISC10          // LSB of ISC10,1 bits (2) Table 12-1 p.54
#else
#define BUTTON_ISC ISC00          // LSB of ISC00,1 bits (0) Table 12-2 p.54
#endif
// NB: 0 is a valid value representing low-lever external wake-up-suitable IRQ
#define BUTTON_SEN ((EICRA&(0x3<<BUTTON_ISC))>>BUTTON_ISC)
#define BUTTON_INT_SLOPE (BUTTON_SLOPE<<BUTTON_ISC)

// cf https://forum.arduino.cc/t/does-the-serial-communication-interfere-with-usage-of-external-interrupts
    /1085101
// boxcar wait interval [ms] during which subsequent button interrupts will be
// considered spurious and ignored to reduce jitter of the button switch
#define BUTTON_GAP 4              // [ms] should be in 1 .. 16
#ifndef BUTTON_LED
#define BUTTON_LED LED_BUILTIN    // LED indicator pin for the button
#endif
#if BUTTON_GAP<=0
#undef BUTTON_GAP
#elif BUTTON_GAP>16
#define BUTTON_GAP 16
#endif

// push button type LOW / HIGH means that the pin's normal state is 0 / Vcc,
// i.e., the pin goes (for a short time) high / low when the button is pressed
// the respective button circuit is that of the (high/low)-end switch
// i.e., the resistor is connected to 0 (type LOW) or to Vcc (type HIGH)

#if ! defined(BUTTON_TYP) || (BUTTON_TYP!=HIGH && BUTTON_TYP!=LOW)
#define BUTTON_TYP LOW            // normal LOW (legacy choice)
#endif

// the mode can be LOW (whenever the pin is low), CHANGE, RISING, FALLING
// https://www.arduino.cc/reference/en/language/functions/external-interrupts/attachinterrupt/
#ifndef BUTTON_SLOPE              // IRQ trigger mode
#if BUTTON_TYP==LOW
#define BUTTON_SLOPE RISING
#else
#define BUTTON_SLOPE FALLING      // i.e., goes from HIGH to LOW
#endif // SLOPE
#endif
#if BUTTON_SLOPE==FALLING         // i.e., the pin goes from HIGH to LOW
#define BUTTON_EXPECT LOW
#elif BUTTON_SLOPE==RISING        // i.e., the pin goes from LOW to HIGH
#define BUTTON_EXPECT HIGH
#endif // EXPECT

#ifdef BUTTON_GAP
/*
   CLKPR & 0xF system clock prescaler (16MHz Arduino Uno default is 0), p.33
   the clock is divided on 2<<(CLKPR & 0xF) first and than by the divisor

   TCCR2B & 0x7 clock select bits CS2[0,1,2] encode Timer 2 divisors

   0x0 no timer source, timer stopped
   0x1 system clock / 2^0 = c/1    @   16 MHz  62.5 ns
   0x2 system clock / 2^3 = c/8    @    2 MHz   0.5 us
   0x3 system clock / 2^5 = c/32   @  500 kHz   2 us
   0x4 system clock / 2^6 = c/64   @  250 kHz   4 us   1ms like micros()
   0x5 system clock / 2^7 = c/128  @  125 kHz   8 us   2ms overrun
   0x6 system clock / 2^8 = c/256  @ 62.5 kHz  16 us   4ms overrun
   0x7 system clock / 2^10= c/1024 @ 15625 Hz  64 us  16ms overrun
```

```
*/

#define BUTTON_CS 7               // default time resolution of 64us (1/16 ms)
#define BUTTON_FREQ 16            // [kHz] (approx), overrun in 16ms
#define BUTTON_TOVF ((1<<8)/BUTTON_FREQ)
#if BUTTON_GAP>BUTTON_TOVF
#define BUTTON_GAP BUTTON_TOVF    // restrict the gap from above to 16ms
#elif BUTTON_GAP<= 1/16           // 1/16 = 0.0625
#define BUTTON_GAP 2/16           // ignore very small and negative values
#endif
#define BUTTON_CNT (0xFF - (BUTTON_FREQ*BUTTON_GAP-1))

#if   BUTTON_CS == 1
#define BUTTON_FDIV 0
#elif BUTTON_CS == 2
#define BUTTON_FDIV 3
#elif BUTTON_CS == 3
#define BUTTON_FDIV 5
#elif BUTTON_CS == 4
#define BUTTON_FDIV 6
#elif BUTTON_CS == 5
#define BUTTON_FDIV 7
#elif BUTTON_CS == 6
#define BUTTON_FDIV 8
#else
#define BUTTON_FDIV 10
#endif
#define BUTTON_FSTP ((float) F_CPU/(1<<BUTTON_FDIV)/1000)

// volatile unsigned char t2ovf = 0;
// ISR for timing button gap using timer 2 overflow (OVF vector 9)
// see arduino-1.8.19/hardware/tools/avr/avr/include/avr/iom328p.h
ISR(TIMER2_OVF_vect) {
#ifdef BUTTON_EXPECT              // verify button state
 if(digitalRead(BUTTON_PIN)==BUTTON_EXPECT)
#endif
   compound_pwm_toggle();                  // toggle pwm after having waited
 TCCR2B = 0;                      // disable Timer 2
 // t2ovf++;
}

// ISR for handling external IRQ triggered by the button switch
// NB: because attachInterrupt (in the setup) for the INT0 vector (button) is
// incompatible with hardcoding INT1 vector (frequency counter) via ISR macro.
// So use ISR here as well
ISR(INT0_vect) {                  // void button_int_handler() {
 TCNT2  = BUTTON_CNT;             // clear (preset) 8-bit counter of Timer 2
 TIFR2 &= ~0x7;                   // clear all Timer 2 interrupt flags
 TCCR2B = BUTTON_CS;              // set Timer 2 system clock divisor
}
#else

// legacy non-debouncing ISR for handling interrupts from the button switch
void button_int_handler() {
#ifdef BUTTON_EXPECT              // verify button state
 if(digitalRead(BUTTON_PIN)==BUTTON_EXPECT)
#endif
   compound_pwm_toggle();
}
#endif
#endif // button pin

// the components of the target frequency [Hz] can be changed/entered separately
// keeping in mind that each can hold numbers up to UINT_MAX = (1<<16)-1 = 65535
unsigned int  f_Hz=0, fkHz=0, fmHz=0;
#define kHz_FACTOR 1000
#define TARGET_PWM_FREQ ( (double) fkHz * kHz_FACTOR + f_Hz + (double) fmHz / kHz_FACTOR )
// absolute target min/max frequency step [Hz]
#define TARGET_F_ASTEP_MAX 2.5

#define __DEBUG_LINE__ Serial.print(F("DEBUG: " __FILE__ " ")); Serial.println(__LINE__); Serial.print('\n');
    Serial.flush()

// NB: the F() moves strings to flash memory instead of wasting SRAM
void print_info(unsigned int typ) {
 // in c++, this can be done more elegantly through an object constructor ...
```

```
 const char *pwm_wgm[] = PWM_WGM_MODE_SYMBOLS;
#ifdef BUTTON_PIN
 const char isc[] = BUTTON_MODE_SYMBOLS;
#endif /* BUTTON_PIN */
 unsigned char i;
 double f;
#if defined(DEBUG) && DEBUG>=0
 if(typ&1) {
#ifndef _UARTOUT_                   // ========= Serial based output ==========
   Serial.print(F("# " _PROGRAM_));
#ifdef BUTTON_PIN                   // optional button setup info
   Serial.print(F((BUTTON_TYP==LOW?" bL":(BUTTON_TYP==HIGH?" bH":" bX"))));
   // the above is treated as a macro at compile time, if in doubt ...
   //   Serial.print(F(
   // #if BUTTON_TYP==LOW
   //    " bL"
   // #elif BUTTON_TYP==HIGH
   //    " bH";
   // #else
   //    " bX";
   // #endif /* BUTTON_TYP */
   // ));
   Serial.print(isc[BUTTON_SEN]);   // dynamical, see Table 12-1,2 on p.54
   Serial.print(digitalRead(BUTTON_PIN));
#if defined(BUTTON_GAP) && DEBUG > 1
   Serial.print('g');               // boxcar gate to suppress jittering
   Serial.print(0xFF-BUTTON_CNT);
   Serial.print(F("/16ms"));
#endif /* BUTTON_GAP && DEBUG > 1 */
#endif /* BUTTON_PIN (end optional button setup info) */
   Serial.print(F( " " ARDUINO_ARCH_NAME_PREF " " _VER_ ", uC " BOARD_TYPE " "));
// #ifdef ARDUINO_ARCH_AVR             // compiled for AVR architecture (16-bit)
//    Serial.print(F(" AVR:"_VER_", uC "BOARD_TYPE" "));
// #else
//    Serial.print(F(    " "_VER_", uC "BOARD_TYPE" "));
// #endif /* ARDUINO_ARCH_AVR */
#else                               // ========= printf based output ==========
   printf_P(PSTR("# " _PROGRAM_));  // short program name (held in low progmem)
#ifdef BUTTON_PIN                   // button setup info (optional)
   //   printf(" b%c%c%d",(BUTTON_TYP==LOW ? 'L' : (BUTTON_TYP==HIGH ? 'H' : 'X')),
   printf_P(PSTR((BUTTON_TYP==LOW ? " bL%c%d":
                 (BUTTON_TYP==HIGH?" bH%c%d":" bX%c%d"))),
          isc[BUTTON_SEN], digitalRead(BUTTON_PIN) );
#endif /* BUTTON_PIN (end optional button setup info) */
   printf_P(PSTR(" " ARDUINO_ARCH_NAME_PREF " " _VER_ ", uC " BOARD_TYPE " "));
#endif /* _UARTOUT_ */
   Serial.print(BOARD_INFO,HEX);
   Serial.print('@'); Serial.print(F_CPU/1000000);
   if(CLKPR & 0xF) {                // MCU clock frequency (known to compiler)
     Serial.print('/');             // with timer clock prescaler if it is >1
     Serial.print(2<<(CLKPR & (0xF)));
   }
   Serial.print(F("MHz"));
   Serial.print(F(" SRAM:")); Serial.print(freeRAM()); // free data RAM avail
#ifdef readVcc_nsamples
   Serial.print(' ');
   Serial.print(uCtemp,DEC);        // MCU temperature [deg C]
   Serial.print('C');
#endif
   Serial.flush(); printlf();       // end first header line
#if DEBUG>1
   Serial.print(F("# compiled on " __DATE__ ", " __TIME__ " from " __FILE__ "\n")); Serial.flush();
#endif // end DEBUG>1
 }
#endif // end DEBUG>=0

 if(typ&0x2) {                      // current pwm duty cycle and Vcc
#ifdef PWM_PIN
   Serial.print(F("# dc="));        // pwm duty cycle
   Serial.print(pwm_get(), DEC);
#endif
#ifdef readVcc_nsamples
   Serial.print(F(" Vcc="));
   Serial.print(Vcc);               // value of Vcc
   Serial.print(F("/"));
   Serial.print(readVcc_cfactor);   // factor used to compute Vcc
```

```
#endif
   printlf();
 }

 if(typ&0x4) {                        // complete information on PWM setup
#ifdef PWM_PIN
   Serial.print(F("# DP"));
   Serial.print(PWM_PIN);             // digital pin in use
#ifdef PWM_ON                         // detect PWM output
   Serial.print( PWM_ON ? '~' : (digitalRead(PWM_PIN)?'+':'-') );
#endif
#ifdef PWM_COM                        // pwm Compare Output Mode (COM) bits
   Serial.print(':');
   Serial.print((PWM_TCCRA>>PWM_COM)&3,HEX);
#endif
#ifdef PWM_WGM
   Serial.print(F(" "));              // display Wave Generation Mode (WGM)
   Serial.print(pwm_wgm[i = PWM_WGM]);
#if defined(DEBUG) && DEBUG>0
   Serial.print(F("["));              // contents of the WGMx bits (aka WG=)
   Serial.print( i, HEX);
   Serial.print(F("]"));
#endif
#endif
   Serial.print(F(" CS="));           // display timer Clock Select (divisor)
   i = PWM_TCCRB&(7);                 // CS bits _BV(CSx2)|_BV(CSx1)|_BV(CSx0)
   Serial.print(PWM_DIV,DEC);         // value of timer divisor
#if defined(DEBUG) && DEBUG>0
   Serial.print(F("["));              // litteral contents CSx2:0
   Serial.print(i,DEC);
   Serial.print(F("]"));
#endif
   Serial.print(F("x"));              // times overall timer frequency prescaler
   Serial.print( 1<<(CLKPR & (0xF)), DEC);
   // " @" F_CPU / (1<<(CLKPR & 0xF)) / 1000000 "MHz"
   if(PWM_MODE && i) {                // for PWM capable WGM modes and CS>0
     if(i<=PWM_MAX_DIVISOR) {
       Serial.print(F(" @"));         // current frequency
       Serial.print(PWM_FREQ,0);
       Serial.print(F("Hz"));
     } else Serial.print(F(" ext"));
   }
   else Serial.print(F(" off"));      // pwm timer is off
   Serial.print(F(" "));              // x2 for phase (and frequency) correct WGM
   if(*pwm_wgm[PWM_WGM]=='p') Serial.print(F("2x"));
   Serial.print(PWM_OCR, DEC);        // duty cycle [clock ticks]
   Serial.print(F("/"));
   Serial.print(PWM_CYCLE);           // full pwm pulse duration (aka period)
#endif
#ifdef PWM_PIN
   Serial.print(F(" "));              // duty cycle [promille]
   Serial.print(pwm_get(), DEC);
   Serial.print(u8"\u2030");          // utf-8 for promille
#endif
#ifdef readVcc_nsamples
   Serial.print(F(" Vcc="));
   Serial.print(Vcc);                 // value of Vcc
#endif
   Serial.flush(); printlf();
 }

 if(typ&0x10) {                       // PWM related register dump
   Serial.print(F("# COM"));          // COM bits
   Serial.print(PWM_TIMER);           // Timer 0,1,2 and type of the OC pin A/B
   Serial.print((char)('A'+PWM_PIN_TYP));
   Serial.print('=');        Serial.print((PWM_TCCRA>>PWM_COM)&3,HEX);
   Serial.print(PWM_ON ? '*' : ' ');
   Serial.print(F(" CS="));  Serial.print(PWM_TCCRB&(7),HEX);
   Serial.print(F(" WGM=")); Serial.print(PWM_WGM,HEX);
   Serial.print(PWM_MODE ? '~' : ' ');
   Serial.print(F(" TOP=")); Serial.print(pwm_get_top(),DEC);
   Serial.print(F(" OCR=")); Serial.print(PWM_OCR,DEC);
#if PWM_TIMER==1
   Serial.print(F(" ICR=")); Serial.print(ICR1,DEC);
#endif
   printlf();
```

```
 }

 if(typ&0x20) {
#ifndef _UARTOUT_                    // ========= Serial based output ==========
   Serial.print(F("# "));
   Serial.print( f = TARGET_PWM_FREQ, 3);
   Serial.print(F("Hz"));
#else                                // ========= printf based output ==========
   printf_P( PSTR("# %lu.%03uHz"),
             (long unsigned int)fkHz*kHz_FACTOR+f_Hz+fmHz/kHz_FACTOR, fmHz%kHz_FACTOR );
   f = TARGET_PWM_FREQ;              // target (or current) frequency [Hz]
#endif /* _UARTOUT_ */
   if(f_Hz>0 || fkHz>0) {
     Serial.print(':');
     Serial.print((double)PWM_SPAN(f),0);
   }
   Serial.print('?');
   if(PWM_MODE) {
     Serial.print(F(" now @"));      // PWM frequency of the current WGM and CS
     Serial.print(PWM_FREQ,3);       // [Hz] with three-digit fractional part
     Serial.print(F("Hz"));
     Serial.print(':');
     Serial.print( PWM_DIV );
     Serial.print('x');
     Serial.print( round(PWM_CYCLE) );
   }

   printlf();
 }

#ifdef FRQCNT_INT
 unsigned long int T;
 if (typ & 0x8) {                    // frequency meter data
   Serial.print('#');                // 'c' (AC 1.1V) vs 'd'(DP TTL levels)
   Serial.print( ACSR&(1<<ACD)?'d':'c' );
   if( frqcnt_pnumb ) {
     Serial.print(F(" T="));         // period [us]
     Serial.print(T = frqcnt_period());
     Serial.print(F("us f="));       // frequency [Hz]
     Serial.print((float) frqcnt_mHz(T) / 1000, 1);
     Serial.print(F("Hz over "));
     Serial.print(frqcnt_pnumb,DEC);
     Serial.print(F(" periods in "));
     Serial.print(micros()-frqcnt_timer,DEC);
     Serial.print(F("us "));
     Serial.print('[');              // raw counter data
     Serial.print(f_carry,DEC);      // carry on count
     Serial.print(',');
     Serial.print(f_cnt,DEC);        // period count
     Serial.print(']');
   }
   else
     Serial.print(F(" no frequency data"));
   printlf();
 }
#endif

}

void setup() {
 pinMode(LED_BUILTIN, OUTPUT);
 digitalWrite(LED_BUILTIN, HIGH);    // turn the onboard LED on = "busy"
#ifdef PWM_PIN
 pinMode(PWM_PIN, OUTPUT);           // activate pwm on pin PWM_PIN
 digitalWrite(PWM_PIN, LOW);         // turn the pwm pin off (V=0)
 pwm_set_duty(0xFF>>1);              // default 50% duty cycle
#endif
#ifdef BUTTON_PIN
#ifdef BUTTON_LED
#if BUTTON_LED != LED_BUILTIN
 pinMode(BUTTON_LED, OUTPUT);        // special LED indicator of the trigger
#endif
 digitalWrite(BUTTON_LED, LOW);      // flush LED indicator of the trigger
#endif
 pinMode(BUTTON_PIN, INPUT);         // pulldown push button or switch
 // attachInterrupt(BUTTON_INT, button_int_handler, BUTTON_SLOPE);
```

```
 // NB: attachInterrupt is incompatible with hardcoding vectors via ISR macro
 EICRA = (EICRA & (~BUTTON_INT_SLOPE)) | BUTTON_INT_SLOPE;
 EIFR&=~BUTTON_INT_CHECK;            // clear stale external IRQ flag (paranoia)
 EIMSK|=BUTTON_INT_CHECK;            // toggle its mask (to enable)
#ifdef BUTTON_GAP
 noInterrupts();                     // protected code cei()
 TCNT2 = TCCR2A = TCCR2B = TIMSK2 = 0;   // reset Timer 2 registers
 // clear Overflow Flag _BV(TOV2) (and OCF2A,B) to avoid bogus interrupts
 TIFR2 &= ~0x7;
 TIMSK2 |= 1;                        // enable Timer 2 Overflow Interrupt
 interrupts();                       // enable all interrupts (unfreeze) sei()
#endif // GAP
#endif // PIN

#ifdef FRQCNT_INT
 pinMode(FRQCNT_PIN, INPUT);         // connector on the photodiode pulldown
 // attachInterrupt(FRQCNT_INT, frqcnt_int_handler, FRQCNT_SLOPE);
 EICRA = (EICRA & (~FRQCNT_INT_SLOPE)) | FRQCNT_INT_SLOPE;
 EIFR&=~FRQCNT_INT_CHECK;            // clear stale external IRQ flag (paranoia)
 EIMSK|=FRQCNT_INT_CHECK;            // toggle its mask (to enable)
#ifdef ACSR_CONFIG                   // by default, analog comparator is disabled
 ACSR = ACSR_CONFIG|(1<<ACD);        // connector on DP7 (analog comparator AIN1)
 DIDR1 = 0;                          // disable Digital Input on AIN0,1 see p.204
#endif /* ACSR_CONFIG */
#endif /* FRQCNT_INT */
#ifdef _UARTOUT_                     // open serial communication port
 printf_begin();                     // enable printf stream (without decimals!)
#else                                // arduino legacy Serial-only setting
 Serial.begin(SERIAL_BPS);
 while(!Serial);                     // wait for it to connect (native USB only)
#if defined(DEBUG) && DEBUG>1
 Serial.print(_UBRRH,HEX); Serial.print(_UBRRL,HEX); // 010 since (16000000/4/115200-1)/2 = 16.86
 Serial.write(' '); Serial.print(_UCSRA,HEX);        //   2
 Serial.write(' '); Serial.print(_UCSRB,HEX);        //  B8
 Serial.write(' '); Serial.print(_UCSRC,HEX);        //   6
 Serial.flush(); printlf();
#endif /* DEBUG>1 */
#endif /* _UARTOUT_ */
#ifdef readVcc_nsamples              // UNO boots as ADMUX=_BV(REFS0) (DEFAULT)
 restore_mux(_BV(REFS0));            // a dummy read sets ADMUX up after boot
 Vcc=readVcc(64);                    // check the actual value of Vcc [mV]
 uCtemp=read_uCtemp();               // uC temperature [C]
#endif /* readVcc_nsamples */
 print_info(0x5);                    // info on uC and its pwm setting
 digitalWrite(LED_BUILTIN, LOW);     // turn the onboard LED off = "done"
}

/* select a fixed-span WGM and CS best suited to reproduce a given total span
   f=clock/freq, preserve the duty cycle, turn pwm on by setting COM, and
   return the new WGM in high bits 4..7 and CS in low bits 0..3 */
unsigned char pwm_set_fwgm(pwm_span_t f) {
 unsigned char cs,i,c, pwm_choice=0;
 pwm_span_t s,d,ds=ULONG_MAX;        // total count and its deviation from f
#if PWM_DEBUG>2
 unsigned char k;
#endif /* PWM_DEBUG>2 */
 for(cs=0; cs++< PWM_MAX_DIVISOR; ) // all possible clock scales (CS)
   for(i=0; i++< PWM_MAX_BITS; )     // all possible timer resolutions 1,2,3
     for(c=2; c--; ) {               // 0 phase-correct, 1 fast pwm
       s = (pwm_span_t) _PWM_DIV(cs) * _PWM_CYCLE(c,_PWM_TOP_FIXED(i));
       // instead of _PWM_FREQ(cs,_PWM_CYCLE(c,_PWM_TOP_FIXED(i)));
       d = f>s ? f-s : s-f;
#if PWM_DEBUG>2
       if( (k=(d <= ds)) ) {         // -Wparentheses can be a pain ...
#else
       if(d <= ds) {
#endif /* PWM_DEBUG>2 */
         pwm_choice = (cs&(0x7)) | (c<<3) | (i<<4) ;
         ds = d;
       }
#if PWM_DEBUG>2
       // NB: Arduino.h redefines round as a macro (!) returning long int,
       // while round from math.h returns (double) just like roundf !
       // utf encodings: \u0394 Delta, \u03b4 delta, \u03bd nu, \u2192 ->
       printf_P(PSTR("# %u %4ux%-4u \u2192 %lu \u03b4=%lu %c\n"),
                cs, _PWM_DIV(cs),
```

```
                _PWM_CYCLE(c,_PWM_TOP_FIXED(i)), s, d, k?'*':' ');
       //                round(s), (unsigned long int) floor(d) );
       // if(d<100) printf_P(PSTR(".%lu"),round((d-floor(d)) * 1000) );
#endif /* PWM_DEBUG>2 */
     }
 d = pwm_get() & 0x3FF;                // current duty cycle (per mille)
 c = (pwm_choice>>4) | (pwm_choice&(0x1<<3) ? (0x1<<(1+PWM_TIMER1)) : 0);
#if PWM_DEBUG>1
 printf_P(PSTR("  set t=%u %u\u2030 %02hx \u2192 WGM=%hi CS=%hi\n"),
          _PWM_CYCLE( (pwm_choice>>4)&1, _PWM_TOP_FIXED(pwm_choice>>4) ),
          (unsigned int) d, pwm_choice, c, pwm_choice&(0x7) );
#endif /* PWM_DEBUG>1 */
 pwm_set_wgm(c);                       // set new WGM
 // set Clock Scale (CS) bits (0..2) and fixed frequency WGM for all timers
 // (TODO: WGM redundant) and turn pwm signal on via COM bits 10 of TCCRxA
#if PWM_TIMER1
 PWM_TCCRB = (PWM_TCCRB&( ~(0xF | (1<<PWM_WGM_MSB)) )) | (pwm_choice&(0xF));
 PWM_TCCRA = (PWM_TCCRA&(~(0x3 | (0x3<<PWM_COM)))) | ((pwm_choice>>4)&0x3) | (0x2<<PWM_COM);
#else
 PWM_TCCRB = (PWM_TCCRB&( ~(0xF | (1<<PWM_WGM_MSB)) )) | (pwm_choice&(0x7));
 PWM_TCCRA = (PWM_TCCRA&(~(0x3 | (0x3<<PWM_COM)))) | ((pwm_choice&(1<<3))>>2) | 0x1 | (0x2<<PWM_COM);
#endif
 if(d && d<1000) pwm_set(d);           // restore previously used duty cycle
 else pwm_set_duty(PWM_TOP_FIXED>>1);  // set default 50% duty cycle
 return (c<<4)+(pwm_choice&(0x7));     // WGM in high 4..7 + CS in low 0..3
}

/* select a variable-span WGM and CS best suited to reproduce a given total
   span f=clock/freq, preserve the duty cycle, turn pwm on by setting COM
   return the new WGM in high bits 4..7 and CS in low bits 0..3

   NB: in WGM's with variable TOP, after dividing on the pwm clock scale (CS),
   the resolution (aka span or count) of the period T of the pwm signal is
   given by an unsigned integer in the range of 0..0xFFFF or 0xFF for Timers 1
   or 0 and 2, respectively. Assuming steps of dT=1 and T>>1, relative steps in
   frequency f = 1/T and period T are equal : df / f = dT / T.

   NB: Variable frequency WGM's on A-pins of Timers 0 and 2 have fixed 50% duty
   cycle d and actual peiod 2T (i.e., T gives half-period) because T is stored
   in OCRA; A-pin WGM's with variable d exist only on Timer 1 with T in ICR1.
*/
unsigned char pwm_set_vwgm(pwm_span_t f) {
 unsigned char cs,c, pwm_choice=0;  // retained WGM : values of cs and c
 pwm_span_t    s,ss, top_choice=0;  // retained TOP : value of timer span
 unsigned int d,ds=UINT_MAX;        // deviation of total span from f
#if PWM_DEBUG>2
 char k=0;                          // k=1 signals new suggestion retained
#endif /* PWM_DEBUG>2 */
 for(cs=0; cs++< PWM_MAX_DIVISOR; ) // check all possible clock scales (CS)
   /* additional period divisor c  min/max values of C for timers 0 and 2
       Timer   pin A  pin B       pin (2-PWM_PIN_TYP) (1<<(2-PWM_PIN_TYP))
         1     1,2,4   1,2         A       2                4
        0,2      2,4   1,2         B       1                2
     c=1 corresponds to fast WGM; c=2 is either a special period/2 WGM for
     A-pins or phase-correct WGM; c=4 on A-pins is a combination of the two */
   for(c= PWM_TIMER&1 ? 1 : 2-PWM_PIN_TYP; c <= (1<<(2-PWM_PIN_TYP)); c<<=1) {
     s = (unsigned long int) _PWM_DIV(cs)*c;
     ss = f/s;                      // required max timer counter value (TOP)
     if( (f%s) > (s>>1) ) ss++;     // rounding correction of integer division
     if( ss > 0 && ss <= PWM_OCR_MAX ) {
#if PWM_DEBUG>2
       k = 0;
#endif /* PWM_DEBUG>2 */
       /* (!cnt || (ss >= TARGET_PWM_SPAN_MIN && f/ss <= TARGET_F_ASTEP_MAX)) */
       s*=ss;                       // min rel step dT/T = df/f equal 1/ss
       d = f>s ? f-s : s-f;
       if(d <= ds || (f/d) > TARGET_PWM_SPAN_MAX  ) {
         // NB: outside the target interval, comparing ss vs top_choice helps
         // achieving an "optimal" rel error (neither too big, nor too small)
         if( ! pwm_choice ||
             (ss < TARGET_PWM_SPAN_MIN && ss >= top_choice) ||
             (ss > TARGET_PWM_SPAN_MAX && ss <= top_choice) ||
             (ss>=TARGET_PWM_SPAN_MIN && ss<=TARGET_PWM_SPAN_MAX) ) {
           pwm_choice = (cs&(0x7)) | (c<<4);
           top_choice = ss;
           ds = d;
```

```
#if PWM_DEBUG>2
          k =1;
#endif /* PWM_DEBUG>2 */
        }
      }
#if PWM_DEBUG>2
      // utf encodings: \u0394 Delta, \u03b4 delta, \u03bd nu, \u2192 ->
      printf_P(PSTR("# %u %4ux%u %lu \u2192 %lu \u03b4=%u %c\n"),
               cs, _PWM_DIV(cs),c, ss, s, d, k?'*':' ');
#endif /* PWM_DEBUG>2 */
    }
  }
 d = pwm_get() & 0x3FF;              // current duty cycle (per mille)
 c = (1<<(PWM_WGM_MSB-PWM_WGM_LSB+2)) | (PWM_TIMER1^1) | (pwm_choice>>6)
   | (pwm_choice&(1<<4) ? (1<<(1+PWM_TIMER1)) | (PWM_TIMER1<<1) : 0);
#if PWM_DEBUG>1
 printf_P(PSTR("  set t=%u %u\u2030 %02hx \u2192 WGM=%hi CS=%hi\n"),
          (unsigned int) top_choice, d, pwm_choice, c, pwm_choice&(0x7) );
#endif /* PWM_DEBUG>1 */
 // set Clock Scale (CS) bits (0..2) and variable frequency WGM for all timers
 PWM_TCCRB = (PWM_TCCRB&(~0xF)) | (1<<PWM_WGM_MSB) | (pwm_choice&(0x7));
 pwm_set_wgm(c);                     // set new WGM
#ifdef PWM_APIN
 // setting COM bits to 01 for A-pin WGM with TOP stored in OCRA helps set_top
 // since this turns pwm on, for consitency, turn it on in all other cases
 if(c&(0x1))
   PWM_TCCRA = (PWM_TCCRA & (~(0x3<<PWM_COM))) | (0x1<<PWM_COM);
 else
#endif
   PWM_TCCRA = (PWM_TCCRA & (~(0x3<<PWM_COM))) | (0x2<<PWM_COM);
 // deduce the extra step in fast modes and set timer counter span (period)
 pwm_set_top(top_choice-((pwm_choice>>4)&1));
 // print_info(0x10);
#ifdef PWM_APIN
 if(! (c&(0x1)) )                    // not for A-pin WGM with TOP stored in OCRA
#endif
   pwm_set(d);                       // restore duty cycle
 return (c<<4)+(pwm_choice&(0x7));   // WGM high 4..7 + CS low 0..3
}

/*
    CLI interface (regexp notation [abc] for "one of" and [a-c] for ranges)
    ------------------------------------------------------------------------

    [0-9]     enter general numeric values <n> as unsigned integers (<65535)
    0         followed by one or more [0-7] digits enters <n> as octal number
    0x or 0b  enter <n> as hexadecimal (radix 16) or binary numbers, resp
    [A-Fa-f]  preceeded by 0x are interpreted as hexadecimal digits
    <space>   terminates the hexadecimal number in cases when the following
              key, such as d, c, or f, is intended as a CLI command
              NB: technically, <space> simply resets the radix to 10,
              so any non-[0-9] characters would be accepted as a command
    :         same as <space> (legacy)

    [+-]      define positive or negative stepping direction for certain CLI
              commands issued without <n> arguments, notably c and b

    The numeric argument <n> is, in most cases, optional, and some action,
    such as displaying relevant information, occurs when <n> is not supplied

    <n>?      display information flagged, optionally, by bits in <n>
    I         (legacy) same as ? but more complete and unconditional

    z         show current frequency meter data, reset frequency meter counter
    Z         toggle IRQ source between digital pin (DP) <-> comparator (COMP)

    <n>[k.f]  define kHz Hz, and mHz parts (each<65535) of the target frequency
    <n>.<m>   in frequency input (f or F), n is stored as frequency [Hz] while
              m defines the fractional part [mHz]; the three parts of the
              target frequency are preserved and can be reused implicitly

    <n>f      set the closest to the target frequency fixed top WGM, turn on
    <n>F      set the closest to the target frequency variable top WGM mode
              for example, to set 1250.5 Hz, enter 1k250.500f or 1250.500f

       S      in most cases, start/stop PWM by toggling MSB bit of COM
```

```
       i      toggle LSB of COM (shifting phase, or set PWM signal on A pins)
       w      toggle Fast <-> Phase Correct WGM's bit in WGMx
    <n>b      for pin 9 or 10 which use Timer 1 : <n> = 1, 2, and 3 sets timer
              resolution of fixed frequency WGM's to 8, 9, 10-bit, respectively;
              without <n>, b cycles through these resolutions stepping by +/-1
    <n>c      define Clock Select (CS) or cycle through all possible CS values
    <n>t      for variable frequency WGM's, set the value of TOP (= frequency)
    <n>d      set the value of the OCRx register (<=TOP) giving the duty cycle
    <n>D      set the duty cycle after interpreting <n> < 1000 as promille value
*/

void loop() {
 static unsigned char key=';';     // initilize the stale key at first pass
 static unsigned char k=0;         // number of entered digits in val
 static unsigned char radix=10,frac=0;
 static unsigned char nk=0, nd=0;  // indices of last entered keys and digit
 static unsigned int  val=0;       // general digital value argument
 static unsigned int iopt=0x5;     // info config of the ? command
 static char stp=1;                // stepping direction can be either +1 or -1
 // This CLI assumes no line endings present, just plain stream of characters.
 // positive integer number argument may prepend certain keys
 while(Serial.available()) {       // read settings and commands
   key = Serial.read(); nk++;
   // NB: Arduino.h includes <ctype.h> and complements its isdigit and isxdigit
   // by their own simple 'inline' wrappers isDigit and isHexadecimalDigit, see
   // hardware/arduino/avr/cores/arduino/WCharacter.h and
   // https://stackoverflow.com/questions/1759300/when-should-i-write-the-keyword-inline-for-a-function-method
   if( isdigit(key) || (radix==16 && isxdigit(key)) ) {
     if(k && !val && nk-nd==1 && key < '8') radix=8;
     if(radix >= 10 || key < '0' + radix) {
       val*= radix;
       val+= toupper(key)-(isdigit(key)?'0':'A'-10);
       k++;                        // count valid digits of the numerical value
       nd = nk;
     }
#if defined(DEBUG) && DEBUG>1
     Serial.print((char) key); Serial.print(F("->")); Serial.println(val);
#endif
   }
   // Octal, binary, or hexadecimal number input exploits counters nd and nk to
   // recognize respective prefix sequences 0[1-7], 0b, or 0x. Hexadecimals
   // also exclude certain single-letter 'legacy' keys that are already in use,
   // notably c, b, d. To keep using them, ':' is used to terminate hex input.
   else if ((key==':' || key==' ') && radix==16)
     radix = 10;                   // end hexadecimal input
   else if ((key=='x' || key=='b') && k==1 && !val && nk-nd==1) {
     k--;
     radix = (key=='x'? 0x10 : 2); // hexadecimal (16) or binary (2) numbers
   }
   else {
      switch(key) {  // handle keys possibly preceeded by a numerical value
        case '?':
          if(k) iopt = val;        // flag information to show
          break;
        case 'k':                  // set kHz part of the target frequency
          if(k) fkHz = val;        // more precisely, times kHz_FACTOR
          break;
        case '.':
          if(k) f_Hz = val;        // Hz part of the target frequency (<65535)
          frac^=1;                 // toggle fractional part numeric entry
          break;
        case 'f':                  // prpose PWM setting for target frequency
        case 'F':
          if(k) {                  // -Wparentheses can be a pain ...
            if(frac) {
              fmHz = val;          // enter mHz part of the target frequency
              frac^=1;
            }
            else f_Hz = val;
          }
          break;
#ifdef PWM_PIN
        case 'c':    // Clock Select bits CSx2:0 define the scale of Timer x
          // if(k) { Serial.println(k,HEX); Serial.println(val); }
          // else Serial.println(stp,HEX);
          // Serial.println(PWM_TCCRB,BIN);
```

```
          PWM_TCCRB = (PWM_TCCRB & (0xFFF8)) | ((k ? (unsigned char) val : ((unsigned char)PWM_TCCRB+stp)) & 0x7
              ) ;
          frqcnt_restart();        // restart period counters
          break;
        case 'd':                  // set PWM duty cycle (raw)
          if(k) pwm_set_duty((pwm_duty_t) (val&(0x3FF)));
          break;
        case 't':                  // set PWM counter TOP value (if applicable)
          if(k) pwm_set_top(val);
          break;
        case 'D':                  // set PWM duty cycle (per mille)
          if(k) pwm_set(val);
          break;
        case 'W':                  // set WGMx bits 0..2 (x=0,2) or 0..3 (x=1)
          if(k) pwm_set_wgm(val);
          break;
#if PWM_TIMER == 1                 // only for pins 9,10 (Timer 1)
        case 'b':                  // correct values are 1,2,3
          pwm_set_bits((k ? (unsigned char) val : ((unsigned char)PWM_BITS+stp)) & 0x3);
          frqcnt_restart();        // restart (half-)period counter
          break;
#endif
#endif /* PWM_PIN */
      }              // end handling keys which accept numerical arguments
      val=k=0;       // reinit digital argument
      radix=10;      // default to digital number input
      switch(key) {  // handle switches
        case 'Z':                    // frequency counter method DP <-> AC
          if(EIMSK&FRQCNT_INT_CHECK) {
            EIMSK^=FRQCNT_INT_CHECK; // toggle external DP IRQ mask (to disable)
            EIFR&=~FRQCNT_INT_CHECK; // clear its flag (paranoia)
            ACSR^=(1<<ACD);          // toggle AC IRQ mask (to enable)
          } else {
            ACSR^=(1<<ACD);          // toggle AC IRQ mask (to disable)
            EIFR&=~FRQCNT_INT_CHECK; // clear stale external IRQ flag (paranoia)
            EIMSK^=FRQCNT_INT_CHECK; // toggle its mask (to enable)
          }
        case 'z':
          print_info(0x8);         // frequency meter results
          frqcnt_restart();        // restart period counters
          break;
        case '+':
          stp = 1;   // step registers forward
          break;
        case '-':
          stp =-1;   // step registers backward
          break;
        case 'S':    // start/stop pwm
          compound_pwm_toggle();
          break;
        case 'i':                  // invert pwm signal (phase shift of pi)
          pwm_invert();
#ifdef BUTTON_LED                  // LED indicator of PWM state
          digitalWrite(BUTTON_LED, PWM_ON ? HIGH : LOW );
#endif
          frqcnt_restart();        // restart (half-)period counter
          break;
        case 'w':
          pwm_toggle_wgm();        // switch fast <-> phase correct modes
          frqcnt_restart();        // restart (half-)period counter
          break;
        case 'D':
          print_info(0x2);         // duty cycle info (promille)
          break;
        case 'f':
          print_info(0x20);        // pwm setup for non-zero target frequency
          if( (f_Hz>0 || fkHz>0) &&
              pwm_set_fwgm(round(PWM_SPAN(TARGET_PWM_FREQ))) ) {
            frqcnt_restart();      // restart (half-)period counter
            print_info(0x4);       // new fixed frequency pwm wgm
          }
          break;
        case 'F':
          print_info(0x20);
          if( (f_Hz>0 || fkHz>0) &&
              pwm_set_vwgm(round(PWM_SPAN(TARGET_PWM_FREQ))) ) {
```

```
            frqcnt_restart();       // restart (half-)period counter
            print_info(0x4);        // new variable frequency pwm wgm
          }
          break;
#if defined(DEBUG) && DEBUG>=1      // confirm COM, WGM, and CS settings
        case 'W':
        case 'c':
        case 't':
          print_info(0x14);          // register dump
          break;
#endif
        case 'I':
          print_info(UINT_MAX);     // full startup and current info (legacy)
          break;
        case '?':
          print_info(iopt);         // various info specified by iopt flags
          break;
        case 'd':
          print_info(0x4);          // detailed info on pwm setup
        case 'Q':
        case ';':     // line ending, helpful for commands with numerical args
        default:      // silently skip all other (incl. unknown) keys
          break;
      } /* end non-value key treatment */
    }   /* end handling symbolic keys  */
 }      /* end command line operation  */
 #if defined(DEBUG) && DEBUG>1      // visualize button gap timing
 while(TCCR2B&0x7) Serial.println(TCNT2,DEC);
 #endif
}
```

Tout programmation de PWM est délégué au fichiers `pwm.h` et `pwm.c`.

```
/*  pwm c-library to provide pwm signal on a given (hard-coded) Arduino pin
    Copyright (C)2025 Dmitrii Sadovskii (sadovski at dvoirah dot ovh)

    This program is free software: you can redistribute it and/or modify
    it under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    This program is distributed in the hope that it will be useful,
    but WITHOUT ANY WARRANTY; without even the implied warranty of
    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
    GNU General Public License for more details.

    You should have received a copy of the GNU General Public License
    along with this program.  If not, see <https://www.gnu.org/licenses/>.
*/
#ifndef _PWM_H
#define _PWM_H
// ATTN: pwm.c is compiled separately and needs a correct PWM_PIN setting!
#if defined __has_include
#  if __has_include ("pwm_pin.h")
#    include "pwm_pin.h"
#  endif
#endif
#ifndef PWM_PIN
#define PWM_PIN 9        // PWM @490 Hz (away from DP2,3 and not used for SPI)
#endif
#define PWM_DEBUG (defined(_UARTOUT_) && defined(LIB_DEBUG) ? LIB_DEBUG : 0)
/*
  https://www.arduino.cc/reference/en/language/functions/analog-io/analogwrite/
  PWM on Uno, Nano, Mini: 490 Hz on pins 3, 9, 10, 11,  980 Hz on pins 5 and 6

  NB: PWM outputs on pins 5 and 6 may have higher-than-expected duty cycles
  because of interactions with functions millis() and delay(), which use the
  same internal timer. This will be noticed mostly on low duty cycle settings
  (e.g. 0-10) and may result in a value of 0 not fully turning off the output.

  ATTN: since this may lead to perturbations on the external interrupt pin 2,3
  https://forum.arduino.cc/t/pwm-appears-to-be-conflicting-with-external-interrupts/117351
  avoid using pin 3 but rather one of 9,10,11 if you use external interrupts
*/
#include <Arduino.h>     // for stand-alone compilation to know LOW, HIGH, etc
// NB: Arduino.h redefines round as a macro (!) returning long int,
```

```
// cf round(x) in /hardware/arduino/avr/cores/arduino/Arduino.h on line 96
// while round from math.h returns (double) just like roundf !
// round_u is a simplified version of Arduino.h round for non-negative x
// NB: the actual computation is performed by compiler, not by preprocessor
#define round_u(x) ((unsigned long int)((x)+0.5))
#include <limits.h>
// see https://avrdudes.github.io/avr-libc/avr-libc-user-manual
// see hardware/arduino/avr/cores/arduino/ and hardware/tools/avr/avr/include/
#include <avr/io.h>      // for direct uC register access
#include <avr/interrupt.h>
#define TARGET_PWM_SPAN_MIN (1<<4)
#define TARGET_PWM_SPAN_MAX (min((1<<12),PWM_MAX))
// relative target min/max frequency change [%%] due to one timer counter step
#define TARGET_F_RSTEP_MAX round_u(1000/TARGET_PWM_SPAN_MIN)  // 62.5 promille
#define TARGET_F_RSTEP_MIN round_u(1000/TARGET_PWM_SPAN_MAX)  // 0.97 promille
// https://forum.arduino.cc/t/how-to-use-additional-c-files-in-a-sketch/41402
#if defined(__cplusplus)
#define _C_ extern "C"   // declare external c-functions within c++ (arduino)
#else
#define _C_ extern       // plain c style, just in case ...
#endif
// NB: normally, under gcc, __BASE_FILE__ indicates the top file invoking pwm.h.
// However, the preprocessor offers no means to compare strings (only chars)
// Thus, we cannot determine automatically whether the inclusion occurs in the
// main sketch file (.ino) or in the library file (.c)

/*
 https://www.arduino.cc/reference/en/language/functions/analog-io/analogwrite/
 analogWrite values go from 0 (always off) to 255 (always on)
 (the PWM mode, which is pre-cofigured on the UNO for all pins, is 8 bit)
 analogWrite has nothing to do with the analog pins and analogRead
 the pin will generate a steady rectangular wave of the specified duty cycle
 until the next analogWrite(), digitalRead(), or digitalWrite() on this pin

 https://passionelectronique.fr/pwm-arduino/
 https://passionelectronique.fr/wp-content/uploads/Atmel-ATmega328P-datasheet.pdf?pseSrc=pgPwmArduino
 The ATmega168P/328P chip has three PWM timers, controlling 6 PWM outputs.
 Timer   Timer   Arduino    IDE default on the UNO
 compare output -> pin
 OCR0A   OC0A      6    Fast PWM Mode          8-bit @976,56Hz, 255 steps
 OCR0B   OC0B      5
 OCR1A   OC1A      9    Phase Correct PWM Mode 8-bit @490,20Hz, 510 steps
 OCR1B   OC1B     10
 OCR2A   OC2A     11    Phase Correct PWM Mode 8-bit @490,20Hz, 510 steps
 OCR2B   OC2B      3
 NB: OC[012][AB] are not known to avr/io.h, OCR1[AB] are 16-bit
*/

#define PWM_BOTTOM 0x0000

// DP5 and 6 use the 8-bit Timer 0 and can only work with 8-bit resolution.
// They are configured by default for Fast (simple) PWM with frequency
// f = 16 MHz / 64 / 256 = 976,5625 (the simple duty cycle of 255+1 steps)
// (here notice that 16*2^20/64/256 = 1024)

#if PWM_PIN==5          // default to Fast PWM Mode 8-bit @976,56 Hz
#define PWM_OCR OCR0A   // Timer 0 register A
#define PWM_COM COM0A0  // LSB of the 2 compare output bits in TCCR0A
#endif
#if PWM_PIN==6          // default to Fast PWM Mode 8-bit @976,56 Hz
#define PWM_OCR OCR0B   // Timer 0 register B
#define PWM_COM COM0B0  // lsb of the 2 compare output bits in TCCR0A
#endif
#if PWM_PIN==5 || PWM_PIN==6
#define PWM_TIMER 0     /* Timer 0 with 8-bit counter register, ch.14 */
#define PWM_OCRA  OCR0A // register which may hold the variable TOP value
#define PWM_TCCRA TCCR0A
#define PWM_TCCRB TCCR0B
#endif
/*
  DP3 and 11 use Timer 2 (8-bit) and can only work with 8-bit resolution; DP9
  and 10 use Timer 1 (16-bit) and support more precise 9- and 10-bit pwm modes
  all DP9, 10, 11, and 3 are configured for Phase Correct PWM with frequency

  f = 16 MHz / 64 / 510 = 490,196 (510 = 2*255 steps in the duty interval)
```

```
  NB: The phase-correct PWM mode and the phase-and-frequency-correct PWM mode
  are based on a dual-slope operation with counter going repeatedly from BOTTOM
  (0) to TOP and back to BOTTOM.  Due to the symmetry of the dual-slope PWM
  modes, the latter are preferable in motor control applications.
*/
#if PWM_PIN==9          // default Phase Correct PWM mode 8-bit @490,20 Hz
#define PWM_OCR OCR1A   // 16-bit Timer 1 register for channel A
#define PWM_COM COM1A0  // lsb of the 2 compare output bits in TCCR1A
#endif
#if PWM_PIN==10         // default Phase Correct PWM mode 8-bit @490,20 Hz
#define PWM_OCR OCR1B   // 16-bit Timer 1 register for channel B
#define PWM_COM COM1B0  // lsb of the 2 compare output bits in TCCR1A
#endif

#if PWM_PIN==9 || PWM_PIN==10
#define PWM_TIMER 1     /* Timer 1 with 16-bit counter register, ch.15 p.109 */
#define PWM_MAX 0xFFFF  // maximum value of the Timer 1 counter (decimal 65535)
#define PWM_OCRA  OCR1A // register which may hold the variable TOP value
#define PWM_TCCRA TCCR1A
#define PWM_TCCRB TCCR1B
#define PWM_TCCRC TCCR1C
// Atmel ATmega328P Waveform Generation Modes (WGM) for Timer 1 are described
// by bits _BV(WGMx3)!_BV(WGMx2) of TCCRB and _BV(WGMx1)|_BV(WGMx0) of TCCRA
#define PWM_BITS (PWM_TCCRA&(0x3))  // 1,2,3 for 8-, 9-, and 10-bit wgm's
#define PWM_BITS_MAX 3              // resolution equals 2^(PWM_MAX_BITS+7)-1
#define PWM_WGM ((PWM_TCCRB&(0x18))>>1 | PWM_BITS)
// the upper value (TOP) of the Timer 1 count sequence for res=1,2,3 in
// 8-bit (default), 9-bit, and 10-bit pwm modes, equals 0x00FF, 0x01FF, 0x03FF
#define _PWM_TOP_FIXED(res) (0x3FF>>(~(res)&(0x3)))
// examine WGMx0,1 (bit 0,1 of TCCRA) to determine resolution 8, 9, 10-bit
// #define PWM_TOP_FIXED (0x3FF>>(~PWM_TCCRA&3) | 0xFF) // paranoid to have >= 0xFF
#define PWM_TOP_FIXED (0x3FF>>(~PWM_TCCRA&(0x3)))
// for WGM modes on pins 9,10 (Timer x=1), the value of TOP is stored in OCRxA
// or in the Input Capture Register (ICR1) for modes 9,11,15 and 8,10,14, resp.
#define PWM_TOP_VAR ((PWM_TCCRA & (1)) ? PWM_OCRA : ICR1)
// check WGMx2 (bit 3 of TCCRxB, x=1) : 0 phase correct, 1 fast pwm (p.109)
#define PWM_WGM_MODE (PWM_TCCRB & (1<<3))
// check WGMx3 (bit 4 in TCCRxB, x=1) : TOP is fixed (0) or given by OCRxA (1)
#define PWM_TOP_MODE (PWM_TCCRB & (1<<4))
// symbolic representation of various PWM modes, see datasheet, 15-5 p.109
#define PWM_WGM_MODE_SYMBOLS { \
  "-",     "pc~8",  "pc~9", "pc~10", "CTC-O", "f~8", "f~9", "f~10", \
  "pfc~I", "pfc~O", "pc~I", "pc~O",  "CTC-I",   "?", "f~I", "f~O" }
#define PWM_WGM_MASK 0xF
// non-PWM modes of Timer 1 are 0 (norm), 4 (CTC-O), 12 (CTC-I), and 13 (void)
#define PWM_MODE (PWM_TOP_MODE ?                                \
                  (PWM_WGM_MODE?(PWM_TCCRA&(2)):1) : (PWM_TCCRA&(3)) )
/*
   ugly, the alternative being to check for the twiddle character '~' in the
   designation strings of the WGM mode. However, this has to be delegated to
   the user space, and only if these strings are loaded in a char * array :

     #include <string.h>
     const char *pwm_wgm[] = PWM_WGM_MODE_SYMBOLS;
     #define PWM_MODE (strchr(pwm_wgm[PWM_WGM],'~'))
     // valid PWM mode, works for any Timer 0,1,2 of ATmega328P
*/
#else                   /* for Timers 0 and 2 with 8-bit counter registers */
#define PWM_MAX 0x00FF  // maximum value of the 8-bit counter
#define PWM_TOP_FIXED PWM_MAX // highest value in the count sequence
#define _PWM_TOP_FIXED(res) PWM_MAX
// on pins 3,11 (Timer x=2) or 5,6 (x=0) OCRxA holds TOP for WGM modes 5,7
#define PWM_TOP_VAR PWM_OCRA
#define PWM_BITS 1      // 8-bit mode only (limited by timer register)
#define PWM_BITS_MAX 1  // hence the resolution of the counter can only be 0xFF
// bits _BV(WGMx2)  and _BV(WGMx1)|_BV(WGMx0) for Timers 0 and 2
#define PWM_WGM ((PWM_TCCRB&(0x08))>>1 | (PWM_TCCRA&(0x3)))
// check WGMx1 (bit 1 of TCCRxA, x=0,2) : 0 phase correct, 1 fast pwm
#define PWM_WGM_MODE (PWM_TCCRA & (0x2))
// check WGMx2 (bit 3 in TCCRxB, x=0,2) : TOP is fixed (0) or given by OCRxA (1)
#define PWM_TOP_MODE (PWM_TCCRB & (1<<3))
// symbolic representation of various PWM modes, see datasheet, 14-8 p.86
#define PWM_WGM_MODE_SYMBOLS { \
  "-",  "pc~",  "CTC", "f~", "?", "pc~O", "?", "f~O"};
#define PWM_WGM_MASK 0x7
// non-PWM modes of Timers 0,2 are 0 (norm), 2 (CTC), 4 (void), and 6 (void)
```

```
#define PWM_MODE (PWM_TCCRA&(1))     // simply check WGMx0 for odd WGM modes
#endif                  /* PWM_PIN==9 || PWM_PIN==10 */

#if PWM_PIN==11         // default to Phase Correct PWM Mode 8-bit @490,20 Hz
#define PWM_OCR OCR2A   // Timer 2 8-bit register A
#define PWM_COM COM2A0  // lsb of the 2 compare output bits in TCCR2A
#endif
#if PWM_PIN==3          // default to Phase Correct PWM Mode 8-bit @490,20 Hz
#define PWM_OCR OCR2B   // Timer 2 8-bit register A
#define PWM_COM COM2B0  // lsb of the 2 compare output bits in TCCR2A
#endif
#if PWM_PIN==3 || PWM_PIN==11
#define PWM_TIMER 2
#define PWM_OCRA  OCR2A
#define PWM_TCCRA TCCR2A
#define PWM_TCCRB TCCR2B
const unsigned char pwm_divisors[] = {0,0,3,5,6,7,8,10}; // Timer 2
#define PWM_MAX_DIVISOR 7                                // (0..7)
#else
const unsigned char pwm_divisors[] = {0,0,3,6,8,10,0,0}; // Timer 0 and 1
#define PWM_MAX_DIVISOR 5                                // (0..5) 6,7 ext
#endif

#define PWM_WGM_LSB 3   // WGM bit(s) LSB WGMx2 in TCCRxB equals 3 for all x
#define PWM_WGM_MSB (PWM_WGM_LSB+(PWM_TIMER&(1)))
#ifndef PWM_TOP_MODE    // TOP and frequency are variable (1), or fixed (0)
#define PWM_TOP_MODE (PWM_TCCRB&(1<<PWM_WGM_MSB))
#endif
#ifndef PWM_WGM_MASK    // 0..7 and 0..15 WGM modes of Timer 0,2 and 1 resp.
#define PWM_WGM_MASK (PWM_TIMER == 1 ? 0xF : 0x7)
#endif
#if ! defined(PWM_APIN) && PWM_COM==6
#define PWM_APIN        // the PWM_PIN is of type A (all Timers)
#endif
#define PWM_COM_VAL ((PWM_TCCRA>>PWM_COM)&3)
#define PWM_WGM_ODD (PWM_TCCRA&(1))
#ifdef PWM_APIN
#define PWM_PIN_TYP 0   // pin of type A
#else
#define PWM_PIN_TYP 1   // pin of type B
#endif
#ifndef PWM_MAX_BITS    // 1 for Timer 0,2 and 3 for Timer 1
#define PWM_MAX_BITS (PWM_TIMER==1?3:1)
#endif

/*
    Arduino timer scaling:
    common prescaler: 1, 2, 4, 8, 16, 32, 64, 128, 256
                      0  1  2  3   4   5   6    7    8   (value of CLKPR & 0xF)
    timer0 or timer1: 0, 1, 8, 64, 256, 1024
                      0  1  2   3    4     5   (value of TCCR0B or TCCR1B & 7)
                      external source on pin T1: 6 falling, 7 rising (p.110)
    timer2:           0, 1, 8, 32,  64,  128, 256, 1024
                      0  1  2   3    4     5    6,    7  (value of TCCR2B & 7)
*/

#define PWM_TOP pwm_get_top()
// the number of steps in the pwm cycle doubles for phase correct modes
// wgm here can be any non-0 for fast and 0 for phase-correct mode
#define _PWM_CYCLE(wgm,top) ((wgm) ? (top)+1 : (top)<<1)
// NB: arduino ide default for pin 9,10 is fcPWM with 2*255 steps per period
#define PWM_CYCLE (PWM_WGM_MODE ? PWM_TOP+1 : PWM_TOP<<1)
#define _PWM_DIV(cs) (1<<pwm_divisors[cs])
#define PWM_DIV  _PWM_DIV(PWM_TCCRB&(7))
// CLKPR defines the main uC frequency prescaler, common to all timers
#define SYSTEM_DIV (1<<(CLKPR & (0xF)))

// NB: F_CPU is (long int), so conversion to (double) keeps decimals
#define SYSTEM_CLOCK ( (double) F_CPU / SYSTEM_DIV )

// attn: steps here can be computed as _PWM_CYCLE(wgm,top) with wgm=0,1
#define _PWM_FREQ(cs,steps) (SYSTEM_CLOCK / _PWM_DIV(cs) / (steps))
#define PWM_FREQ (SYSTEM_CLOCK / PWM_DIV / PWM_CYCLE)
// assuming that the overall uC timer clock prescaler is left untached,
// compute the required total "resolution" required to reproduce f [Hz]
// this resolution is equivalent to the product (PWM_DIV * PWM_CYCLE) of
```

```
// the timer scaler (defined by the CS bits) times timer resolution (OCRx)
#define PWM_SPAN(f) (SYSTEM_CLOCK / f)
// ratio of system clock to frequency may be a large number of timer counts
typedef long unsigned int pwm_span_t;

#ifdef PWM_TIMER
#define PWM_TIMER1 (PWM_TIMER&(1)) // 1 for Timer 1, 0 for Timer 0 and 2
/* ATTN: normally in c, typdef's can appear anywhere. However, Arduino c++ IDE
   _ignores_ (!) any typedef's issued _after_ function definitions (seems a
   stupid bug), i.e., if any explicit function definitions appear here or
   in some other (unrelated) preceeding short-hand custom .h file !! */
#if PWM_TIMER == 1
typedef unsigned int  pwm_duty_t;  // only for pins 9,10 (16-bit Timer 1)
#else
typedef unsigned char pwm_duty_t;  // for other PWM pins (8-bit Timers 0 and 2)
#endif
#define PWM_OCR_MAX ((1LU<<(sizeof(pwm_duty_t)*CHAR_BIT)) - 1)
#endif /* PWM_TIMER */

// check the MSB bit COMx[AB]1 of Compare Output Mode (COM) bit sequence
#define _PWM_ON (PWM_TCCRA&(0x2<<PWM_COM))
#ifndef PWM_APIN                    // for B pins, checking COMxB1 suffices
#define PWM_ON (PWM_MODE && _PWM_ON)
#else                               // for A pins, further checks are needed
/* on A-type pins in modes with the variable TOP value, i.e., when the MSB of
   WGMx is set, and COMxA1=1, the PWM signal is, generally, not available.
   This holds for all odd WGM modes, i.e., when the LSB bit WGMx0 is set,
   notably modes 9,11,15 of Timer 1 (Table 15-5 on p.109) and 5,7 of Timer 0,2
   (cf Table 14-8 on p.86) because they use OCRxA to store the value of TOP.
   There is one special case with COMxA=01, the only case with COMxA1=0, with
   both the MSB of WGMx and WGMx0 (odd modes) set. Then OCRxA stores TOP/2 and
   a pwm signal of fixed 50% duty cycle is supplied to the A pin. */
#if PWM_TIMER == 1
// Even variable TOP modes 8,10,14 exist only for Timer 1. They store the value
// of TOP in its ICR1 and continue using OCR1A normally for the duty cycle.
#define PWM_ON (PWM_MODE &&                                             \
                ( PWM_TOP_MODE ?                                        \
                  (PWM_WGM_ODD ? PWM_COM_VAL==1 : _PWM_ON) : _PWM_ON ))
#else
#define PWM_ON (PWM_MODE && ( PWM_TOP_MODE ? PWM_COM_VAL==1 : _PWM_ON ))
#endif /* PWM_TIMER == 1 */
#endif /* PWM_APIN */

// declare external plain c functions which are stored separately in pwm.c

_C_ bool pwm_toggle();              // pwm mode on/off, returns PWM_ON state
// toggling the LSB COM bit inverts the mode in most cases
_C_ bool pwm_invert();              // returns the state of COM LSB

_C_ pwm_duty_t pwm_get_duty();      // raw duty cycle value 0..255 or 1023
_C_ pwm_duty_t pwm_set_duty(pwm_duty_t d);
// while PWM_TOP_FIXED macro gives the TOP value only for fixed frequency modes,
// this function provides its value also for variable frequency modes, including
// the special case of A-pin and COM=01 with the effective TOP equal 2*OCRxA
_C_ pwm_duty_t pwm_get_top();
// for modes with variable frequency, set the TOP value of the timer counter
_C_ pwm_duty_t pwm_set_top(pwm_duty_t d);

_C_ unsigned int pwm_get();         // return per mille parts 0..1000 of PWM_TOP
// NB: per mille (Latin meaning 'in each thousand') indicates parts per thousand
// aka promille and permille, the symbol being o/oo (the usual percentage o/o)
_C_ unsigned int pwm_set(unsigned int v);
// switch between 8,9,10-bit pwm modes while preserving (roughly) the duty cycle
// (WGMx 1:0 for Timer x=1 only) and return the (new) TOP value (for all timers)
_C_ unsigned char pwm_set_bits(unsigned char i);
#ifdef PWM_TIMER
// toggle WGMx1 or WGMx2 bits which distinguish fast <-> phase-correct modes
_C_ unsigned char pwm_toggle_wgm();
// toggle MSB bit of WGMx to change fixed <-> variable frequency pwm modes
_C_ unsigned char pwm_toggle_fwgm();
// set all WGMx bits 2:0 and 3:0 in i for Timers x=0,2 and 1, respectively
_C_ unsigned char pwm_set_wgm(unsigned char i);
#endif

#endif /* !_PWM_H */
```

```
#include "pwm.h"

unsigned int pwm_map(unsigned int x, unsigned int in, unsigned int out) {
 return (unsigned int) roundf((float) x * out / in);
}

// toggling the MSB Compare Output Mode (COM) bit switches PWM on/off
// COM bits: 00 PWM disabled (aka normal), 01 n/a, 10 PWM, 11 inverted PWM
bool pwm_toggle() {
#ifdef PWM_TCCRA
 PWM_TCCRA ^= (2<<PWM_COM);          // toggle the MSB COM bit on/off bit
 /* legacy code ...
 bool state = (PWM_TCCRA&(1<<PWM_COM));
 if(state) PWM_TCCRA &= ~(0x2<<PWM_COM);  // disable, OR MUST BE 0x3 !?
 else      PWM_TCCRA |=   0x2<<PWM_COM ;  // enable as normal
 */
 return PWM_ON;                      // NB takes care also of the case COM=01
#else
 return 0;
#endif
}

// toggling the LSB Compare Output Mode (COM) bit inverts the mode in most cases
bool pwm_invert() {
#ifdef PWM_TCCRA
 PWM_TCCRA ^= (1<<PWM_COM);          // toggle LSB COM bit on/off bit
 return (PWM_TCCRA & (1<<PWM_COM)); // current (new) state LSB COM bit
#else
 return 0;
#endif
}

// NB: in the special cases of variable TOP value on A pins, when OCRxA holds
// TOP, returning OCRxA implies automatically (!) the 50% duty cycle
pwm_duty_t pwm_get_duty() {          // current raw value of pwm duty cycle
#ifdef PWM_OCR
 return PWM_OCR;                     // return the value of OCRxA/B
#else
 return digitalRead(PWM_PIN);        // paranoidal case against screw-ups
#endif
}

// store the raw duty cycle value avoiding cases when PWM_OCR holds TOP
pwm_duty_t pwm_set_duty(pwm_duty_t d) {
#if PWM_TIMER == 1
 // enforce the upper limits 0x3F, 0x1F, 0xF of fixed TOP modes for Timer 1
 if( ! PWM_TOP_MODE ) d &= ((0x7F<<PWM_BITS) | 0xF);
#endif
#if defined(__cplusplus) && defined(DEBUG) && DEBUG>1
 Serial.println(d,DEC);
#endif
#ifdef PWM_OCR
#ifdef PWM_APIN                      // for A pins, OCRxA may hold TOP
 /* The duty cycle on A pins can be defined for any WGM mode with fixed TOP, or
    for even WGM modes 8,10,12,14 of pin 9 (Timer 1) which hold TOP in ICR1.
    In all other cases of variable TOP value, notably for pin 3 (Timer 2) or 5
    (Timer 0), and for odd modes 9,11,15 on pin 9 (Timer 1), TOP is stored in
    OCRxA and the duty cycle is fixed to 50% (when COM=01) */
 if( !PWM_TOP_MODE || !(PWM_TCCRA&(0x1)) )
#endif /* PWM_APIN */
   PWM_OCR = d;                      // store the value of the duty cycle
   return PWM_OCR;
#endif /* PWM_OCR */
}

// for modes with variable frequency, set the TOP value of the timer counter
// NB: when OCRxA holds TOP, the A pin provides no pwm signal unless COM=01
pwm_duty_t pwm_set_top(pwm_duty_t d) {
 if(PWM_TOP_MODE) {                  // for variable TOP modes only
#if PWM_TIMER == 1                   // for pins 9,10 using Timer x=1
   if(PWM_TCCRA&1) PWM_OCRA = d;     // TOP stored in OCR1A: WGM modes 9,11,15
   else ICR1 = d;                    // TOP in Input Capture Register: 8,10,14
#else                                // for pins 3,11 (Timer x=2) or 5,6 (x=0)
   PWM_OCRA = d;                     // OCRxA holds TOP in WGM modes 5,7
```

```
#endif /* PWM_TIMER == 1 */
 }
 return pwm_get_top();
}

// while PWM_TOP_FIXED macro gives the TOP value only for fixed frequency modes,
// this function provides this value also for variable frequency modes
pwm_duty_t pwm_get_top() {
 pwm_duty_t d;
 if(PWM_TOP_MODE) {                 // for WGM modes which allow variable TOP
   d = PWM_TOP_VAR;
#ifdef PWM_APIN
   // in the special case of A-pin and COM=01, the effective TOP equals 2*OCRxA
   if( (PWM_TCCRA&(0x1)) && ((PWM_TCCRA>>PWM_COM)&3)==1 ) d<<=1;
#endif
   return d;
 } else return PWM_TOP_FIXED;       // TOP and frequency have a fixed value
}

// read the appropriate PWM timer output control register (the raw value of pwm
// duty cycle) and return it as per mille parts 0..1000 of TOP value
unsigned int pwm_get() {
#ifdef PWM_OCR
 return pwm_map(PWM_OCR, PWM_TOP, 1000);
#else
 return digitalRead(PWM_PIN);       // paranoia patch against screw-ups
#endif
}

// assuming v = 0..1000 per mille parts, set the PWM duty cycle
// https://en.wikipedia.org/wiki/Pulse-width_modulation
// the average voltage for PWM with duty cycle D equals D*Vmax + (1-D)*Vmin

unsigned int pwm_set(unsigned int v) {
 v %= 1001;
 switch(v) {
   case 0:
#ifdef PWM_PIN
     digitalWrite(PWM_PIN, LOW);    // bring PWM down (V=0)
#endif
     break;
   case 1000:
#ifdef PWM_PIN
     digitalWrite(PWM_PIN, HIGH);   // bring PWM down (V=100%)
#endif
     break;
   default:                         // do not alter the state of the pwm signal
#ifdef PWM_OCR                      // 0..1000 -> 0..PWM_TOP
     PWM_OCR = pwm_map(v, 1000, PWM_TOP);
#endif
 }
 return pwm_get();
}

#ifdef PWM_TIMER
// switch between 8,9,10-bit pwm modes while preserving (roughly) the duty cycle
unsigned char pwm_set_bits(unsigned char i) {
#if PWM_TIMER == 1                  // only for pins 9,10 (Timer 1)
 unsigned char b = PWM_BITS;        // current mode (1,2,3) and wgm state (0,1)
 // unsigned char w = ! ((PWM_TCCRB>>3)&1); // ! WGM12
 if(!(i &= 3)) i++;                 // 1,2,3 for 8-, 9-, and 10-bit modes
#if defined(__cplusplus) && defined(DEBUG) && DEBUG>1
 Serial.print(b); Serial.print(' ');
 Serial.print(PWM_TOP ); Serial.print(' ');
 Serial.print(PWM_TCCRA,HEX); Serial.print(' ');
 Serial.println(PWM_OCR);
#endif
 if(b!=i) {
   PWM_TCCRA = (PWM_TCCRA&(0xFF^3)) | i;
   PWM_OCR = b>i ? PWM_OCR>>(b-i) : PWM_OCR<<(i-b); // rescale duty cycle
#if defined(__cplusplus) && defined(DEBUG) && DEBUG>1
   Serial.print(PWM_BITS); Serial.print(' ');
   Serial.print(PWM_TOP ); Serial.print(' ');
   Serial.print(PWM_TCCRA,HEX); Serial.print(' ');
   Serial.println(PWM_OCR);
#endif
```

```
 }
#endif /* PWM_TIMER == 1 */
 return PWM_TOP_FIXED;                  // the upper value of the timer count
}

unsigned char pwm_toggle_wgm() {  // toggle fast <-> phase-correct modes
 unsigned int v = pwm_get();      // preserving the duty cycle
#if PWM_TIMER == 1
 PWM_TCCRB ^= (1<<3);             // WGM12  is bit 3 of TCCR1B
#else
 PWM_TCCRA ^= 2;                  // WGMx1, x=0,2 is bit 1 of TCCRxA
#endif
 pwm_set(v);                      // ATTN. small errors due to truncation
 return PWM_WGM_MODE;             // 0 phase correct, 1 fast pwm
}
#endif /* PWM_TIMER */

unsigned char pwm_toggle_fwgm() { // toggle fixed <-> variable frequency
 unsigned int v = pwm_get();      // preserving the duty cycle
 PWM_TCCRB ^= (1<<PWM_WGM_MSB);   // bit 4 (WGM13) or 3 (WGMx2) of TCCRxB
 pwm_set(v);                      // ATTN. small errors due to truncation
 return PWM_TOP_MODE;             // 1 or 0 for fixed or variable resp.
}

// set wave generation mode (WGMx) bits 0:3 and 0:2 for Timer x=1 and x=0,2
unsigned char pwm_set_wgm(unsigned char i) {
 i &= PWM_WGM_MASK;               // safety check
 PWM_TCCRA = (PWM_TCCRA&(0xFF^3)) | (i&3);
 PWM_TCCRB = (PWM_TCCRB&(0xFF^((PWM_WGM_MASK>>2)<<PWM_WGM_LSB))) | ((i>>2)<<PWM_WGM_LSB);
 return PWM_WGM;                  // current (new) WGM setting
}
```

D Colorimètre ~/Arduino/cmeter/cmeter.ino

```
/*
  Simple precise fixed-time-interval ADC sampler used as colorimeter;
  can sample at periods larger than 300 usec (safer 500 usec, see code)
  and thus has maximal theoretical sampling rate of 3kHz (more like 2kHz)

  TP: examine the relationship between the absorbance and concentration
  of a copper (II) sulfate CuSO4 solution (Beer's law) and measure
  the concentration of unknown copper (II) sulfate solution samples.
  The molar absorptivity of CuSO4 at lambda_max=635 nm is 2.81/M/cm.
  see more in http://dvoirah.ovh/etudes/Arduino/BQE-Arduino.pdf

  minimum parts required (in addition to Arduino UNO and computer):
    generic red LED (around 630 or 660 nm), 220 and 10k Ohm resistors
    photoresistor (3.1k Ohm exposed by red LED, closed at 0.35M Ohm)
    the LED-photoresistor distance on the breadbord is about 15mm
    use Arduino's 3.3V supply as VCC (more stable) to power the photoresistor
    use generic colorimeter 12.55x12.65x44.55 plastic cuvettes (with caps)
    for CuSO4 standard solutions of n/10 M with n=0,1,2,3,4,5 and 2-3 controls

  On interfacing with linux serial port, normally at /dev/ttyACM0, see
  https://playground.arduino.cc/Interfacing/LinuxTTY
  stty -F /dev/ttyACM0 cs8 115200 ignbrk -brkint -icrnl -imaxbel -opost -onlcr -isig -icanon -iexten -echo -
      echoe -echok -echoctl -echoke noflsh -ixon -crtscts
  stty -F /dev/ttyACM0 115200 cs8 cread clocal
  screen /dev/ttyACM0 115200

  When doing analog readings, especially with a 'noisy' board like the Arduino,
  we suggest two tricks to improve results. One is to use the 3.3V voltage pin
  as an analog reference, and the other is to take several readings in a row
  and average them. The 3.3V goes through a secondary filter/regulator stage
  and is less noisy to use it, connect 3.3V to AREF (next to digital GND).
  Several measurements can be averaged in time-independent or slow systems.

*/
#define _DEBUG_                    /* debugging, undefine to save space */
#include <EEPROM.h>                /* for nonvolatile memory access */

#define ADC_PIN A0                 /* pins A0..A5 for analog 10-bit ADC */
#define LED_PIN 10                 // digital pin for controlling LED source
// NB: use a PWM (~) pin to use analog fading effect, e.g. pin 8 is not suitable
```

```
// TODO: use PWM to control LED intensity
#define DIVISOR_R 10000            // 10K Ohm resistor in series with photo-R
#define STD_MAX 8                  // max number of calibration standards (>=2)

/* TODO: keep calibration coeffs and settings in nonvolatile memory */
double a1_coeff=.1;                // linear regression coefficients
double a0_coeff= 0;
double linreg_stderr=0;            // error of linear regression approximation
unsigned int  npts = 0;            // number of samples to measure
uint8_t apts = 2;                  // number of samples for averaging each point
unsigned long READ_PERIOD = 4000;  // 4000 us gives 250 Hz sampling rate

unsigned long lastRead = 0;        // time in us of the durrent point sequence
uint8_t hex_format=0;              // default raw integer output format
uint8_t std_num=0;                 // num of calibration data points (standards)
unsigned int std_C[STD_MAX], std_R[STD_MAX];

double Rval(unsigned int adc) {    // ADC data -> resistance of photo resistor
 return( DIVISOR_R / (1024 / ((double) adc) - 1) ); /* 10K / (1023/ADC - 1) */
}

double Cval(unsigned int adc) {    // ADC data -> concentration
 return(a1_coeff * ((double) adc) + a0_coeff);
}

double get_stderr() {              /* compute std error of linear regression */
 unsigned int *x=std_R, *y=std_C;  /* for the current calibration data set   */
 int k; double s=0,f;
 if(std_num>1) {                   /* check if there's enough data */
   for(k=std_num; k--; s+=f*f) f = Cval(*x++) - *y++;
   s /= (double) std_num;          /* unshifted normalized standard error   */
 }
 return(sqrt(s));                  /* sqrt( (sum_i (f(x_i) - y_i)^2) / N ) */
}

double linreg() {                  /* linear regression of current std data */
 unsigned int *x=std_R, *y=std_C;
 int k; double xx,_y,_x,xy;
 if(std_num>1) {                   /* check if there's enough standard data */
   for(k=std_num, _x  = _y = xx = xy = 0; k--; _x += *x++, _y += *y++) {
     xx += ((double) *x) * *x;                /* average of x^2 */
     xy += ((double) *x) * *y;                /* average of x*y */
   }
   a1_coeff = (xy*std_num - _x*_y) / (xx*std_num - _x*_x); /* slope of y(x) */
   a0_coeff = _y - _x * a1_coeff;
   a0_coeff/= (double) std_num;               /* shift of y(x) */
   linreg_stderr = get_stderr();              /* resulting std error */
  /* TODO: store calibration coeffs in nonvolatile memory */
 }
 return(linreg_stderr);
}

unsigned long min_val=0, max_val=1023;         /* 10-bit ADC value bracket */
void setup() {
  /* LED_BUILTIN is set to the correct LED digital pin for the board in use;
     it is 13 on the UNO, MEGA and ZERO, 6 on MKR1000. */
  pinMode(LED_BUILTIN, OUTPUT);
  digitalWrite(LED_BUILTIN, HIGH);  // turn the onboard LED on = "busy"
  /* set up faster serial communication (instead of default 9600bps) */
  Serial.begin(115200); // 115200 bps = 14400 bytes/sec, 70 usec/byte
  /* determine actual bracket of ADC values with light source on/off */
  pinMode(LED_PIN, OUTPUT);          // LED light-source controlling pin
  digitalWrite(LED_PIN,  LOW);       // make sure the LED is off
  delay(50);                         // wait for a 50 msec and measure
  max_val = analogRead(ADC_PIN);     // Vmax for maximum resistance (dark)
  digitalWrite(LED_PIN, HIGH);       // turn the LED on
  delay(200);                        // wait for a 200 msec and measure
  min_val = analogRead(ADC_PIN);     // Vmin for minimum resistance (max light)
  while (!Serial) {
    ; /* wait for serial port to connect (for native USB port only) */
  }
  // connect AREF to 3.3V and use that as VCC, because it is less noisy!
  analogReference(EXTERNAL);
  /* TODO: restore calibration coeffs and settings from nonvolatile memory */
  Serial.print("# real time colorimeter ");
  Serial.print(min_val);
```

```
  Serial.print("..");
  Serial.print(max_val);
#ifdef _DEBUG_
  Serial.print(" (debug)");
#endif
  Serial.println();
  digitalWrite(LED_BUILTIN, LOW);   // onboard LED off (end of setup)
}

void loop() {
 static unsigned int ncnt=0;
 static uint8_t acnt=0;
 static char key='Q';
 static unsigned long val=0,sval=0;
 char r,cflag=0;
  while(Serial.available()) { // read settings
    r = Serial.read();
    if(isDigit(r)) {
      val*=10;
      val+= r-'0';
    }
    else {
      switch(key) {  // keys that preceed numerical values
        case 'T':    // set sampling period (usec)
          if(val) READ_PERIOD = val;
          break;
        case 'D':    // concentration value for calibration data (standards)
    std_num=0; // reset calibration data storage
  case 'd':    // prepare new calibration point
    if(std_num >= STD_MAX) std_num--; // prevent storage overflow
    std_C[std_num] = val;             // enter calibration data y(x) >= 0
    std_R[std_num] = 0;               // clear x data (raw ADC values)
    cflag = 1;                        // calibration flag on
          break;
        case 'A':    // set number of averaged points
          if(val) apts = val;
          break;
        case 'N':    // set number of samples
          if(val) npts = val;
          /* TODO: store READ_PERIOD and/or npts to nonvolatile memory */
      }
      val=0;
      key=r;
      switch(key) {  // switches
        case 'h':    // set hex format
    hex_format =0;
        case 'H':    // toggle hex format (legacy)
          hex_format^=1;
          break;
        case 'c':    // concentration format toggle
          hex_format^=2;
          break;
        case 'r':    // resistance format toggle
          hex_format^=4;
          break;
        case 'C':    // (re)calibrate using currently stored data
          if(std_num>1) {
      Serial.print("# N=");    // number of points (>=2)
      Serial.print(std_num);
      Serial.print(" sigma="); // standard error
      Serial.print((unsigned int) floor(linreg()+.5));
      Serial.print(" a0 = ");  // shift
      Serial.print(a0_coeff);
      Serial.print(" a1 = ");  // slope
      Serial.print(a1_coeff);
      Serial.println();
#ifdef _DEBUG_
      for(acnt=0; acnt < std_num; acnt++) {
        Serial.print("# (V,C)_");
        Serial.print(acnt+1);
        Serial.print(" = (");
        Serial.print(std_R[acnt]);
        Serial.print(",");
        Serial.print(std_C[acnt]);
        Serial.print(") -> ");
        Serial.print((int) round(Cval(std_R[acnt])-std_C[acnt]) );
```

```
        Serial.println();
      }
#endif
    }
    else
      Serial.println("# Insufficient data for calibration");
        break;
  case 'X':    // toggle the LED on/off by bringing the voltage level HIGH/LOW
    digitalWrite(LED_PIN, !digitalRead(LED_PIN));
    delay(50);                      // wait for a 20 milliseconds
    break;
        case 'S':    // start sampling by rewinding npts
          ncnt=npts;
    acnt=apts;
    sval=0;
    if(!digitalRead(LED_PIN)) {     // make sure the LED is on
      digitalWrite(LED_PIN, HIGH);  // turn the LED on
      delay(50);                    // wait for a 50 milliseconds
    }
        default:
          Serial.print("# T=");
          Serial.print(READ_PERIOD);
          Serial.print(" usec, samples=");
          Serial.print(npts);
          Serial.print("/");
          Serial.print(apts);
          Serial.print(" @ ");
          Serial.print(1000000.0 / READ_PERIOD);
          Serial.print(" Hz, err=");
    Serial.print((unsigned int) floor(linreg_stderr+.5));
          Serial.println();
        case 'D':
        case 'd':
        case 'A':
        case 'N':
        case 'T':
        case 'Q':
          break;
      }
    }
  }
  if(ncnt) {
    /* acquire and display npts samples. ATTN: on 16MHz boards, the time
       resolution is 4 usec, and ncnt is overrun in approx 70 min */
    digitalWrite(LED_BUILTIN,HIGH);// onboard LED on = "busy"
    for(sval=ncnt=0, acnt=apts, lastRead=micros()+8; ncnt<npts; ncnt++){
      while(micros()<lastRead);     // wait for the next read
      lastRead += READ_PERIOD;
      val = analogRead(ADC_PIN);    // 1024 ADC takes about 100 usec (10kHz)
      acnt--; sval+=val;            // accumulate data
      if(!acnt) {                   // display every apts values only
  val = (unsigned long) (sval / apts); // average of apts measurements
  sval=0; acnt=apts;
  if(cflag) {
    std_R[std_num]=val;       // store raw value of x for calibration
    std_num++;              // add new point
  }
  switch(hex_format) {        // modify the raw value if necessary
    case 2:                   // concentration from linear regression
    case 3:                   // (may be negative when close to 0)
      val = (unsigned long) abs(round(Cval(val)));
      break;
    case 4:
    case 5:                   // resistance in Ohm
      val = (unsigned long) Rval(val);
      break;
  }
  if(hex_format&1)
    Serial.println(val,HEX); // at least 200 usec for 3 bytes
  else
    Serial.println(val);
  if(cflag) {
    cflag=0;
    ncnt=npts-1;              // disable any subsequent sampling
#ifdef _DEBUG_
    Serial.print("# C[");
```

```
    Serial.print(std_num);
    Serial.print("] = ");
    Serial.print(std_C[std_num-1]);
    Serial.println();
#endif
  }
      }
    }
    ncnt -= npts;
    digitalWrite(LED_BUILTIN, LOW);// onboard LED off = "idle"
  }
}
```

E Scanner le bus I²C avec ~/Arduino/scanI2C.h

Le bus I²C (dit *two-wire*) accepte plusieurs cartes, ou «slaves», à condition que leurs 7-bit adresses sont distinctes. Il est parfois très utile de découvrir tout les cartes qui sont actives sur ce bus. Pour ceci, on essaye de communiquer à 127 adresses dans la plage de `0x01` à `0x7F` et consulte le code d'erreur. Si ce code est 0 (sans erreur), on reporte l'adresse active détectée et, en option, l'interprète selon sa définition fournie alleurs.

```
// simple I2C bus scan for valid addresses
unsigned char scanI2C() {
 unsigned char err, i, n = 0;
#ifdef DEBUG
 Serial.print(F("I2C scan:"));
#endif
 for (i = 1; i < 0x7F; i++ ) {
     Wire.beginTransmission(i);
     // use the return value to see if device i responded to the query
     switch(err = Wire.endTransmission()) {
       case 4:
#ifdef DEBUG
   Serial.print(F(" ERROR 0x"));
   Serial.print(i,HEX);
#endif
       case 0:
#ifdef DEBUG
   Serial.print( (n ? F(", ") : F(" ")) );
   switch(i) { // display recognized devices
#ifdef MCP3424_ADDR
     case MCP3424_ADDR: Serial.print(F("MCP3224 "));
       break;
#endif
#ifdef DS3231_ADDR
     case DS3231_ADDR: Serial.print(F("DS3231 "));
       break;
#endif
#ifdef RTC_EEPROM_ADDR
     case RTC_EEPROM_ADDR: Serial.print(F("EEPROM "));
#endif
#ifdef _SPARKFUN_MMC5983MA_
     case I2C_ADDR: Serial.print(F("MMC5983 "));
#endif
#ifdef RTC_ADDR
     case RTC_ADDR: Serial.print(F("RTC "));
#endif
#ifdef MMC56X3_DEFAULT_ADDRESS
#ifdef MMC56X3_ADDR
     case MMC56X3_ADDR:
#else
     case MMC56X3_DEFAULT_ADDRESS:
#endif
       Serial.print(F("MMC56X3 ")); // 0x30
       break;
#endif // end for MMC56X3_DEFAULT_ADDRESS
     default:
       break;
   }
#endif // end for DEBUG, TODO: print space for all recognized addresses
#ifdef DEBUG
   Serial.print(F("0x"));
   if(i<0x10) Serial.print(F("0"));
   Serial.print(i,HEX);
#endif
   n++;
```

```
      default:
   break;
     }
   }
#ifdef DEBUG
 if(n) {
   Serial.print(F(" -> "));
   Serial.print((int) n);
   Serial.print(F(" device"));
   if(n>1) Serial.print(F("s"));
 }
 Serial.print(F("\n")); // avoid DOS-style \r\n that println implements
#endif
 return n;
}
```

Pour ne pas encombrer votre fichier principal avec extension `.ino` (le sketch), placez le code ci-dessus dans un fichier `scanI2C.h` (ou faites un link symbolique à ce fichier situé ailleurs) dans le même répertoire que votre `.ino` et engagez le avec la directive **`#include`**, voir l'example ci-dessous.

```
#define DEBUG 1
#define MCP3424_ADDR 0x6C // MCP3424 diff amp ADC (0x68 is reserved for the RTC)
#include <Wire.h>         // I2C bus library
#if defined(DEBUG) && DEBUG>0
#include "scanI2C.h"      // auxiliary code for I2C address scan
#endif

void setup() {
 Wire.begin();            // enable I2C communications (RTC,MCP)
#if defined(DEBUG) && DEBUG>0
 scanI2C();
#endif
}
```

F Méthode des moindres carrés. Régression linéaire

L'idée centrale de cette méthode (appelée *least squares fit* en anglais) est de trouver les valeurs de paramètres α, pour lesquelles la déviation moyenne quadratique entre la théorie et l'expérience

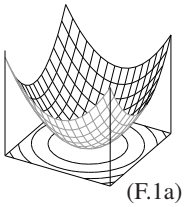
$$F(\alpha) = \sum_{i=1}^{N} \left(\frac{y_i - f(x_i, \alpha)}{\sigma_i} \right)^2$$

devient minimale. Ici les écarts types σ_i de chaque y_i jouent le rôle de «'poids» permettant de donner plus «d'importance» aux mesures plus précises. Dans le cas d'une régression *linéaire* où

$$y = f(x) = a_1 x + a_0, \quad \alpha = \{a_1, a_0\}$$

on a *deux* paramètres, et on cherche

$$\min_{a_1,a_0} F(a_1, a_0) = \min_{a_1,a_0} \sum_{i=1}^{N} \left(\frac{y_i - (a_1 x_i + a_0)}{\sigma_i} \right)^2 .$$



En introduisant les «moyennes pondérées»

$$[g] := \frac{1}{S} \sum_{i=1}^{N} \frac{g_i}{\sigma_i^2}, \quad \text{où} \quad S = \sum_{i=1}^{N} \frac{1}{\sigma_i^2} \quad \text{et} \quad g_i = x_i, y_i, x_i y_i, x_i^2 \tag{F.1a}$$

(devenant les moyennes $[g] = \bar{g}$ dans le cas simple où tous $\sigma_i \equiv \sigma$), on trouve les valeurs (voir le code `linreg` dans la sec. 4.1.3)

$$a_1 = \frac{[xy] - [x][y]}{[xx] - [x][x]}, \quad a_0 = [y] - [x]\, a_1 \tag{F.1b}$$

qui minimisent $F(a_1, a_0)$ avec leurs écarts types correspondant[40]

$$\sigma_{a_1}^2 = \frac{S^{-1}}{[xx] - [x][x]}, \quad \sigma_{a_0}^2 = \sigma_{a_1}^2 [xx]. \tag{F.1c}$$

Démonstration. Au minimum de $F(a_1, a_0)$, on a $dF = 0$. On obtient le système de deux équations linéaires de variables (a_1, a_0)

$$\left\{ \begin{array}{l} \dfrac{\partial F}{\partial a_1} = 0 \\ \dfrac{\partial F}{\partial a_0} = 0 \end{array} \right. \Rightarrow \left\{ \begin{array}{l} \displaystyle\sum_{i=1}^{N} x_i (y_i - (a_1 x_i + a_0)) \sigma_i^{-2} = 0 \\ \displaystyle\sum_{i=1}^{N} (y_i - (a_1 x_i + a_0)) \sigma_i^{-2} = 0 \end{array} \right. \Rightarrow \left\{ \begin{array}{l} [xy] - a_1 [xx] - a_0 [x] = 0 \\ [y] - a_1 [x] - a_0 = 0 \end{array} \right.$$

dont la solution est donnée par (F.1b). □

[40] Dans le cas simplifié $S^{-1} \approx \sigma^2 N^{-1}$, et par conséquence, les incertitudes Δa_1 et Δa_0 décroissent comme $1/\sqrt{N}$. On retrouve ainsi un résultat fondamental.

```
struct {float a0,a1;} linreg;       // linear regression coefficients (global)
float linreg_fun(float x) { linreg.a0 + linreg.a1*x; }
/* compute linear regression coefficients for n integer data set y(x) */
void linreg_define(unsigned int n, unsigned int *x, unsigned int *y) {
 int k; float xx,_y,_x,xy;
 if(n>1) {                          /* check if there's enough standard data */
   for(k=n, _x  = _y = xx = xy = 0; k--; _x += *x++, _y += *y++) {
     xx += ((float) *x) * *x;                /* average of x^2 */
     xy += ((float) *x) * *y;                /* average of x*y */
   }
   linreg.a1 = (xy*n - _x*_y) / (xx*n - _x*_x); /* slope of y(x) */
   linreg.a0 = _y - _x * linreg.a1;
   linreg.a0/= n;                              /* shift of y(x) */
 }
 else linreg.a1 = linreg.a0 = 0;
}
/* compute standard unshifted error (sigma) for n integer data set y-f(x) */
float linreg_stderr(unsigned int n, unsigned int *x, unsigned int *y) {
 int k; float s=0,f;
 if(n>1) {                          /* check if there's enough data */
   for(k=n; k--; s+=f*f) f = linreg_fun(*x++) - *y++;
   s /= n;                          /* unshifted normalized standard error   */
 }
 return(sqrt(s));                   /* sqrt( (sum_i (f(x_i) - y_i)^2) / N ) */
}
```

G Échantillonnement de gyroscopes du pendule du *Pohl* `gyrosampler.ino`

Voir les sections 2.4.3 (interruptions), 4.11, 4.11.3, et son appendice 4.11.3C.

```
/* -*- mode: c++; coding: utf-8; current-input-method: latin-1-prefix; ispell-local-dictionary: "english"; fill-
    column: 79; comment-column: 0; eval: (local-set-key "\C-c\C-\M-u" 'browse-url-firefox); -*-
  simple precise fixed-time-interval 2-channel gyro sampler (2x pololu L3GD20)
  see the gyro datasheet at https://www.pololu.com/file/0J731/L3GD20H.pdf and
  https://www.st.com/resource/en/application_note/an4505-l3gd20-3axis-digital-output-gyroscope-
      stmicroelectronics.pdf
  with QRE1113 based phase locking aka zero crossing detector aka "button"
  intended for the Pohl's pendulum student lab setup, see PHYWE ref. P2132705
  https://www.phywe.com/experiments-sets/university-experiments/forced-oscillations-pohl-s-pendulum_10989_12022/

  TODO: preserve PWM in sleep when driving the motor (and solenoid). Currently,
  we use SLEEP_MODE_PWR_DOWN while waiting for the interrupt to begin sampling
  at the fixed initial phase. The SLEEP_MODE_IDLE is the only way to continue
  PWM while sleeping. NB: in SLEEP_MODE_IDLE, you can use the power_*_disable()
  function from <avr/power.h> in order to further reduce the power consumption.

  DS 2023-08-20
*/
#define _PROGRAM_ "gyrosampler"  // program name
#define DEBUG 1                  // set to 0 to suppress extra msg
#define _VER_ "Aug 30 2024"      // program version
#define TIME_DEFAULT "12:30:00"
#if !defined(__DATE__)           // normally these are set by compiler
#define __DATE__ _VER_
#endif
#if !defined(__TIME__)
#define __TIME__ TIME_DEFAULT
#endif
#define F_CPU_SCL (16000000/F_CPU) // CPU frequency scaling (2 for 3.3V 8MHz)

/* instead of the usual speed of 9600 bps, a faster serial communication line,
   with speed of 115200 bps = 14400 bytes/sec, 70 usec/byte can be used */
#define SERIAL_BPS 115200
// ATTN: on Arduino Uno and other 328p-based boards, digital pins 2 and 3
// are the only ones that can be used for programmable external interrupts 0,1
#define BUTTON_PIN  2            // external interrupt trigger pin
/*
  NB: the role of the button can be also provided (in parallel or solely) by
  the infrared line sensor such as SparkFun ROB-09453 Breakout (Analog) for
  QRE1113 (https://www.sparkfun.com/datasheets/Robotics/QR_QRE1113.GR.pdf), see
  https://www.sparkfun.com/products/9453, which has a 10K resistor in series
  with the IR photo-transistor making a voltage divider for Vout so that
  without reflection, Vout is high, i.e., the photo-cell makes a low-end
  switch. Note that the SparkFun ROB-09454 Breakout (Digital) in has a 10nF
```

```
  capacitor instead and an additional 200 Ohm protecting the Vout line, see
  https://www.sparkfun.com/products/9454. This board can be converted to the
  analog (continuous) sensor by adding a 10K resistor between Vcc and Vout. The
  concrete results for UNO Vcc=5V are 0.5 to 0.7V for reflection off a white
  painted metal plate at approx 4mm distance, and 4.9V without any reflection
  (the sensor pointed in the open). These are sufficient to trigger HIGH/LOW
  logic events, see https://docs.arduino.cc/learn/microcontrollers/5v-3v3 and
  especially
  https://learn.sparkfun.com/tutorials/logic-levels/arduino-logic-levels and
  https://forum.arduino.cc/uploads/short-url/2rCNgicUzd2hDpinV0C9C8RpWdx.pdf .
  The schematics for the abovementioned breakout boards are available at
  http://cdn.sparkfun.com/datasheets/Sensors/Proximity/QRE1113-Digital-Breakout-v11.pdf.
  http://cdn.sparkfun.com/datasheets/Sensors/Infrared/QRE1113%20Line%20Sensor%20Breakout%20-%20Analog.pdf
*/
/*
  https://www.arduino.cc/reference/en/language/functions/analog-io/analogwrite/
  PWM on Uno, Nano, Mini: 490 Hz on pins 3, 9, 10, 11,  980 Hz on pins 5 and 6

  NB: PWM outputs on pins 5 and 6 may have higher-than-expected duty cycles
  because of interactions with functions millis() and delay(), which use the
  same internal timer. This will be noticed mostly on low duty cycle settings
  (e.g. 0-10) and may result in a value of 0 not fully turning off the output.

  so better choose 490 MHz because the sampler code relies heavily on millis()
  ATTN: since this may lead to perturbations on the external interrupt pin 2
  https://forum.arduino.cc/t/pwm-appears-to-be-conflicting-with-external-interrupts/117351
  do not use pin 3 but rather one of 9,10,11

*/
#define PWM_PIN 9        // PWM @490 Hz (away from DP2 and not used for SPI)

#define AMP_PIN A0       // analog input from the current sensor

// 20000us gives 50Hz sampling rate => 50 samples per 1/2-period of the pendulum
// 40000us gives 25Hz sampling rate => 50 samples per one period of the pendulum
unsigned long READ_PERIOD = 40000;
unsigned long lastRead=0;
unsigned int  npts=0;    // number of collected samples
unsigned char gyros=0;   // bits 0,1: gyroscope board presence and usage
#define GYRO_ALIGN -1    // should be -1 for opposite axis alignment

#include <limits.h>
#include <math.h>        // some math constants
#include <avr/sleep.h>   // AVR library for sleep modes
#include <avr/power.h>
#include <avr/wdt.h>     // WDT timer can be potentially used if thigs go south

#include <Wire.h>        // I2C bus, DS3231 0x68, mcp3424 (DFR0316) 0x6A
#include <L3G.h>         // L3GD20, L3GD20H, and L3G4200D gyros on Pololu boards
L3G gyro[2];             // prowide for two-gyroscope-board configuration

#include "printlf.h"     // awkward ad-hoc replacement for Serial.println()
#if defined(DEBUG)
// see /opt/arduino/hardware/arduino/avr/libraries/Wire/src/Wire.cpp
#if DEBUG>0 && defined(TwoWire_h)
#include "scanI2C.h"     // auxiliary code for I2C address scan
#endif
#include "freeRAM.h"     // free RAM information
#include "boardinfo.h"   // extract board information from chip signature
#endif
#ifdef AMP_PIN
#define ACS712_TYPE 5    // our chip is ACS712T ELC-05B 22121 (5A max)
#define AMP_SIGN -1
#include "acs712.h"      // ACS_712 current sensor IC
#define readVcc_nsamples 16
// NB: with 16 samples, the sampling accuracy is about 10mV but it is fast
#include "readVcc.h"     // internal voltage reference
#endif

#ifdef BUTTON_PIN
#include <time.h>        // modified C header file for avr-libc and AVR-GCC
// time in ms of the first and last trigger events
volatile time_t        button_start=0, button_timer=0;
// number of button trigger instances
volatile unsigned int  button_count=0;
```

```
#define BUTTON_INT digitalPinToInterrupt(BUTTON_PIN)
// cf https://forum.arduino.cc/t/does-the-serial-communication-interfere-with-usage-of-external-interrupts
    /1085101
// boxcar wait interval (ms) during which subsequent interrupts will be ignored
// as spurious to reduce jitter
#define BUTTON_GAP 2
#define BUTTON_LED LED_BUILTIN
// ISR for handling interrupt triggers arising from associated button switch
void button_int_handler() {
#ifdef BUTTON_GAP
 time_t t = millis();             // bail out if the events get too close
 if (button_count && t-button_timer < BUTTON_GAP) return;
 button_timer = t;                // new event is outside the gap
#else
 button_timer = millis();         // time of the event and number of events
#endif
 if(!button_count++) button_start=button_timer;
#ifdef BUTTON_LED                 // toggle LED indicator
 digitalWrite(BUTTON_LED, !digitalRead(BUTTON_LED));
#endif // button led
}
#endif // button pin

#ifdef PWM_PIN
// https://www.arduino.cc/reference/en/language/functions/analog-io/analogwrite/
// analogWrite values go from 0 (always off) to 255 (always on)
// (the PWM mode which is pre-cofigured on the UNO for all pins is 8 bit)
// analogWrite has nothing to do with the analog pins and analogRead
// the pin will generate a steady rectangular wave of the specified duty cycle
// until the next analogWrite(), digitalRead(), or digitalWrite() on this pin

// https://passionelectronique.fr/pwm-arduino/
// https://passionelectronique.fr/wp-content/uploads/Atmel-ATmega328P-datasheet.pdf?pseSrc=pgPwmArduino
// The ATmega168P/328P chip has three PWM timers, controlling 6 PWM outputs.
// Timer   Timer   Arduino
// compare output -> pin
// OCR0A   OC0A       6    Fast PWM Mode 8-bit @976,56 Hz, 255 steps
// OCR0B   OC0B       5    Fast PWM Mode 8-bit @976,56 Hz, 255 steps
// OCR1A   OC1A       9    Phase Correct PWM Mode 8-bit @490,20 Hz, 510 steps
// OCR1B   OC1B      10    Phase Correct PWM Mode 8-bit @490,20 Hz, 510 steps
// OCR2A   OC2A      11    Phase Correct PWM Mode 8-bit @490,20 Hz, 510 steps
// OCR2B   OC2B       3    Phase Correct PWM Mode 8-bit @490,20 Hz, 510 steps

// NB: by default, on the UNO, the pwm is configured so that

#define PWM_BOTTOM 0x0000

// DP5 and 6 use the 8-bit Timer 0 and can only work with 8-bit resolution.
// They are configured by default for Fast (simple) PWM with frequency
// f = 16 MHz / 64 / 256 = 976,5625 (the simple duity cycle of 255+1 steps)
// (here notice that 16*2^20/64/256 = 1024)

#if PWM_PIN==5          // Fast PWM Mode 8-bit @976,56 Hz
#define PWM_OCR OCR0A   // Timer 0 register A
#define PWM_OC  OC0A
#define PWM_COM COM0A0  // lsb of the 2 compare output bits in TCCR0A
#endif
#if PWM_PIN==6          // Fast PWM Mode 8-bit @976,56 Hz
#define PWM_OCR OCR0B   // Timer 0 register B
#define PWM_OC  OC0B
#define PWM_COM COM0B0  // lsb of the 2 compare output bits in TCCR0A
#endif
#if PWM_PIN==5 || PWM_PIN==6
#define PWM_MAX 0x00FF  // MAXimum of the Timer 0 counter
#define PWM_TOP PWM_MAX // highest value in the Timer 0 count sequence, 8-bit
#define PWM_CYCLE 256   // number of steps in the period
#define PWM_TCCRA TCCR0A
#define PWM_TCCRB TCCR0B
#endif
/*
  DP3 and 11 use Timer 2 (8-bit) and can only work with 8-bit resolution; DP9
  and 10 use Timer 1 (16-bit) and support more precise 9- and 10-bit pwm modes
  all DP9, 10, 11, and 3 are configured for Phase Correct PWM with frequency

  f = 16 MHz / 64 / 510 = 490,196 (510 = 2*255 steps in the duty interval)
```

```
  NB: The phase-correct PWM mode and the phase-and-frequency-correct PWM mode
  are based on a dual-slope operation with counter going repeatedly from BOTTOM
  (0) to TOP and back to BOTTOM.  Due to the symmetry of the dual-slope PWM
  modes, the latter are preferable in motor control applications.
*/
#if PWM_PIN==9         // Phase Correct PWM Mode 8-bit @490,20 Hz
#define PWM_OCR OCR1A  // Timer 1 register for channel A
#define PWM_OC  OC1A
#define PWM_COM COM1A0 // lsb of the 2 compare output bits in TCCR1A
#endif
#if PWM_PIN==10        // Phase Correct PWM Mode 8-bit @490,20 Hz
#define PWM_OCR OCR1B  // Timer 1 register for channel B
#define PWM_OC  OC1B
#define PWM_COM COM1B0 // lsb of the 2 compare output bits in TCCR1A
#endif
#if PWM_PIN==9 || PWM_PIN==10
#define PWM_MAX 0xFFFF // MAXimum of the Timer 1 counter (decimal 65535)
#define PWM_TOP 0x00FF // upper value in the Timer 1 count sequence, 8-bit
// depending on the operation mode, Timer 1 TOP equals 0x00FF (default 8-bit
// PWM), 0x01FF, 0x03FF, or the value stored in the OCR1A or ICR1 register.
#define PWM_CYCLE 510  // number of steps in the default fcPWM period 2*255
#define PWM_TCCRA TCCR1A
#define PWM_TCCRB TCCR1B
#define PWM_TCCRC TCCR1C
// Atmel ATmega328P Waveform Generation Modes (WGM)
// bits _BV(WGMx3)!_BV(WGMx2)  and _BV(WGMx1)|_BV(WGMx0) for Timer 1
#define PWM_WGM ((PWM_TCCRB&0x18)>>1 | (PWM_TCCRA&3))
const char *pwm_wgm[] = { // datasheet, 15-5 p.109
  "-",     "pc~8",  "pc~9", "pc~10", "CTC-O", "f~8", "f~9", "f~10",
  "pfc~I", "pfc~O", "pc~I", "pc~O",  "CTC-I",   "?", "f~I", "f~O" };
#else
// bits _BV(WGMx2)  and _BV(WGMx1)|_BV(WGMx0) for Timers 0 and 2
#define PWM_WGM ((PWM_TCCRB&0x08)>>1 | (PWM_TCCRA&3))
const char *pwm_wgm[] = { // datasheet, 14-8 p.86
  "-",  "pc~",  "CTC", "f~", "?", "pc~O", "?", "f~O"};
#endif

#if PWM_PIN==11         // Phase Correct PWM Mode 8-bit @490,20 Hz
#define PWM_OCR OCR2A   // Timer 2 8-bit register A
#define PWM_OC  OC2A
#define PWM_COM COM2A0  // lsb of the 2 compare output bits in TCCR2A
#endif
#if PWM_PIN==3          // Phase Correct PWM Mode 8-bit @490,20 Hz
#define PWM_OCR OCR2B   // Timer 2 8-bit register A
#define PWM_OC  OC2B
#define PWM_COM COM2B0  // lsb of the 2 compare output bits in TCCR2A
#endif
#if PWM_PIN==3 || PWM_PIN==11
#define PWM_MAX 0x00FF  // MAXimum of the Timer 2 counter
#define PWM_TOP PWM_MAX // highest value in the Timer 0 count sequence, 8-bit
#define PWM_CYCLE 510   // default number of steps in the period
#define PWM_TCCRA TCCR2A
#define PWM_TCCRB TCCR2B
const unsigned char pwm_divisors[] = {0,0,3,5,6,7,8,10}; // Timer 2
#else
const unsigned char pwm_divisors[] = {0,0,3,6,8,10};     // Timer 0 and 1
#endif
/*
    Arduino timer scaling:
    common prescaler: 1, 2, 4, 8, 16, 32, 64, 128, 256
                      0  1  2  3   4   5   6    7    8   (value of CLKPR & 0xF)
    timer0 or timer1: 0, 1, 8, 64, 256, 1024
                      0  1  2   3    4     5   (value of TCCR0B or TCCR1B & 7)
    timer2:           0, 1, 8, 32,  64,  128, 256, 1024
                      0  1  2   3    4     5    6,    7  (value of TCCR2B & 7)
*/

#define PWM_ON   (PWM_TCCRA>>PWM_COM)&0x3
#define PWM_DIV  (1<<pwm_divisors[PWM_TCCRB & 7])
#define PWM_FREQ ((float) F_CPU / (1<<(CLKPR & 0xF)) / PWM_DIV / PWM_CYCLE)

// per mille (Latin meaning 'in each thousand') indicates parts per thousand
// aka promille and permille, the symbol being o/oo (the usual percentage o/o)

// read the appropriate PWM timer control register (the raw value of pwm)
// and return as per mille parts 0..1000 of PWM_TOP
```

```
unsigned int pwm_get() {
#ifdef PWM_OCR
 return map(PWM_OCR, 0, PWM_TOP, 0, 1000);
#else
// analogRead(PWM_PIN) with values of 0..1023 is silly: it cannot read DP's !
 return digitalRead(PWM_PIN); // paranoid for screw-ups
#endif
}
// assuming v=0..1000 per mille parts, convert percents to the 0..PWM_TOP range
// and set the PWM signal on the PWM_PIN
unsigned int pwm_set(unsigned int v) {
 v = map(v, 0, 1000, 0, PWM_TOP);
 analogWrite(PWM_PIN, v);
 return pwm_get();
}
// https://en.wikipedia.org/wiki/Pulse-width_modulation
// the average voltage for PWM with duty cycle D equals D*Vmax + (1-D)*Vmin
// compare this code to the map function available in arduino lib
#endif

#ifdef AMP_PIN
/*
  The DC current in the electromagnet of the Eddy current brake or dumper
  or retarder, also known induction, Faraday, or electromagnetic brake.
  French: ralentisseur Ãl'lectromagnÃl'tique / frein de Foucault
*/
#define AMP_SAMPLES_MAX 64          // TODO: use integers for A and dA ?!
float A=0, dA=0;                    // running mean value and std error
unsigned int nA=0;                  // number of collected samples
int amp_sample(int i) {             // acquire i samples x_k of AMP_PIN
 float d; int x;
 while(i--) {
   x = analogRead(AMP_PIN)-512;     // load x with k-th value x_k, k=nA+1
#if defined(AMP_SIGN) && AMP_SIGN<0
   x*=AMP_SIGN;                     // legacy for wrongly connected acs712
#endif
   // Serial.println(x);
   if(nA++) {           // for k>1
     d=x-A;             // x_k - A_{k-1}
     A+=d/nA;           // A_k = A_{k-1} + (x_k-A_{k-1})/k
     dA+=d*(x-A);       // Q_k = Q_{k-1} + (x_k-A_{k-1})*(x_k-A_k)
   }
   else {               // for k=1 (since A_0 = Q_0 = 0)
     A=x;               // A_1 = x_1
     dA=0;              // Q_1 = 0
   }
 }
 return (int) A; // ACS712_mA(A);
}

#endif

// NB: the F() moves strings to flash memory instead of wasting SRAM
void print_info(unsigned char typ, unsigned char flags, unsigned char pcnt) {
 unsigned char i;
#if defined(DEBUG) && DEBUG>=0
 if(typ&1) {
   Serial.print(F("# "));
   Serial.print(F(_PROGRAM_));
#if defined(ARDUINO_ARCH_AVR)
   Serial.print(F(" AVR:"));        // compiled for avr architecture (16-bit)
#else
   Serial.print(F(" "));
#endif // end arch
   Serial.print(F(_VER_));  Serial.print(F(", MCU "));
   Serial.print(BOARD_INFO,HEX);
   Serial.print(F("@")); Serial.print(F_CPU/1000000); Serial.print(F("MHz "));
   Serial.print(BOARD_TYPE);        // Arduino board name
   Serial.print(F(" SRAM:")); Serial.print(freeRAM()); // free data RAM available
   printlf();                       // end first header line
#if DEBUG>1
   Serial.print(F("# compile date "));
   Serial.print(F(__DATE__));  Serial.print(F(", ")); Serial.print(F(__TIME__));
   Serial.print(F(" from ")); Serial.print(F(__FILE__));
   printlf();
#endif // end DEBUG>1
```

```
 }
#endif // end DEBUG>=0
#if defined(TwoWire_h) && defined(DEBUG) && DEBUG>0
 if(typ&2) {                          // if I2C library is loaded
   Serial.print(F("# "));
   scanI2C();                          // scan and report all devices on I2C
 }
#endif // end I2C
#ifdef DEBUG
 // if asked for info on gyro[i] and it is available
 // i=0 for 0x6a (front, pendulum axis, large amplitude)
 // i=1 for 0x6b (rear, driving force, small ampllitude)
 for(i=0; i<=1; i++) {
   if(typ&(4<<i) && gyros&(1<<i)) {
     Serial.print(F("# gyro[")); Serial.print(i);  Serial.print(F("] "));
     // device id from register 0x0F and from L3G identification
     Serial.print(gyro[i].readReg(L3G::WHO_AM_I), HEX);
     Serial.print(F("(")); Serial.print((int) gyro[i].getDeviceType());
     // I2C address  0x6a+i (in L3G.cpp D20_SA0_LOW_ADDRESS+i)
     Serial.print(F(")@0x")); Serial.print(0x6a+(i?L3G::sa0_high:L3G::sa0_low),HEX);
     Serial.print(F(" t="));
     // temperature data (-1LSB/deg, 8 bit) from register 0x26
     // NB: the L3GD20 includes an on-chip temperature sensor that is suitable
     // for differential temperature measurements: updated @1Hz, the register
     // provides a (negative?) temperature relative to an uncalibrated /
     // unspecified offset to account for temperature variations over time.
     Serial.print( (int) gyro[i].readReg(L3G::OUT_TEMP) );
     // active axes
     Serial.print(F(" R1,4="));
     Serial.print(gyro[i].readReg(L3G::CTRL_REG1),HEX);
     // FS1-FS0 bits define full scale (FS) in dps
     Serial.print(F(","));
     Serial.print(gyro[i].readReg(L3G::CTRL_REG4),HEX);
     printlf();
   }
 }
#endif // end gyro info
#if defined(DEBUG) && defined(BUTTON_PIN)
 if(typ&0x10) {
   Serial.print(F("# trig="));       // current trigger state (0=equilibrium)
   Serial.print(digitalRead(BUTTON_PIN));
#ifdef BUTTON_GAP
   Serial.print(F(" gate="));        // boxcar gate to suppress jittering
   Serial.print(BUTTON_GAP);
   Serial.print(F("ms"));
#endif
   printlf();
 }
#endif
 if(typ&0x20) {                       // current sampling parameter values
   Serial.print(F("# time="));
   Serial.print(READ_PERIOD);
   Serial.print(F(" usec, "));
   Serial.print(npts);
   Serial.print(F(" samples at "));
   Serial.print(1000000.0 / READ_PERIOD);
   Serial.print(F(" Hz"));
   if(pcnt) {
     Serial.print(F(" or "));
     Serial.print((float) pcnt / 2);
     Serial.print(F(" periods"));
   }
   Serial.print(F(" flags "));
   Serial.print(flags,BIN);
#ifdef PWM_PIN
   Serial.print(F(" V="));           // voltage mV in the motor driver circuit
   Serial.print(pwm_get(), DEC);
#endif
#ifdef AMP_PIN
   /*
     The DC current in the electromagnet of the Eddy current brake /
     dumper / retarder, aka induction, Faraday, or electromagnetic
     brake. French: ralentisseur Ã‍l'lectromagnÃ‍l'tique / frein de Foucault
   */
   Serial.print(F(" J="));           // DC current in the dumper electromagnet
   Serial.print(ACS712_mA(A));
```

```
   Serial.print(F("("));
   Serial.print((nA>0?ACS712_mA(sqrtf(dA/nA)):0),DEC);
   Serial.print(F(")mA"));          // Serial.print(nA);
#ifdef readVcc_nsamples
   Serial.print(F(" Vcc="));
   Serial.print(Vcc);               // current value of Vcc
#endif
#endif
   printlf();
 }
 if(typ&0x40) {
   Serial.print(F("#BEGIN flag=")); Serial.print(flags,BIN);
   for(i=0; i<=1; i++)              // loop on gyros i=0,1
     if(gyros&(1<<i)) {
       Serial.print(F(" s")); Serial.print(i);  Serial.print(F("="));
       Serial.print(gyro[i].readReg(L3G::CTRL_REG4),HEX);
     }
#ifdef GYRO_ALIGN
   Serial.print(F(" sgn="));        // gyroscope alignment correction
   Serial.print(GYRO_ALIGN);
#endif
#ifdef PWM_PIN
   Serial.print(F(" V="));          // driver motor voltage mV
   Serial.print(pwm_get(), DEC);
#endif
#ifdef AMP_PIN
   Serial.print(F(" J="));          // current in the electromagnetic dumper
   Serial.print(ACS712_mA(A));
#endif
 }
 if(typ&0x80) {
   Serial.print(F("# scl="));       // overall timer frequency prescaler
   Serial.print( 1<<(CLKPR & 0xF), DEC);
   // " @" F_CPU / (1<<(CLKPR & 0xF)) / 1000000 "MHz"
#ifdef PWM_PIN                      // pwm setup information
   Serial.print(F(" DP"));
   Serial.print(PWM_PIN);           // digital pin used
#ifdef PWM_COM                      // examine pwm compare output mode bits
   Serial.print((i=PWM_ON)?"~":"=");
   if(!i)                           // if pwm is off, show pin status 0/1
     Serial.print(digitalRead(PWM_PIN));
#endif
#ifdef PWM_WGM
   Serial.print(F(" "));            // Wave Generation Mode
   Serial.print(pwm_wgm[i = PWM_WGM]);
#if defined(DEBUG) && DEBUG>0
   Serial.print(F("["));            // content of the WGM bits (aka WG=)
   Serial.print( i, BIN);
   Serial.print(F("]"));
#endif
#endif
   Serial.print(F(" CS="));         // timer clock select (divisor) bits
   i=PWM_TCCRB&0x07;                // _BV(CSx2)|_BV(CSx1)|_BV(CSx0)
   Serial.print(1<<pwm_divisors[i],DEC);
#if defined(DEBUG) && DEBUG>0
   Serial.print(F("["));            // content of the CS bits
   Serial.print(i,BIN);
   Serial.print(F("]"));
#endif
   if(i) {
     Serial.print(F(" @"));         // frequency
     Serial.print(PWM_FREQ,0);
     Serial.print(F("Hz"));
     }
   else Serial.print(F(" off"));    // pwm timer is off
   Serial.print(F(" "));            // x2 for phase (and frequency) correct WGM
   if(*pwm_wgm[PWM_WGM]=='p') Serial.print(F("2x"));
   Serial.print(PWM_OCR, DEC);      // duty cycle [clock ticks]
   Serial.print(F("/"));
   Serial.print(PWM_CYCLE);         // full pwm pulse duration (aka period)
#endif
   printlf();
 }
}

void setup() {
```

```
 unsigned char i;
 pinMode(LED_BUILTIN, OUTPUT);
 digitalWrite(LED_BUILTIN, HIGH);   // turn the onboard LED on = "busy"
#ifdef PWM_PIN
 pinMode(PWM_PIN, OUTPUT);          // activate pwm on pin PWM_PIN
 digitalWrite(PWM_PIN, LOW);        // turn the pwm pin off (V=0)
#endif
#ifdef BUTTON_PIN
#ifdef BUTTON_LED
#if BUTTON_LED != LED_BUILTIN
 pinMode(BUTTON_LED, OUTPUT);       // special LED indicator of the trigger
#endif
 digitalWrite(BUTTON_LED, LOW);     // flush LED indicator of the trigger
#endif
 /* declare and set interrupt pins: on UNO, pins 2,3 give interrupt 0,1  */
 pinMode(BUTTON_PIN, INPUT);        // pulldown push button or switch
 attachInterrupt(BUTTON_INT, button_int_handler, FALLING);
 // the mode can be LOW (whenever the pin is low), CHANGE, RISING, FALLING
 // https://www.arduino.cc/reference/en/language/functions/external-interrupts/attachinterrupt/
 // NB: FALLING triggers when the sensor signal goes from HIGH to LOW, which
 // means when the reflecting obstacle comes into sensor's sight. In the Pohl's
 // pendulum setup with the sensor placed at the equilibrium position, this
 // avoids an artifact interrupt occuring when the pendulum is pulled out of
 // its equilibrium manually and then launched.
#endif
 Serial.begin(SERIAL_BPS);          // open serial communication port
 while(!Serial);                    // wait for it to connect (native USB only)
 print_info(1,0,0);
#if defined(TwoWire_h)              // if I2C library is loaded
 Wire.begin();                      // enable I2C communications (RTC,MCP)
 print_info(2,0,0);
#endif // end I2C
 // i=0 for 0x6a (front, pendulum axis, large amplitude)
 // i=1 for 0x6b (rear, driving force, small ampllitude)
 for(i=0; i<=1; i++) {
   /* The I2C Slave ADdress (SAD) associated to the L3GD20H is binary 110101x.
      The SDO/SA0 pin can be used to modify the less significant bit (LSB) x of
      SAD: if it is connected to Vcc, then x=1 (SAD = 1101011 or 0x6b) else if it
      is connected to the ground then x=0 (SAD = 1101010 or 0x6a). This permits
      to connect and address two different gyroscopes to the same I2C bus. */
   // is the board D20H or D20 ? it returns 0xD7, so D20H
   // D20H_WHO_ID     0xD7
   // D20_WHO_ID      0xD4
   // L3G4200D_WHO_ID 0xD3
   if(!gyro[i].init(L3G::device_auto, (i?L3G::sa0_high:L3G::sa0_low) )) continue;
   gyros |= 1<<i; // availability flag
   gyro[i].enableDefault();
   /* Estimate the maximal angular velocity amplitude of the Pohl's pendulum:
      ----------------------------------------------------------------------
      The protractor circle of the apparatus is graduated in somewhat arbitrary
      units (au), such that 10 au correspond to 1/5th of the full circle, i.e.,
      2pi/5 = 10 units => pi/25 = 7.2 deg = 1 unit and pi/5 = 36 deg = 5 units.
      The maximal deviation angle A is (restricted to) about +/-20 au = 144
      deg.  The period of free oscillations is approximately 1.8 sec, and the
      proper (resonant) circular frequency w0 is, therefore, about
      2pi/1.8. This means that the maximal amplitude of the angular velocity of
      the pendulum equals A w0 = 144 2pi/1.8 = 160 pi = 502.65 dps (deg/sec). */

   // The default L3G gyro's full scale is set to +/-250 dps (FS1-FS0=0x00 in
   // register 0x23, see sec. 7.5, pp. 39-40 of the L3GD20 datasheet),
   // the call below selects the 500 dps scale (0x10).
   if(!i) gyro[0].writeReg(L3G::CTRL_REG4,0x10);
   // z-axis data only in register 0x20, see sec. 7.2 on p.36 of the datasheet
   gyro[i].writeReg(L3G::CTRL_REG1, gyro[i].readReg(L3G::CTRL_REG1)&0xFC);
   print_info(4<<i,0,0);
 } // end of loop on boards 0,1
#ifdef readVcc_nsamples
 Vcc=readVcc();                     // check the actual value of Vcc
#endif
 print_info(0x10,0,0);
 digitalWrite(LED_BUILTIN, LOW);    // turn the onboard LED off = "done"
}

void loop() {
 static unsigned int  ncnt=0;       // number of samples
 static unsigned char pcnt=0;       // number of full half-periods
```

```
 static unsigned char flags=0;     // flag bits: 0 hex format, 1 trigger wait
 static unsigned char key=';';     // initilize the stale key at first pass
 static long val=0;
 // because r is used as index in gyro[r], it should rather be nonnegative!
 // https://stackoverflow.com/questions/18502257/array-subscript-has-type-char-wchar-subscripts
 unsigned char r;
 // This CLI assumes no line endings present, just plain stream of characters.
 // Integer number arguments refer to the last non-number key (legacy mode).
 // Consequently, commands with number arguments should terminate with a
 // command that takes no such arguments, for example `i' or `;' (big
 // headache...)
 while(Serial.available()) {       // read settings and commands
    r = Serial.read();
    if(isDigit(r)) {               // swallow the possible numerical value
      val*=10;
      val+= r-'0';
    }
    else {
      switch(key) {  // handle the PREVIOUS key preceeding a numerical value
        case 'T':    // sampling interval (usec)
          if(val) READ_PERIOD = val;
          break;
        case 'N':    // number of samples
          if(val) npts=val;
    break;
  case 'P':    // number of full half-periods to span (0..255)
          if(val) pcnt = val;
    break;
#ifdef PWM_PIN
  case 'V':    // set PWM external voltage control (0..1023)
    switch(val) {
      case 0:
        digitalWrite(PWM_PIN, LOW);    // bring PWM down (V=0)
        break;
      case 1000:
        digitalWrite(PWM_PIN, HIGH);   // bring PWM down (V=100%)
        break;
      default:
        pwm_set(val);
    }
    print_info(0x80,flags,pcnt);       // detailed info on pwm
    break;
#endif
      }
      val=0;
      key=r;          // === LAST READ byte becomes CURRENT key ===
      switch(key) {  // handle switches
  case 'Z':    // reset trigger count and display avg trigger time (period)
#ifdef BUTTON_PIN
          Serial.print(F("# "));
          Serial.print(button_count);
          Serial.print(F(" counts"));
    if(button_count>1) {
      Serial.print(F(" with T="));
      Serial.print( floor((float) (button_timer-button_start) / --button_count), 0);
      Serial.print(F("ms"));
    }
          printlf();
    button_count=0;
#endif
    break;
        case 'H':    // toggle hex format
          flags^=1;
          break;
        case 'W':    // toggle waiting for trigger
          flags^=2;
          break;
        case 'F':    // toggle sampling of second gyro, if available
          flags^=(gyros&2)<<1;
          break;
  case 'I':    // full startup header and current parameter values
    print_info(0x3F,flags,pcnt);
    break;
        case 'S':    // reset sample counter to start sampling
          ncnt=npts;
    /* The sampling time is the time interval between successive samples,
```

```
       also called the sampling interval or the sampling period, and
       denoted T. The sampling rate is the number of samples per
       second. It is the reciprocal of the sampling time, i.e. 1/T, also
       called the sampling frequency, and denoted Fs */
        default:     // general information on sampling parameters
#ifdef AMP_PIN
  case 'J':    // resample current in the electromagnetic dumper cirquit
    nA=0;
#ifdef readVcc_nsamples
    Vcc=readVcc();  // NB: the Vcc sampling may take a bit of time
#endif
    amp_sample(AMP_SAMPLES_MAX);
#endif
  case 'i':    // brief header and current parameter values
    print_info(0x20,flags,pcnt);
        case 'N':    // NB: commands that accept numerical values are postpended
  case 'P':    // and CANNOT be used to display information at this point
        case 'T':
#ifdef BUTTON_PIN
  case 'V':
#endif
        case 'Q':
        case ';':    // line ending, helpful for commands with numerical args
          break;
      }
    }
  }
  /* time resolution on 16MHz boards (such as 5V UNO) is 4 usec (1/16*64), and
     at this min sampling time ncnt will be overrun in approx 70 min, which is
     an ample amount of time ... */
  if(ncnt) {          // proceed to acquire and display npts samples
    print_info(0x40,flags,pcnt);
    Serial.flush();  // clear all output (i.e., wait) before going to sleep
    button_count=0;  // reset trigger count (aka number of 1/2 periods)
    //#if defined(DEBUG) && DEBUG > 2
    //#endif
    if(flags&2) {    // postpone sampling until the first trigger event
      interrupts();  // re-enable interrupts (if/after disabled by noInterrupts()
      set_sleep_mode(SLEEP_MODE_PWR_DOWN);   // full sleep mode
      sleep_enable();                    // (re)enable sleep mode
      sleep_cpu();                       // ***** ACTIVATE SLEEP *****
      sleep_disable();                   // on comeback: disable the sleep mode
      power_all_enable();                // re-enable peripherals, is it needed?
    }
    printlf();
    for(ncnt=0, lastRead=micros()+8; ncnt<npts && ! (pcnt && button_count>pcnt); ncnt++){
      while(micros()<lastRead);
      lastRead += READ_PERIOD;
      for(r=0; r<=((flags>>2)&1); r++) {
  // take a reading from the gyro number r=0,1 and store the value
  gyro[r].read();
        /*
        NB: these are raw 16-bit values obtained by concatenating the 8-bit
        high and low gyro data registers.  They can be converted to units of
        dps (degrees per second) using the conversion factors specified in the
        datasheet for your particular device and full scale setting (gain).
        Specifically, the L3GD20H datasheet on p.10 gives conversion factors
        for Sensitivity (So) 8.75, 17.50, and 70.00 mdps/digit (aka mdps/LSB)
        corresponding to full scale (FS) ranges of +/- 245, 500, and 2000 dps,
        respectively.  So, for example: if an L3GD20H with its default FS
        setting of 245 gives a reading of 345, this corresponds to 345 * 8.75 =
        3020 mdps = 3.02 dps, see examples of the L3G library
        https://github.com/pololu/l3g-arduino.
        */
  val = (int) gyro[r].g.z;
#ifdef GYRO_ALIGN
  if(r) val*=GYRO_ALIGN;
#endif
  // NB: at 115200 bps it takes at least 200 usec for 3 bytes
  // HEX is more economic but less readable, especially for negaives
  // TODO: use HEX as binary format (max 5 bytes)
  if(flags&1)
    if(val>0) Serial.print(val,HEX);   // positive hexadecimal
    else {
      Serial.print(F("-"));            // negative hexadecimal
      Serial.print(-val,HEX);
```

```
    }
  else Serial.print(val);              // decimal
  Serial.print(F(" "));
      }
      Serial.print(button_count); printlf(); // number of periods
    }
    Serial.print(F("#END n=")); Serial.print(ncnt); printlf();
    if(pcnt) ncnt=0; else ncnt-=npts;
  }
}
```

H Un schéma, un prototype, un PCB, un objet

Voir la fig. 10

TO DO: Logiciel pcb-rnd

TO DO: Logiciel BRL-CAD de design pour l'impression 3D

I Gestion des projets (AC)

Objectifs : Acquérir les méthodes et les outils fondamentaux de la gestion de projet pour piloter un projet avec succès et se doter d'une boîte à outils. A l'issue de la formation, les apprenants seront capables de :

1. S'approprier les notions clés de la gestion de projet ;
2. Identifier le rôle et les responsabilités des pilotes ;
3. Identifier les étapes clés d'un projet et le processus de mise en œuvre ;
4. Conduire un projet en mettant en œuvre une méthode et des outils opérationnels ;
5. Identifier les ressources pour la réussite d'un projet ;
6. Débloquer les situations difficiles dans la gestion de projet.

Compétences et savoirs Les étudiants devront ensuite valider les compétences et savoirs suivants :

1. Appliquer un QQOQCP ; (méthode couramment utilisée en entreprise à appliquer ici sur un «petit projet»)
2. Définir un plan d'actions ; (décomposition en tâches et sous-tâches, attribution des rôles, identifications des ressources, risques et parades. . .)
3. Organiser ce plan d'actions dans le temps et définir des livrables (diagramme de Gantt) ;

J Organisation de projets, année 2020

L'UO Arduino est offert à ULCO depuis 2019 et se distingue par son mode de travail. Il s'agit principalement des *projets* autour de micro-contrôleur Arduino que les étudiants réalisent en *quasi-autonomie* en groupes de 3-4, et, au moins en partie, en dehors de ses 20h heures «officielles». Cet approche engage non seulement les connaissances en informatique, électronique, physique, mathématique, chimie, et biologie—en fonction de la nature du projet, mais les aspects logistiques, relationnels et organisationnels, et elle demande une grande motivation de la part des étudiants. Alors tous ceux qui cherchent une note «facile» pour comforter leur moyenne, tous ceux qui anticipent les cours-TD-TP classiques où on peut dormir tranquillement, copier les notes et comptes rendus de son voisin, et attendre que l'enseignant effectue les manips du TP feront mieux d'abstenir et de libérer plus de nos ressources aux autres étudiants.

Dans ce contexte, le rôle principale des enseignants devient l'accompagnement. Après une courte introduction (4h), nous tiendrons 4 séances (16h au maximum) de «permanence» pour permettre aux étudiants d'avancer leur projets et de résoudre tous problèmes. Les groupes se rencontrent en dehors de ses heures «officielles» et restent en contact par mél avec leur enseignants «mentors». Vu la nature des projets et le niveau des étudiants très variées, l'énseignant ne peut pas s'occupper de 3 (au maximum 4) groupes à la fois. Cette année nos intervenants «mentors» seront :

- Dimitri Sadovskií (DS), professeur à ULCO, département de physique, sadovski@univ-littoral.fr
- Robin Bocquet (RB), professeur à ULCO, département de physique, robin.bocquet@univ-littoral.fr
- Arnaud Cuisset (AC), professeur à ULCO, département de physique, arnaud.cuisset@univ-littoral.fr
- Marc-Alexandre CARPENTIER (MAC), étudiant en L1 math Calais, marcal.carpentier@gmail.com
  après son IUT info, il a enseigné à l'IUT et a créé son propre entreprise : l'appli qu'il a développé (une base des données pour un site des rencontres) a déjà 150000 utilisateurs inscrits et cela lui paye une rente..
- Sébastien MENIERE (SM), étudiant en L3 math Calais, sebastien.meniere@gmail.com
  Sébastien a eu son propre entreprise d'électronique il est expert en Arduino et son projet de l'an passe était excellent (générateur des codes aléatoires Arduino pour porte monnaie bitcoin)

Pour tout commandes de matériel et composantes, des petites repartions ou d'autres aides (par exemple soudures), on peut s'adresser à

- Wilfried Montagnier (WM), soutien technique de TP physique Calais, wilfried.montagnier@univ-littoral.fr

Les 34 étudiants de l'année 2025 formiront 8 groupes de projet de 4 étudiants en moyen par groupe. Il est, bien sur, envisageable qu'ils ne sont pas tous sérieux, mais on peut, quand même, anticiper d'avoir 4 à 6 projets à suivre réellement. Comme l'équipement, nous donnons 1-2 coffrets Arduino par groupe/projet. Certains composants nécessaires se trouvent dans nos kits. Pour les composants spécifiques aux projets, il sera indispensable de choisir son projet au plus vite afin de pouvoir les commander et recevoir au temps. Tout demandes sont à adresser au sérvice TP physique (WM).

On peut commencer soit 24/1 soit 31/1 avec une séance d'introduction (4h) où tout le monde vient ensemble. La séance est découpée en deux parties : 2h dans une amphi (de 8h30 à 10h15) pour discuter des questions générales (RB) et «Gestion de projets» (AC) suivis par la présentation des projets possible (RB+DS+ ?) de 10h30 à 12h30. On choisit son projet. On prépare son choix en regardant la sec. 4, l'internet, en discutant par mél avec nous, et suite à la présentation des projets et aux échanges dans l'amphi. On forme les groupes (binômes, trinômes, 4 max) pour chaque projet. Les groupes choisissent leur «mentors». A ce point les aspects relationnels et la capacité de l'orientation rapide jouent un rôle important : il faut choisir ceux avec qui on peut travailler et surtout fuir ceux qui cherchent à ne rien faire. Par la suite, on fait 2h dans les classes info en deux groupes séparés (RB et DS) pour réaliser certains manips éducatifs dans la sec. 2. On a l'occasion de tester la cohésion des groupes.

Dès que les étudiants se départagent en groupes de projet, SM et MAC prendront chaque 1-2 groupes des étudiants motivés. RB et DS garderons les autres 4 groupes. SM et MAC organisent leur travail comme ils peuvent. Ils sont présents sur site. RB et DS tiendrons les permanences les vendredis. Ainsi pour ceux qui auront besoin de plus d'introduction à Arduino, nous ferons une séance d'entraînement de plus (voir sec. 2.1.1), mais, peut être, déjà pas pour tout le monde. On décidera après la formation des groupes, choix de projets, etc.

A la fin du semestre, ou même après les examens, on se revoit *tous* en amphi pour une *soutenance publique* (4h), où les mentors forment le jury pour donner la note à chaque groupe/projet, et où des autres enseignants seront invités pour donner leur avis.

Acronymes

Informatique

ASCII American Standard Code for Information Interchange (codage de lettres alphabetiques à 7 bits)
CLI Command Line Interface
IRQ Interrupt ReQuest (interruption matérielle ou demade d'interruption)
ISR Interrupt Service Routine, ou Interrupt Handler
LSB Least Significant Bit
MCU Micro Controller Unit
μC Micro Controller
μCU voir MCU

Électronique

ADC Analog to Digital Converter (or simply A/D)
AC Analog Comparator
AC Alternative Current
AVR la famille de μCs de Atmel (dépuis 1997 désigne le coeur du processeur «Alf et Vegard's RISC processor»)
BJT Bipolar Junction Transistor (transistor bipolaire)
DAC Digital to Analog Converter
DAD Diode Array Detector, cf PDA
DC Direct Current
DEL diode électroluminescente (LED)
DP digital pin (broche numérique)
EEPROM Electrically Erasable Programmable ROM
E/S entrée/sortie (d'un cirquit, une broche, voir I/O)
ESD Electro-Static Discharge
GND GrouND (masse, point de référence, cf. V_{EE} et V_{SS})
GSM Global System for Mobile communication
I^{2}C Inter-Integrated Circuit (denommé aussi I2C ou IIC), un protocole informatique et bus d'échange de données
IC Integrated Circuit
I/O Input/Output (fr. E/S)
LDR Light-Dependent Resistor (connue également comme photorésistance ou photocellule)
LED Light Emitting Diode
MOS-FET Metal-Oxide-Semiconductor Field-Effect Transistor (transistor à effet de champ à grille isolée)
PCB Printed Circuit Board, circuit imprimé (en éléctronique)
PDA Photo Diode Array, un réseau linéaire de photodiodes, connu aussi comme DAD
PMMA Poly Methyl MethAcrylate (connue également comme «verre acrylique» ou encore *Plexiglas* ™)
PWM Pulse-Width Modulation
RAM Random-Access Memory
RISC Reduced Instruction Set Computer
ROM Read-Only Memory
RTC Real TIme Clock
SCL Serial CLock line
SDA Serial DAta line
SPI Serial Peripheral Interface (un bus de données série synchrone)
SRAM Static RAM
TTL Transistor-Transistor Logic (en électronique)
USB Universal Serial Bus
V_{CC} Common Collector Voltage (Collector to Collector, alimentation principale d'un cirquit de BJT)
V_{DD} Drain-to-Drain Voltage (dans un cirquit de BJT, ou la tension de fonctionnement d'un cirquit numérique)
V_{EE} Emitter-to-Emitter Voltage (tension d'alimentation négative d'un circuit de BJT, équivalent à la GND pour les applications numériques)
V_{OUT} Output voltage (tension de sortie)
V_{SS} Source-to-Source Voltage (même que V_{EE}, mais dans un circuit de MOS-FET)

Éducation

PHYWE Physikalische Werkstätten depuis 1918, la société allemende à Göttingen, qui met au point, produit, fournit, et installe l'équipement scientifique pour les collèges et lycées, ainsi que pour la recherche scientifique.

TP Travaux partiques

ULCO Université du Littoral – Côte d'Opale (Dunkerque, France)

UO Unité d'ouverture (licence ULCO)

Index

Arduino IDE
  installation, 3
  sketch, 3
ATmega328p, 2

broche, *voir* pin

CLI, 39
comparateur, 16, 39
courants de *Foucault*, 30
C++
  types de données
    char, 7
    int, 7
    unsigned, 7

DEL, *voir* LED
diode
  clamping, 7
  Schottky, 7
  Zener, 7

état logique, *voir* niveau logique

GPL, 35

HX711, 21

I^2C, 63
impédance d'entrée, 16
interruption
  IRQ, 8, 38
  ISR, 8

LED, 5
loi
  Beer-Lambert, 24
  d'Ohm, 5
  des mailles, 19

MCP3424, 21
MCU, 1
μCU, *voir* MCU
μC, *voir* MCU
méthode
  des «ellipses», 32
  des moindres carrées, 64

niveau logique, 2, 16, 38

optocoupleur, 17

PDA, 27
pendule
  de *Pohl*, 30
  Foucault, 29
  sensibilité de gyroscopes, 33
pin
  analogique, 8
  Ai, $i = 1 \ldots 5$, *voir* analogique
  digital, *voir* numérique
  DP$_i$, $i = 1 \ldots 13$, *voir* numérique
  numérique, 4
Pont de Wheatstone, 19
PWM, 16, 17, 38, 39

  facteur de forme, *voir* rapport cyclique
  rapport cyclique, 39

QRE1113, 12

registre
  ACSR, 16, 39, 46, 47, 51
  DIDR1, 47
  EICRA, 11, 41, 42, 47
  EIFR, 47, 51
  EIMSK, 47, 51
  TCNT2, 11, 43, 47
  TIFR2, 11, 43, 47
  TIMSK2, 11, 47
résonance
  courbes de, 32

string, 7
survoltage, 7

Table des matières

Title 1

1 Introduction 1

2 Initiation à ARDUINO 2
2.1 Programmation de ARDUINO . . . 2
2.1.1 Installer et utiliser le logiciel ARDUINO (IDE) . . . 3
2.1.2 Brancher de la carte ARDUINO à l'ordinateur . . . 3
2.1.3 Programme Blink . . . 4
2.1.4 Programme BlinkSOS . . . 4
2.1.5 Montage d'une LED . . . 4
2.1.6 Remarque : Connexion de LED's . . . 5
2.1.7 Remarque : Niveaux logiques de ARDUINO . . . 5
2.1.8 Capteur de lumière comme interrupteur . . . 6
2.2 Montage à 4 LED's . . . 6
2.2.1 Remarque : Le courant maximal qu'on peut tirer des pins digitaux de ARDUINO en mode OUTPUT . . . 7
2.2.2 Remarque : protection des pins digitaux de ARDUINO en mode INPUT . . . 7
2.2.3 Interaction sur le port série . . . 7
Sortir l'information : le texte (string) . . . 7
Entrer l'information . . . 7
2.2.4 Montage du potentiomètre et le convertisseur A/N (ADC) . . . 8
Programme final (avec option potentiomètre) . . . 8
2.3 Alimentation . . . 8
2.4 Interruptions . . . 8
2.4.1 Anti-rebond . . . 10
2.4.2 Réduire la consommation . . . 12
2.4.3 Detecteurs d'obstacles et suiveurs de ligne . . . 12
2.4.4 Utilisation d'une horloge RTC pour reveiller ARDUINO . . . 13
2.4.5 TP : Le temps de réponse et la fréquence d'échantillonage. Photorésistance vs photodiode . . . 15
2.4.6 IRQ comparateur . . . 16
2.5 ADC sampling . . . 16
2.6 Simple DAC . . . 17
2.7 Commande des appareils à forte puissance . . . 17
2.7.1 Régler la vitesse d'un moteur DC conventionnel . . . 18

3 Les capteurs 18
3.1 La thermistance . . . 18
Calibration d'une thermistance : . . . 20
3.2 La photo-résistance . . . 20
3.3 Capteur de force et poids . . . 20
3.4 Amplificateurs opérationnels . . . 21
Réalisation : . . . 22
3.5 Accéléromètre et gyroscope . . . 22

4 Projets UO ARDUINO 2025 24
Traitement des données . . . 24
4.1 Colorimétrie . . . 24
4.1.1 Absorption de la lumière, loi de *Beer-Lambert* . . . 24
4.1.2 Régression linéaire . . . 25
4.1.3 Colorimètre avec ARDUINO . . . 25
Matériel : . . . 25
Le principe : . . . 25
Le code et l'interface : . . . 25
4.1.4 TP Mesures de concentration . . . 26
Objectif : . . . 26
Matériel : . . . 26
Mode opératoire : . . . 26
4.1.5 TP Mesures de la constante cinétique . . . 26
Objectif : . . . 26
Exposé : . . . 26
Mode opératoire : . . . 26
Exploitation des mesures : . . . 27
4.1.6 Développement . . . 27
4.2 Spectrophotomètre (DS) . . . 27
4.3 Mesure de la chaleur massique de l'eau et de la chaleur latente de fusion de la glace (DS) . . . 27

4.4 Expérience Clément–Désormes (DS) . . . 27
4.5 Pèse-ruches à distance . . . 28
4.6 Capteur de vitesse de vent par sonde Pitot (DS) . . . 28
4.7 Capteurs environnementaux . . . 28
4.8 Viscosimètre avec un capteur inductif de déplacement/vitesse (DS) . . . 28
4.9 Expérience de Rüchardt . . . 29
4.10 Magnétomètre . . . 29
4.11 Projets autour de accéléromètre et gyroscope . . . 29
4.11.1 Rotation de la Terre . . . 30
4.11.2 Indicateur de freinage style F1 . . . 30
4.11.3 Pendule de Pohl . . . 30
TP pendule de *Pohl* . . . 31
Appendice 4.11.3C : logiciels `gyrosampler` . . . 32
Appendice 4.11.3A : calibrer la plage de sensibilité de gyroscopes . . . 33
Appendice 4.11.3B : montage des capteurs . . . 34
4.12 PCR et qPCR thermal cycler (DS) . . . 34
4.13 Capteur déplacement/vitesse (DS) . . . 34
4.14 Anémomètre et girouette à effet Doppler (R. Bocquet) . . . 34
4.15 Cinémomètre à GPS (R. Bocquet) . . . 35
4.16 Optimisation de la consommation d'un micro-contrôleur ATtiny 85 (R. Bocquet) . . . 35
4.17 Centrale météorologique – 2 groupes de travail (R. Bocquet) . . . 35

A Branchement de LEDs et barographe `~/Arduino/BlinkSOS/BlinkSOS.ino` 36

B Échantillonnage de ADC `~/Arduino/adcsampler/adcsampler.ino` 37

C Exploration de PWM et IRQ : générateur des signaux créneau avec fréquencemètre 38

D Colorimètre `~/Arduino/cmeter/cmeter.ino` 59

E Scanner le bus I²C avec `~/Arduino/scanI2C.h` 63

F Méthode des moindres carrés. Régression linéaire 64

G Échantillonnement de gyroscopes du pendule du *Pohl* `gyrosampler.ino` 65

H Un schéma, un prototype, un PCB, un objet 75

I Gestion des projets (AC) 75
Objectifs : . . . 75
Compétences et savoirs . . . 75

J Organisation de projets, année 2020 75

Acronymes 77

Index 78

Table des figures

accompagnés par les broches de sortie d'alimentation (3.3 V stabilisée, 5 V, et la masse GND) dont la VIN permet également entrer une alimentation extérieure de 5 V DC à la carte. 1figure.caption.3

2 Caractéristiques courant-tension de diodes électroluminescentes (LEDs) dont la cathode est repérée par la patte plus courte (a), et leur montage de base biaisé par une résistance de type *pullup* (b) et *pulldown* (c). Les résistances sont marguées par des bandes colorées (d). 5
3 Niveaux logiques . . . 5
4 Bouton poussoir SW de type «high-end» (a)-(b) ou «low-end» (c) montée avec une resistance R de type «pull down» or «pull up», respectivement, pour commander une interruption IRQ externe sur le pin DP2 ou 3. Le circuit anti-rebond et une photorésistance LDR optionnels sont en gris. . . . 9
5 Boutons-poussoirs en action (circuit de la fig. 4c sans filtre passe bas) : trois appuis rapides (a) et profile du signal de fermeture (b) du bouton DIP 6×6 mm «à cliquet» qu'on trouve couramment sur des circuits imprimés ; les rebondissements du signal d'un bouton poissoir classique (c). . . . 10
6 La réponse (a) d'une LDR et (c) de la photodiode SFH 203 à une LED (⌀ 5 mm, rouge) alimentée par le μCU ARDUINO UNO en mode PWM (pin DP9) à 7.63 Hz avec rapport cyclique de 20% et 4 kHz avec 50%, respectivement ; les traces jaune (CH1) et bleue (CH2) correspondent à la tension aux bornes de la LED, et celle aux bornes de la résistance biais de la LDR (10 k) ou de la PD (300 k) dans le circuit (b). . . . 15
7 Le teste de la sec. 2.4.5 modifié (a) par le guide de lumière (b) (PMMA ⌀ 3 mm, longueur ≈20 mm) placé entre la LED et la photodiode SFH 203 avec R_{biais} de 470 K dans le circuit de la fig. 6b ; la réponse (c) de la photodiode à 4 kHz. . . . 16

8 Montage pour tester l'échantillonneur rapide en teps réel (a), la tension passée sur le pin A0 (b), les données (c). 17
9 Les possibilités différentes de commander l'alimentation d'une charge à forte puissanse (dit «load») : (a) interrupteur côte alimentation (dit «high end power switch») permettant des mesures de tension (pin A_0 de μCU) aux bornes d'une charge ohmique ; (b) le même, mais avec une charge inductive (moteur, bobine d'un relais) et une diode (dit «flywheel» ou «snubbing») protégeant l'interrupteur au moments de déconnexion ; (c) interrupteur déconnectant la masse (dit «low end») ; (d) le cirquit (c) dont l'interrupteur est remplacé par un transistor npn commandé par le pin digital DP$_i$ de μCU (0 pour ouvrir et $+V_{DD}$ pour enclencher). Notez, que la tension +V dans (d) peut parvenir d'une source d'alimentation dédiée à forte consommation avec sa masse *connectée* à celle de μCU, et peut être différente (supérieure !) de (à) la $V_{CC} = V_{DD}$ operée par le μCU. Au contraire, le curquit (e) utilise un optocoupleur et un N-MOS-FET pour commander les appareils «externes» dont V_{IN} et la masse GND sont *isolées* de l'μCU. 17
10 Contrôleur de la vitesse d'un moteur DC conventionnel (i.e., avec un collecteur et balais ou charbons) sans charge mécanique importante 18
11 La thermistance (a), ses paramètres (b), et son montage dans un diviseur de tension (c) et sur le pont de Wheatstone (d). 19
12 Calibration d'une thermistance CTN 20
13 La photo-résistance (a) ; cablage en «diviseur de tension» avec V_{CC} de 3.3 V (b) ; mesure de R_{min} (c) ; comme capteur digital (d). . . . 20
14 Montage de 4 cellules de force à 3 fils et leur calibration 21
15 Amplificateur de mesure : (a) partie tampon, (b) résultats de sa réalisation (voir texte), et (c) partie differentielle. 22
16 Montage du colorimètre avec $CuSO_4$ dans une cuvette. 25
17 Etudes colorimétriques : (a) courbe de calibration de colorimètre ; (b) observations en temps réel. 26
18 Rotation de la Terre et du gyroscope. Le vecteur $\vec{\Omega}$ représente la vitesse de rotation de la Terre ; le pesenteur $\vec{g}$ pointe vers le centre ; (x, y, z) marquent les axes du gyroscope. (a) La hémisphère nord avec deux positions successives du gyroscope sur l'équateur ; z est aligné avec $-\vec{g}$ et pointe «verticalement vers le haut», y donne le ouest, tandis que $\vec{\Omega}$ et x (orienté N) sont pointés vers nous. (b) Le gyro à la lattitude $\theta = 50.7^\circ$ avec la même orientation des axes. (c) L'orientation de $\vec{\Omega}$ par rapport à l'orbite de la Terre autour du Soleil (axe horizontal). 30
19 Pendule de *Pohl* : la présentation et l'analyse des mesures avec l'utilitaire `gyrosampler.gp` (scripte GNUPLOT) des oscillations (a) libres, (b) forcées, et (c) forcées sous forme de l'ellipse de Lissajous, dit «mode x-y». Les graphes représentent les données de gyroscopes `0x6a` et `0x6b` sur les vitesses $\mathrm{d}\theta/\mathrm{d}t$ du pendule et de la force avec les amplitudes respectives sont ωA et $\omega\, a$. 33
20 Pendule de *Pohl : montage des capteurs à l'avant (a) et à l'arrière (b) du dispositif* PHYWE P2132705, voir l'appendice 4.11.3B. . . . 33